Chapter

8

Boolean Algebra and Computer Logic

CHAPTER OBJECTIVES

After studying this chapter, you will be able to:

- Determine whether a given mathematical structure is a Boolean algebra.
- Prove properties about Boolean algebras.
- Understand what it means for an isomorphism function to preserve the effects of a binary operation or other property.
- Draw a logic network to represent a Boolean expression.
- Write a Boolean expression to represent a logic network.
- Write the truth function for a Boolean expression or logic network.
- Write a Boolean expression in canonical sum-of-products form for a given truth function.
- Use NAND and NOR gates as well as AND, OR, and NOT gates to build logic networks.
- Write a truth function from a description of a logical control device.
- Simplify Boolean expressions and logic networks using Karnaugh maps.
- Simplify Boolean expressions and logic networks using the Quine–McCluskey method.

You have been hired by Rats R Us to build the control logic for the production facilities for a new anticancer chemical compound being tested on rats. The control logic must manage the opening and closing of two valves, A and B, downstream of the mixing vat. Valve A is to open whenever the pressure in the vat exceeds 50 psi (pounds per square inch) and the salinity of the mixture exceeds 45 g/L (grams per liter). Valve *B* is to open whenever valve A is closed and the temperature exceeds 53°C and the acidity falls below 7.0 pH (lower pH values mean more acidity).

Question: **How many and what type of logic gates will be needed in the circuit?**

The answer to this electronics problem lies, surprisingly, in a branch of mathematics developed around 1850 by George Boole, an English mathematician. Boole was interested in developing rules of "algebra" for logical thinking, similar to the rules of algebra for numerical thinking. Derided at the time as useless, if harmless, Boole's work is the foundation for the electronics found in computers today.

In Section 8.1 we define Boolean algebra as a mathematical model of both propositional logic and set theory. The definition requires every Boolean algebra to have certain properties, from which many additional properties can be derived. This section also discusses what it means for two instances of a Boolean algebra to be isomorphic.

Section 8.2 establishes a relationship between the Boolean algebra structure and the wiring diagrams for the electronic circuits in computers, calculators, industrial control devices, telephone systems, and so forth. Indeed, we will see that truth functions, expressions made up of variables and the operations of Boolean algebra, and these wiring diagrams are all related. As a result, we can effectively pass from one formulation to another and still preserve characteristic behavior with respect to truth values. We will also find that we can simplify wiring diagrams by using properties of Boolean algebras. In Section 8.3 we will look at two other procedures for simplifying wiring diagrams.

SECTION 8.1 BOOLEAN ALGEBRA STRUCTURE

Let us revisit the wffs of propositional logic and associate with them a certain type of function. Suppose a propositional wff P has n statement letters. Then each row of the truth table for that wff associates a value of T or F with an n-tuple of T–F values. The entire truth table defines a function f such that f: $\{T, F\}^n \rightarrow \{T, F\}$. The function associated with a tautology maps $\{T, F\}^n \rightarrow \{T\}$, and the function associated with a contradiction maps $\{T, F\}^n \rightarrow \{F\}$.

EXAMPLE 1 We've seen this idea before. From Example 31 in Chapter 5, the function f: $\{T, F\}^2 \rightarrow \{T, F\}$ for the wff $A \vee B'$ is given by the following truth table.

A	B	B'	$A \vee B'$
T	T	F	T
T	F	T	T
F	T	F	F
F	F	T	T

Here $f(T, F) = T$ and $f(F, T) = F$.

Suppose we agree, for any propositional wff P with n statement letters, to let the symbol P denote not only the wff but also the corresponding function defined by the truth table. If P and Q are equivalent wffs, then they have the same truth tables and therefore define the same function. Then we can write $P = Q$ rather than $P \Leftrightarrow Q$. This simply confirms that a given function has multiple names, although a given wff defines a unique function.

With this agreement, the short list of tautological equivalences from Section 1.1 can be written as follows, where $\vee$ and $\wedge$ denote disjunction and conjunction, respectively, A' denotes the negation of a statement A, 0 stands for any contradiction, and 1 stands for any tautology:

1a. $A \vee B = B \vee A$	lb. $A \wedge B = B \wedge A$	(commutative properties)
2a. $(A \vee B) \vee C = A \vee (B \vee C)$	2b. $(A \wedge B) \wedge C = A \wedge (B \wedge C)$	(associative properties)
3a. $A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$	3b. $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$	(distributive properties)
4a. $A \vee 0 = A$	4b. $A \wedge 1 = A$	(identity properties)
5a. $A \vee A' = 1$	5b. $A \wedge A' = 0$	(complement properties)

Switching gears a bit, in Section 4.1 we studied set identities among the subsets of a set S (the elements of $\wp(S)$). We found the following list of set identities, where $\cup$ and $\cap$ denote the union and intersection of sets, respectively, A' is the complement of a set A, and $\varnothing$ is the empty set:

1a. $A \cup B = B \cup A$	lb. $A \cap B = B \cap A$	(commutative properties)
2a. $(A \cup B) \cup C = A \cup (B \cup C)$	2b. $(A \cap B) \cap C = A \cap (B \cap C)$	(associative properties)
3a. $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	3b. $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$	(distributive properties)
4a. $A \cup \varnothing = A$	4b. $A \cap S = A$	(identity properties)
5a. $A \cup A' = S$	5b. $A \cap A' = \varnothing$	(complement properties)

These two lists of properties are similar. The disjunction of statements and the union of sets seem to play the same roles in their respective environments. So do the conjunction of statements and the intersection of sets. A contradiction seems to correspond to the empty set and a tautology to S. What should we make of this resemblance?

Models or Abstractions

We seem to have found two different examples—propositional logic and set theory—that share some common properties. One of the hallmarks of scientific thought is to look for patterns or similarities among various observed phenomena. Are these similarities manifestations of some underlying general principle? Can the principle itself be identified and studied? Could this research shed light on the behavior of various instances of this principle? Sometimes, as seems to be the case with propositional logic and set theory, similar mathematical properties or behavior can be seen in different contexts. A mathematical structure is a formal model that serves to embody or explain this commonality, just as in physics the law of gravity is a formal model of why apples fall, the ocean has tides, and the planets revolve around the sun.

Mathematical principles are models or abstractions intended to capture properties that may be common to different instances or manifestations. These principles are sometimes expressed as *mathematical structures*—abstract sets of objects, together with operations on or relationships among those objects that obey certain rules. (This concept may give you a clue about why this book is titled as it is.)

We can liken a mathematical structure to a human skeleton. We can think of the skeleton as the basic structure of the human body. People may be thin or fat, short or tall, black or white, and so on, but stripped down to skeletons they all look pretty much alike. Although the outward appearances differ, the inward structure, the shape and arrangement of the bones, is the same. Similarly, mathematical

structures represent the underlying sameness in situations that may appear outwardly different.

It appears reasonable to abstract the common properties (tautological equivalences and set identities) for propositional wffs and set theory. Thus we will soon define a mathematical structure called a Boolean algebra that incorporates these properties. First, however, we note that modeling or abstracting is not an entirely new idea to us.

1. We used predicate logic to model reasoning and formally defined an interpretation as a specific instance of predicate logic (Section 1.3).
2. We defined the abstract ideas of partial ordering and equivalence relation, and considered a number of specific instances that could be modeled as posets or sets on which an equivalence relation is defined (Section 5.1).
3. We noted that graph and tree structures can model a great variety of instances (Sections 6.1 and 6.2).

Boolean algebra is just another model or abstraction for which we already have two instances.

Definition and Properties

Let us characterize formally the similarities between propositional logic and set theory. In each case we are talking about items from a set: a set of wffs or a set of subsets of a set S. In each case we have two binary operations and one unary operation on the members of the set: disjunction/conjunction/negation or union/intersection/complementation. In each case there are two distinguished elements of the set: 0/1 or $\varnothing/S$. Finally, there are the 10 properties that hold in each case. Whenever all these features are present, we say that we have a Boolean algebra.

DEFINITION **BOOLEAN ALGEBRA**

A **Boolean algebra** is a set B on which are defined two binary operations $+$ and $\cdot$ and one unary operation $'$ and in which there are two distinct elements 0 and 1 such that the following properties hold for all $x, y, z \in B$:

1a. $x + y = y + x$	1b. $x \cdot y = y \cdot x$	(commutative properties)
2a. $(x + y) + z = x + (y + z)$	2b. $(x \cdot y) \cdot z = x \cdot (y \cdot z)$	(associative properties)
3a. $x + (y \cdot z) = (x + y) \cdot (x + z)$	3b. $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$	(distributive properties)
4a. $x + 0 = x$	4b. $x \cdot 1 = x$	(identity properties)
5a. $x + x' = 1$	5b. $x \cdot x' = 0$	(complement properties)

What, then, is the Boolean algebra structure? It is a formalization that abstracts, or models, the two cases we have considered (and perhaps others as well). There is a subtle philosophical distinction between the formalization itself, the *idea* of the Boolean algebra structure, and any instance of the formalization, such as these two cases. Nevertheless, we will often use the term *Boolean algebra* to describe both the idea and its occurrences. This usage should not be confusing. We often have a mental idea ("chair," for example), and whenever we encounter a

concrete example of the idea, we also call it by our word for the idea (this object is a "chair").

The formalization helps us focus on the essential features common to all examples of Boolean algebras, and we can use these features—these facts from the definition of a Boolean algebra—to prove other facts about Boolean algebras. Then these new facts, once proved in general, hold in any particular instance of a Boolean algebra. To use our analogy, if we ascertain that in a typical human skeleton, "the thighbone is connected to the kneebone," then we don't need to reconfirm the fact in every person we meet.

We denote a Boolean algebra by $[B, +, \cdot, ', 0, 1]$.

EXAMPLE 2 Let $B = \{0, 1\}$ (the set of integers 0 and 1) and define binary operations $+$ and $\cdot$ on B by $x + y = \max(x, y), x \cdot y = \min(x, y)$. Then we can illustrate the operations of $+$ and $\cdot$ by the following tables.

$+$	0	1
0	0	1
1	1	1

$\cdot$	0	1
0	0	0
1	0	1

A unary operation $'$ can be defined by means of a table, as follows, instead of by a verbal description.

$'$	
0	1
1	0

Thus $0' = 1$and $1' = 0$. Then $[B, +, \cdot, ', 0, 1]$ is a Boolean algebra. We can verify the 10 properties by checking all possible cases. Thus, for property 2b, the associativity of $\cdot$, we show that

$$\begin{aligned}
(0 \cdot 0) \cdot 0 &= 0 \cdot (0 \cdot 0) = 0 \\
(0 \cdot 0) \cdot 1 &= 0 \cdot (0 \cdot 1) = 0 \\
(0 \cdot 1) \cdot 0 &= 0 \cdot (1 \cdot 0) = 0 \\
(0 \cdot 1) \cdot 1 &= 0 \cdot (1 \cdot 1) = 0 \\
(1 \cdot 0) \cdot 0 &= 1 \cdot (0 \cdot 0) = 0 \\
(1 \cdot 0) \cdot 1 &= 1 \cdot (0 \cdot 1) = 0 \\
(1 \cdot 1) \cdot 0 &= 1 \cdot (1 \cdot 0) = 0 \\
(1 \cdot 1) \cdot 1 &= 1 \cdot (1 \cdot 1) = 1
\end{aligned}$$

For property 4a, we show that

$$\begin{aligned}
0 + 0 &= 0 \\
1 + 0 &= 1
\end{aligned}$$

■

PRACTICE 1 Verify property 4b for the Boolean algebra of Example 2. ■

Bear in mind that Example 2 illustrates a particular instance of the Boolean algebra structure. Any Boolean algebra must, by the definition, have at least two elements, a 0 element and a 1 element. The specific Boolean algebra in Example 2 has just those two elements, the integers 0 and 1. But if you want to do a proof about Boolean algebras in general, you cannot assume that 0 and 1 are the only elements. This means that if you know that x and y are two elements in an arbitrary Boolean algebra and that $x \neq y$, that does not mean that $y = x'$.

EXAMPLE 3 Let $S = \{a, b, c\}$. Then $\wp(S)$ has eight elements:

$$\varnothing, \{a\}, \{b\}, \{c\}, \{a.\ b\}, \{a, c\}, \{b, c\}, \{a, b, c\}$$

so using those eight elements together with the operations of union, intersection, and complementation gives an eight-element Boolean algebra. The empty set is the 0 element, and $\{1, 2, 3\}$ is the 1 element. We could show the tables for union and intersection, but let's just look at complementation. Remember that $x \cup x' = \{a, b, c\}$.

$'$	
$\varnothing$	$\{a, b, c\}$
$\{a\}$	$\{b, c\}$
$\{b\}$	$\{a, c\}$
$\{c\}$	$\{a.\ b\}$
$\{a.\ b\}$	$\{c\}$
$\{a, c\}$	$\{b\}$
$\{b, c\}$	$\{a\}$
$\{a, b, c\}$	$\varnothing$

Here $\{a, c\}$ and $\{c\}$ are two distinct elements of this Boolean algebra, but $\{a, c\}$ is not $\{c\}'$.

There are many other properties that hold in any Boolean algebra. We can prove these additional properties by using the properties in the definition.

EXAMPLE 4 The **idempotent** (pronounced eye′-dem-po-tent) **property**

$$x + x = x$$

holds in any Boolean algebra because

$$\begin{aligned} x + x &= (x + x) \cdot 1 && \text{(4b)} \\ &= (x + x) \cdot (x + x') && \text{(5a)} \\ &= x + (x \cdot x') && \text{(3a)} \\ &= x + 0 && \text{(5b)} \\ &= x && \text{(4a)} \end{aligned}$$

REMINDER

A Boolean algebra property may be applied only when your expression exactly matches the pattern of one side of the property.

Although ordinary arithmetic of integers has many of the properties of a Boolean algebra, the idempotent property should convince you that arithmetic is not a Boolean algebra. The property $x + x = x$ does not hold for ordinary numbers and ordinary addition unless x is zero.

In the proof of Example 4, we used property 5a to replace 1 with $x + x'$. The properties of Boolean algebra are equalities, and either side of an equal sign can be replaced with the other side. The Boolean algebra properties (rules) are like the equivalence rules in logic; to apply the rule, your situation must match exactly the pattern of the rule. For example, it is legal to replace

$$(y \cdot z) + x$$

with

$$x + (y \cdot z)$$

using property 1a, because $(y \cdot z) + x$ matches the right side of 1a where y is the Boolean algebra element $y \cdot z$, and $x + (y \cdot z)$ matches the left side of 1a under the same interpretation of y. We cannot say

$$x + (y \cdot z) = (x \cdot y) + (x \cdot z)$$

using either property 3a or 3b because we have mixed up the two properties. And, strictly speaking, we cannot replace

$$(y \cdot z) + x$$

with

$$(y + x) \cdot (z + x)$$

and claim that we are using property 3a because in property 3a the addition is to the left of the multiplication. We must reason as follows:

$$\begin{aligned} (y \cdot z) + x &= x + (y \cdot z) && \text{(1a)} \\ &= (x + y) \cdot (x + z) && \text{(3a)} \\ &= (y + x) \cdot (z + x) && \text{(1a twice)} \end{aligned}$$

However, we will sometimes make implicit use of the associative property and write

$$x + y + z$$

with no parentheses.

Each property in the definition of a Boolean algebra has its dual as part of the definition, where the **dual** is obtained by interchanging $+$ and $\cdot$, and 1 and 0. For example, $x + 0 = x$ and $x \cdot 1 = x$ are duals of each other. Therefore, every time a new property P about Boolean algebras is proved, each step in that proof can be replaced by the dual of that step. The result is a proof of the dual of P. Thus, once we have proved P, we know that the dual of P also holds. It's a two-for-one deal!

EXAMPLE 5 The dual of the idempotent property (Example 4), $x \cdot x = x$, is true in any Boolean algebra. ●

PRACTICE 2

a. What does the idempotent property of Example 4 become in the context of propositional logic?
b. What does it become in the context of set theory? ■

Once a property about Boolean algebra is proved, we can use it to prove new properties.

PRACTICE 3

a. Prove that the **universal bound property** $x + 1 = 1$ holds in any Boolean algebra. Give a reason for each step.
b. What is the dual property? ■

More properties of Boolean algebras appear in the exercises at the end of this section. The most important of these properties are double negation and De Morgan's laws:

$(x')' = x$ (**double negation**—Exercise 7)
$(x + y)' = x' \cdot y' \quad (x \cdot y)' = x' + y'$ (**De Morgan's laws**—Exercise 8)

Table 8.1 suggests hints that may help when trying to prove a Boolean algebra property of the form

some expression = some other expression

TABLE 8.1

Hints for Proving Boolean Algebra Equalities
Usually the best approach is to start with the more complicated expression and try to show that it reduces to the simpler expression.
Think of adding some form of 0 (like $x \cdot x'$) or multiplying by some form of 1 (like $x + x'$).
Remember property 3a, the distributive property of addition over multiplication—it is easy to forget because it doesn't look like arithmetic.
Remember the idempotent property $x + x = x$ and its dual $x \cdot x = x$.
Remember the universal bound property $x + 1 = 1$ and its dual $x \cdot 0 = 0$.

EXAMPLE 6 Prove that $x' \cdot y = x' \cdot y + x' \cdot y \cdot z$ in any Boolean algebra.

Following the suggestion in Table 8.1, we start with the more complicated expression, $x' \cdot y + x' \cdot y \cdot z$. There is a "common factor" of $x' \cdot y$, so we should

use a distributive property, although it's not clear at this point how the z is going to disappear.

$$\begin{aligned} x' \cdot y + x' \cdot y \cdot z &= x' \cdot y \cdot 1 + x' \cdot y \cdot z && \text{(4b)} \\ &= x' \cdot y \cdot (1 + z) && \text{(3b)} \\ &= x' \cdot y \cdot (z + 1) && \text{(1a)} \\ &= x' \cdot y \cdot (1) && \text{(universal bound—and that's how } z \text{ disappears)} \\ &= x' \cdot y && \text{(4b)} \end{aligned}$$

For x an element of a Boolean algebra B, the element x' is called the **complement** of x (picking up the terminology from set theory). The complement of x satisfies

$$x + x' = 1 \qquad \text{and} \qquad x \cdot x' = 0$$

Indeed, x' is the unique element with these two properties. To prove it, suppose x_1 is an element of B with these same properties,

$$x + x_1 = 1 \qquad \text{and} \qquad x \cdot x_1 = 0$$

REMINDER

To prove that something is unique, assume that there are two of them and prove that they must be the same.

Then

$$\begin{aligned} x_1 &= x_1 \cdot 1 && \text{(4b)} \\ &= x_1 \cdot (x + x') && (x + x' = 1) \\ &= (x_1 \cdot x) + (x_1 \cdot x') && \text{(3b)} \\ &= (x \cdot x_1) + (x' \cdot x_1) && \text{(1b)} \\ &= 0 + (x' \cdot x_1) && (x \cdot x_1 = 0) \\ &= (x \cdot x') + (x' \cdot x_1) && (x \cdot x' = 0) \\ &= (x' \cdot x) + (x' \cdot x_1) && \text{(1b)} \\ &= x' \cdot (x + x_1) && \text{(3b)} \\ &= x' \cdot 1 && (x + x_1 = 1) \\ &= x' && \text{(4b)} \end{aligned}$$

REMINDER

If it walks like a duck and it quacks like a duck, it must be a duck.
If it has the two properties of the complement, then by uniqueness it must be the complement.

Thus $x_1 = x'$, and x' is unique. (Uniqueness in the context of propositional logic means that the truth table is unique, but there can be many different wffs associated with any particular truth table.)

The following theorem summarizes our observations.

THEOREM **ON THE UNIQUENESS OF COMPLEMENTS**

For any x in a Boolean algebra, if an element x_1 exists such that

$$x + x_1 = 1 \qquad \text{and} \qquad x \cdot x_1 = 0$$

then $x_1 = x'$.

PRACTICE 4 Prove that $0' = 1$ and $1' = 0$. (*Hint:* $1' = 0$ will follow by duality from $0' = 1$. To show $0' = 1$, use the theorem on the uniqueness of complements.) ■

There are many ways to define a Boolean algebra. Indeed, in our definition of Boolean algebra, we could have omitted the associative properties, since these can be derived from the remaining properties of the definition. It is much more convenient, however, to include them.

Isomorphic Boolean Algebras

What Is Isomorphism?

Two instances of a structure are **isomorphic** if there is a bijection (called an **isomorphism**) that maps the elements of one instance onto the elements of the other so that important properties are preserved. (Isomorphic graphs were discussed in Section 6.1.) If two instances of a structure are isomorphic, each is a mirror image of the other, with the elements simply relabeled. The two instances are essentially the same. Therefore, we can use the idea of isomorphism to classify instances of a structure, lumping together those that are isomorphic.

EXAMPLE 7 Consider the two partially ordered sets

$$S_1 = \{1, 2, 3, 5, 6, 10, 15, 30\};\ x \rho y \leftrightarrow x \text{ divides } y$$
$$S_2 = \wp(\{1, 2, 3\});\ A \sigma B \leftrightarrow A \subseteq B$$

The Hasse diagram of each partially ordered set appears in Figure 8. 1. These two diagrams certainly appear to be mirror images of each other; just by looking at the diagrams, an obvious relabeling of the nodes, as shown in Figure 8.2, suggests itself. The important properties of a partially ordered set are which elements are related, and the Hasse diagram displays this information. For example, Figure 8.1a shows that 1, because of its position at the bottom of the graph, is related to every element in S_1. Is this property preserved under the relabeling of Figure 8.2? Yes, because $\varnothing$ is the image of 1 under that relabeling, and $\varnothing$ is related to every element in S_2. Similarly, all the other "is related to" properties are preserved under the relabeling.

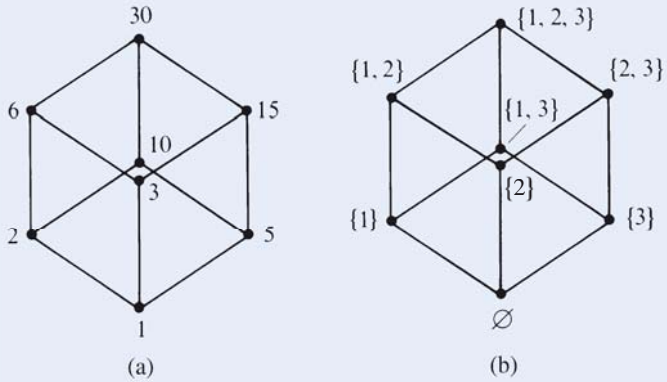


Figure 8.1

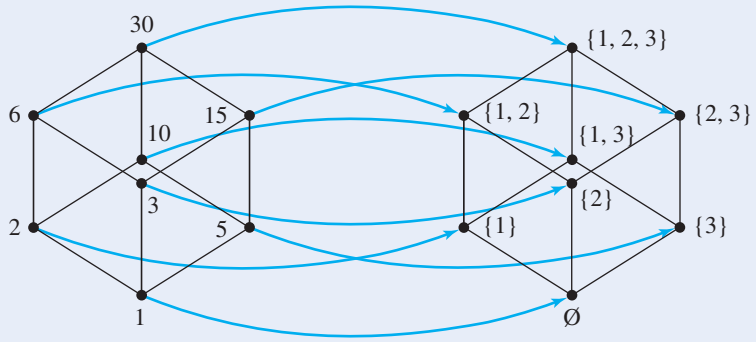


Figure 8.2

More formally, the relabeling is accomplished by the following bijection f from the set of nodes in Figure 8.1a onto the set of nodes in Figure 8.1b.

$$f(1) = \varnothing \qquad f(2) = \{1\} \qquad f(3) = \{2\} \qquad f(5) = \{3\}$$
$$f(6) = \{1, 2\} \qquad f(10) = \{1, 3\} \qquad f(15) = \{2, 3\} \qquad f(30) = \{1, 2, 3\}$$

The bijection f is an isomorphism from poset (S_1, ρ) to poset (S_2, σ) Because this isomorphism exists, the posets (S_1, ρ) and (S_2, σ) are isomorphic. (The function f^{-1} would be an isomorphism from (S_2, σ) to (S_1, ρ).)

In Example 7 it was relatively easy to find an isomorphism because of the visual representation that captured the important properties (which elements are related). Suppose that instead of a partially ordered set, we have a structure (like a Boolean algebra) where binary or unary operations are defined on a set. Then the important properties pertain to how these operations act. An isomorphism must preserve the effects of performing these operations. Each instance of two such structures that are isomorphic must be the mirror image of the other in the sense that "operate and then map" must equal "map and then operate."

Figure 8.3 illustrates this general idea for a binary operation. In Figure 8.3a, the binary operation is performed on a and b, resulting in c, then c is mapped to d. In Figure 8.3b, a and b are mapped to e and f, on which a binary operation is performed, resulting in the same element d as before. Remember,

operate and map = map and operate

Still another view of this little equation appears in the commutative diagram of Figure 8.4.

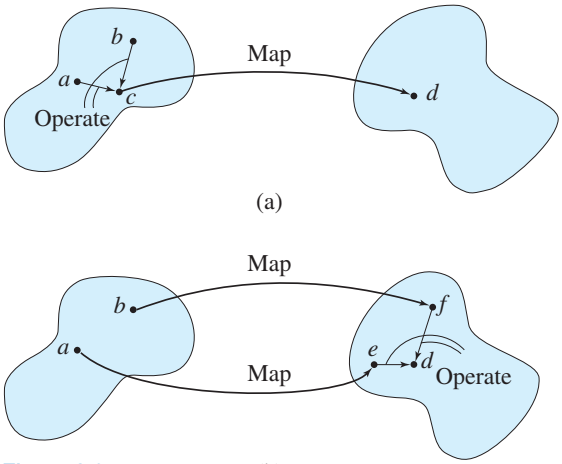


Figure 8.3

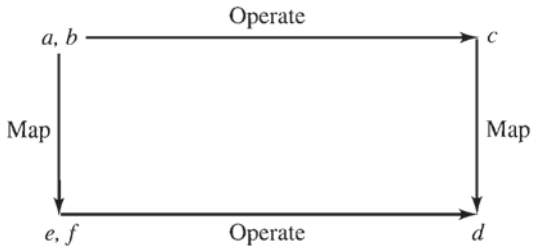


Figure 8.4

Isomorphism as Applied to Boolean Algebra

Now let's determine specifically what is involved when two instances of a Boolean algebra are isomorphic. Suppose we have two Boolean algebras, $[B, +, \cdot, ', 0, 1]$ and $[b, \&, *, '', \phi, Y]$. This notation means that, for example, if x is in B, x' is the result of performing on x the unary operation defined in B, and if z is an element of b, z'' is the result of performing on z the unary operation defined in b. How would we define isomorphism between these two Boolean algebras? First, we would need a bijection f from B onto b. Then f must preserve in b the effects of the various operations in B. There are three operations, so we use three equations to express these preservations. To preserve the operation $+$, we want to be able to operate using $+$ on two elements in B and then map the result to b, or to map the two elements to b and operate using the corresponding operation $\&$ on the results there. (Think "operate and map $=$ map and operate.") Thus, for x and y in B, we require

$$f(x + y) = f(x) \,\&\, f(y)$$

PRACTICE 5

a. Write the equation requiring f to preserve the effect of the binary operation $\cdot$.
b. Write the equation requiring f to preserve the effect of the unary operation $'$. ■

Here is the definition of an isomorphism for Boolean algebras.

DEFINITION **ISOMORPHISM FOR BOOLEAN ALGEBRAS**

Let $[B, +, \cdot, ', 0, 1]$ and $[b, \&, *, '', \phi, Y]$ be Boolean algebras. A function $B \to b$ is an **isomorphism** from $[B, +, \cdot, ', 0, 1]$ to $[b, \&, *, '', \phi, Y]$ if

1. f is a bijection
2. $f(x + y) = f(x) \,\&\, f(y)$
3. $f(x \cdot y) = f(x) * f(y)$
4. $f(x') = (f(x))''$

PRACTICE 6 Illustrate properties 2, 3, and 4 in the definition by commutative diagrams. ■

We already know (it was one of our original inspirations) that for any set S, $\wp(S)$ under the operations of union, intersection, and complementation constitutes a Boolean algebra. Example 3 talks about this type of Boolean algebra where $S = \{a, b, c\}$. If we pick $S = \{1, 2\}$, then the elements of $\wp(S)$ are $\varnothing$, $\{1\}$, $\{2\}$, and $\{1, 2\}$. The operations are given by the following tables.

$\cup$	$\varnothing$	$\{1, 2\}$	$\{1\}$	$\{2\}$
$\varnothing$	$\varnothing$	$\{1,2\}$	$\{1\}$	$\{2\}$
$\{1, 2\}$	$\{1,2\}$	$\{1, 2\}$	$\{1, 2\}$	$\{1, 2\}$
$\{1\}$	$\{1\}$	$\{1, 2\}$	$\{1\}$	$\{1, 2\}$
$\{2\}$	$\{2\}$	$\{1, 2\}$	$\{1, 2\}$	$\{2\}$

$\cap$	$\varnothing$	$\{1, 2\}$	$\{1\}$	$\{2\}$
$\varnothing$	$\varnothing$	$\varnothing$	$\varnothing$	$\varnothing$
$\{1, 2\}$	$\varnothing$	$\{1, 2\}$	$\{1\}$	$\{2\}$
$\{1\}$	$\varnothing$	$\{1\}$	$\{1\}$	$\varnothing$
$\{2\}$	$\varnothing$	$\{2\}$	$\varnothing$	$\{2\}$

$'$	
$\varnothing$	$\{1, 2\}$
$\{1, 2\}$	$\varnothing$
$\{1\}$	$\{2\}$
$\{2\}$	$\{1\}$

A Boolean algebra can be defined on the set $B = \{0, 1, a, a'\}$ where the operations of $+$, $\cdot$, and $'$ are defined by the following tables (see Exercise 1).

$+$	0	1	a	a'
0	0	1	a	a'
1	1	1	1	1
a	a	1	a	1
a'	a'	1	1	a'

$\cdot$	0	1	a	a'
0	0	0	0	0
1	0	1	a	a'
a	0	a	a	0
a'	0	a'	0	a'

$'$	
0	1
1	0
a	a'
a'	a

We claim that the mapping $f: B \to \wp(S)$ given by

$$\begin{aligned} f(0) &= \varnothing \\ f(1) &= \{1, 2\} \\ f(a) &= \{1\} \\ f(a') &= \{2\} \end{aligned}$$

is an isomorphism. Certainly it is a bijection. For $x, y \in B$, we can verify each of the equations

$$\begin{aligned} f(x + y) &= f(x) \cup f(y) \\ f(x \cdot y) &= f(x) \cap f(y) \\ f(x') &= (f(x))' \end{aligned}$$

by examining all possible cases. Thus, for example,

$$f(a \cdot 1) = f(a) = \{1\} = \{1\} \cap \{1, 2\} = f(a) \cap f(1)$$

PRACTICE 7 Verify the following equations.

a. $f(0 + a) = f(0) \cup f(a)$
b. $f(a + a') = f(a) \cup f(a')$
c. $f(a \cdot a') = f(a) \cap f(a')$
d. $f(1') = (f(1))'$ ■

The remaining cases also hold. Even without testing all cases, it is pretty clear here that f is going to work because it merely relabels the entries in the tables for B so that they resemble the tables for $\wp(S)$. In general, however, it may not be so easy to decide whether a given f is an isomorphism between two instances of a structure. Even harder to answer is the question of whether two given instances of a structure are isomorphic; we must either think up a function that works or show that no such function exists. One case where no such function exists is when the sets involved are not the same size; we cannot have a four-element Boolean algebra isomorphic to an eight-element Boolean algebra.

We just showed that a particular four-element Boolean algebra is isomorphic to $\wp(\{1, 2\})$. It turns out that any finite Boolean algebra is isomorphic to the Boolean algebra of a power set. Although we state this as a theorem, we will not prove it.

THEOREM **ON FINITE BOOLEAN ALGEBRAS**

Let B be any Boolean algebra with n elements. Then $n = 2^m$ for some m, and B is isomorphic to $\wp(\{1, 2, \ldots, m\})$.

This theorem gives us two pieces of information. The number of elements in a finite Boolean algebra must be a power of 2. Also we learn that finite Boolean algebras that are power sets are—in our lumping together of isomorphic things—

really the only kinds of finite Boolean algebras. In a sense we have come full circle. We defined a Boolean algebra to represent many kinds of situations; now we find that (for the finite case) the situations, except for the labels of objects, are the same anyway!

SECTION 8.1 REVIEW

TECHNIQUES

- Decide whether something is a Boolean algebra.
- Prove properties about Boolean algebras.
- Write the equation meaning that a function f preserves an operation from one instance of a structure to another, and verify or disprove such an equation.

MAIN IDEAS

- Mathematical structures serve as models or abstractions of common properties found in diverse situations.
- If there is an isomorphism (a bijection that preserves properties) from A to B, where A and B are instances of a structure, then except for labels, A and B are the same.
- All finite Boolean algebras are isomorphic to Boolean algebras that are power sets.

EXERCISES 8.1

1. Let $B = \{0, 1, a, a'\}$, and let $+$ and $\cdot$ be binary operations on B. The unary operation $'$ is defined by the table

$'$	
0	1
1	0
a	a'
a'	a

Suppose you know that $[B, +, \cdot, ', 0, 1]$ is a Boolean algebra. Making use of the properties that must hold in any Boolean algebra, fill in the following tables defining the binary operations $+$ and $\cdot$:

$+$	0	1	a	a'
0				
1				
a				
a'				

$\cdot$	0	1	a	a'
0				
1				
a				
a'				

2. a. What does the universal bound property (Practice 3) become in the context of propositional logic?
   b. What does it become in the context of set theory?
3. Define two binary operations $+$ and $\cdot$ on the set $\mathbb{Z}$ of integers by $x + y = \max(x, y)$ and $x \cdot y = \min(x, y)$.
   a. Show that the commutative, associative, and distributive properties of a Boolean algebra hold for these two operations on $\mathbb{Z}$.
   b. Show that no matter what element of $\mathbb{Z}$ is chosen to be 0, the property $x + 0 = x$ of a Boolean algebra fails to hold.

4. Let $M_2(\mathbb{Z})$ denote the set of 2×2 matrices with integer entries, and let $+$ denote matrix addition and $\cdot$ denote matrix multiplication. Given

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \text{ then } \mathbf{A}' = \begin{bmatrix} -a & -b \\ -c & -d \end{bmatrix}.$$

Using $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$ and $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ as the 0 element and the 1 element, respectively, either prove that $[M_2(\mathbb{Z}), +, \cdot, ', 0, 1]$ is a Boolean algebra or give a reason why it is not.

5. Let S be the set $\{0, 1\}$. Then S^2 is the set of all ordered pairs of $0s$ and $1s$; $S^2 = \{(0, 0), (0, 1), (1, 0), (1, 1)\}$. Consider the set B of all functions mapping S^2 to S. For example, one such function, $f(x,y)$, is given by

$$\begin{aligned} f(0, 0) &= 0 \\ f(0, 1) &= 1 \\ f(1, 0) &= 1 \\ f(1, 1) &= 1 \end{aligned}$$

a. How many elements are in B?

b. For f_1 and f_2 members of B and $(x, y) \in S^2$, define

$$\begin{aligned} (f_1 + f_2)(x, y) &= \max(f_1(x, y), f_2(x, y)) \\ (f_1 \cdot f_2)(x, y) &= \min(f_1(x, y), f_2(x, y)) \\ f'(x, y) &= \begin{cases} 1 \text{ if } f_1(x, y) = 0 \\ 0 \text{ if } f_1(x, y) = 1 \end{cases} \end{aligned}$$

Suppose

$$\begin{aligned} f_1(0, 0) &= 1 & f_2(0, 0) &= 1 \\ f_1(0, 1) &= 0 & f_2(0, 1) &= 1 \\ f_1(1, 0) &= 1 & f_2(1, 0) &= 0 \\ f_1(1, 1) &= 0 & f_2(1, 1) &= 0 \end{aligned}$$

What are the functions $f_1 + f_2$, $f_1 \cdot f_2$, and f_1'?

c. Prove that $[B, +, \cdot, ', 0, 1]$ is a Boolean algebra where the functions 0 and 1 are defined by

$$\begin{aligned} 0(0, 0) &= 0 & 1(0, 0) &= 1 \\ 0(0, 1) &= 0 & 1(0, 1) &= 1 \\ 0(1, 0) &= 0 & 1(1, 0) &= 1 \\ 0(1, 1) &= 0 & 1(1, 1) &= 1 \end{aligned}$$

6. Let n be a positive integer whose decomposition into prime factors has no repeated prime. Let $B = \{x | x \text{ is a divisor of } n\}$. For example, if $n = 21 = 3 \cdot 7$, then $B = \{1, 3, 7, 21\}$. Let the following operations be defined on B:

$$x + y = \text{lcm}(x, y) \qquad x \cdot y = \gcd(x, y) \qquad x' = n/x$$

Then $+$ and $\cdot$ are binary operations on B and $'$ is a unary operation on B.

a. For $n = 21$, find

(i) $3 \cdot 7$
(ii) $7 \cdot 21$
(iii) $1 + 3$
(iv) $3 + 21$
(v) $3'$

b. Prove that the commutative, associative, and distributive properties hold for both $+$ and $\cdot$.

c. Find the value of the "0" element and the "1" element, then prove properties 4 and 5 for both $+$ and $\cdot$.

d. Consider a value for n whose decomposition has repeated primes. In particular, let $n = 12 = 2 \cdot 2 \cdot 3$. Prove that, using the above definitions for $+$ and $\cdot$, it's not possible to define a complement for 6 in the set $\{1, 2, 3, 4, 6, 12\}$. Therefore a Boolean algebra cannot be constructed with $n = 12$ using the process described.

7. Prove the following property of Boolean algebras. Give a reason for each step. (*Hint:* Remember the uniqueness of the complement.)

$$(x')' = x \qquad \text{(double negation)}$$

8. Prove the following property of Boolean algebras. Give a reason for each step. (*Hint:* Remember the uniqueness of the complement.)

$$(x + y)' = x' \cdot y' \qquad (x \cdot y)' = x' + y' \quad \text{(De Morgan's laws)}$$

9. Prove the following properties of Boolean algebras. Give a reason for each step.

a. $x + (x \cdot y) = x$ (absorption properties)
$x \cdot (x + y) = x$

b. $x \cdot [y + (x \cdot z)] = (x \cdot y) + (x \cdot z)$ (modular properties)
$x + [y \cdot (x + z)] = (x + y) \cdot (x + z)$

c. $(x + y) \cdot (x' + y) = y$
$(x \cdot y) + (x' \cdot y) = y$

d. $(x + (y \cdot z))' = x' \cdot y' + x' \cdot z'$
$(x \cdot (y + z))' = (x' + y') \cdot (x' + z')$

e. $(x + y) \cdot (x + 1) = x + (x \cdot y) + y$
$(x \cdot y) + (x \cdot 0) = x \cdot (x + y) \cdot y$

10. Prove the following properties of Boolean algebras. Give a reason for each step.

a. $(x + y) + (y \cdot x') = x + y$

b. $(y + x) \cdot (z + y) + x \cdot z \cdot (z + z') = y + x \cdot z$

c. $(y' \cdot x) + x + (y + x) \cdot y' = x + (y' \cdot x)$

d. $(x + y') \cdot z = [(x' + z') \cdot (y + z')]'$

e. $(x \cdot y) + (x' \cdot z) + (x' \cdot y \cdot z') = y + (x' \cdot z)$

11. Prove the following properties of Boolean algebras. Give a reason for each step.
    a. $x + y' = x + (x' \cdot y + x \cdot y)'$
    b. $[(x \cdot y) \cdot z] + (y \cdot z) = y \cdot z$
    c. $x \cdot y + y \cdot x' = x \cdot y + y$
    d. $(x + y)' \cdot z + x' \cdot z \cdot y = x' \cdot z$
    e. $(x \cdot y') + (y \cdot z') + (x' \cdot z) = (x' \cdot y) + (y' \cdot z) + (x \cdot z')$
12. Prove the following properties of Boolean algebras. Give a reason for each step.
    a. $(x + y \cdot x)' = x'$
    b. $x \cdot (z + y) + (x' + y)' = x$
    c. $(x \cdot y)' + x' \cdot z + y' \cdot z = x' + y'$
    d. $x \cdot y + x' = y + x' \cdot y'$
    e. $x \cdot y + y \cdot z \cdot x' = y \cdot z + y \cdot x \cdot z'$
13. Prove that in any Boolean algebra, $x \cdot y' + x' \cdot y = y$ if and only if $x = 0$.
14. Prove that in any Boolean algebra, $x \cdot y' = 0$ if and only if $x \cdot y = x$.
15. A new binary operation $\oplus$ in a Boolean algebra (*exclusive OR*) is defined by

$$x \oplus y = x \cdot y' + y \cdot x'$$

    Prove that
    a. $x \oplus y = y \oplus x$
    b. $x \oplus x = 0$
    c. $0 \oplus x = x$
    d. $1 \oplus x = x'$
16. Prove that for any Boolean algebra:
    a. If $x + y = 0$, then $x = 0$ and $y = 0$.
    b. $x = y$ if and only if $x \cdot y' + y \cdot x' = 0$.
17. Prove that the 0 element in any Boolean algebra is unique; prove that the 1 element in any Boolean algebra is unique.
18. a. Find an example of a Boolean algebra with elements x, y, and z for which $x + y = x + z$ but $y \neq z$. (Here is further evidence that ordinary arithmetic of integers is not a Boolean algebra.)
    b. Prove that in any Boolean algebra, if $x + y = x + z$ and $x' + y = x' + z$, then $y = z$.
19. Let $(S, \preccurlyeq)$ and $(S', \preccurlyeq')$ be two partially ordered sets. $(S, \preccurlyeq)$ is isomorphic to $(S', \preccurlyeq')$ if there is a bijection $f: S \to S'$ such that for x, y in S, $x \prec y \to f(x) \prec' f(y)$ and $f(x) \prec' f(y) \to x \prec y$.
    a. Show that there are exactly two nonisomorphic, partially ordered sets with two elements (use diagrams).
    b. Show that there are exactly five nonisomorphic, partially ordered sets with three elements.
    c. How many nonisomorphic, partially ordered sets with four elements are there?
20. Find an example of two partially ordered sets $(S, \preccurlyeq)$ and $(S', \preccurlyeq')$ and a bijection $f: S \to S'$ where, for x, y in S, $x \prec y \to f(x) \prec' f(y)$ but $f(x) \prec' f(y) \not\to x \prec y$.

21. Let $S = \{0, 1\}$ and let a binary operation $\cdot$ be defined on S by

$\cdot$	0	1
0	1	0
1	0	1

Let $T = \{5, 7\}$, and let a binary operation $+$ be defined on T by

$+$	5	7
5	7	5
7	5	7

Consider $[S, \cdot]$ and $[T, +]$ as mathematical structures.

a. If a function f is an isomorphism from $[S, \cdot]$ to $[T, +]$, what two properties must f satisfy?

b. Define a function $f: S \to T$ and prove that it is an isomorphism from $[S, \cdot]$ to $[T, +]$.

22. Consider the four-element Boolean algebra defined in Exercise 6 with $n = 21$. Find an isomorphism from this Boolean algebra to the four-element Boolean algebra with set $\wp(\{1, 2\})$ that was defined in this section.

23. Let $\mathbb{R}$ denote the real numbers and $\mathbb{R}^+$ the positive real numbers. Addition is a binary operation on $\mathbb{R}$, and multiplication is a binary operation on $\mathbb{R}^+$. Consider $[\mathbb{R}, +]$ and $[\mathbb{R}^+, \cdot]$ as mathematical structures.

a. Prove that the function f defined by $f(x) = 2^x$ is a bijection from $\mathbb{R}$ to $\mathbb{R}^+$.

b. Write the equation that an isomorphism from $[\mathbb{R}, +]$ to $[\mathbb{R}^+, \cdot]$ must satisfy.

c. Prove that the function f of part (a) is an isomorphism from $[\mathbb{R}, +]$ to $[\mathbb{R}^+, \cdot]$.

d. What is f^{-1} for this function?

e. Prove that f^{-1} is an isomorphism from $[\mathbb{R}^+, \cdot]$ to $[\mathbb{R}, +]$.

24. An isomorphism from the Boolean algebra with set $B = \{0, 1, a, a'\}$ to the Boolean algebra with set $\wp(\{1, 2\})$ was defined in this section. Because the two Boolean algebras are essentially the same, an operation in one can be simulated by mapping to the other, operating there, and mapping back.

a. Use the Boolean algebra on $\wp(\{1, 2\})$ to simulate the computation $1 \cdot a'$ in the Boolean algebra on B.

b. Use the Boolean algebra on $\wp(\{1, 2\})$ to simulate the computation $(a)'$ in the Boolean algebra on B.

c. Use the Boolean algebra on B to simulate the computation $\{1\} \cup \{2\}$ in the Boolean algebra on $\wp(\{1, 2\})$.

25. Consider the set B of all functions mapping $\{0, 1\}^2$ to $\{0, 1\}$. We can define operations of $+$, $\cdot$, and $'$ on B by

$$(f_1 + f_2)(x, y) = \max(f_1(x, y), f_2(x, y))$$
$$(f_1 \cdot f_2)(x, y) = \min(f_1(x, y), f_2(x, y))$$
$$f_1'(x, y) = \begin{cases} 1 & \text{if } f_1(x, y) = 0 \\ 0 & \text{if } f_1(x, y) = 1 \end{cases}$$

Then $[B, +, \cdot, ', 0, 1]$ is a Boolean algebra of 16 elements (see Exercise 5). The following table assigns names to these 16 functions.

(x, y)	0	1	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}
(0, 0)	0	1	1	1	1	1	1	1	0	0	0	1	0	0	0	0
(0, 1)	0	1	0	1	1	0	1	0	1	1	1	0	0	1	0	0
(1, 0)	0	1	1	0	1	0	0	1	1	1	0	0	1	0	1	0
(1, 1)	0	1	0	0	0	0	1	1	1	0	1	1	1	0	0	1

According to the theorem on finite Boolean algebras, this Boolean algebra is isomorphic to $[\wp(\{1, 2, 3, 4\}), \cup, \cap, ', \varnothing, \{1, 2, 3, 4\}]$. Complete the following definition of an isomorphism from B to $\wp(\{1, 2, 3, 4\})$.

$$\begin{aligned} 0 &\to \varnothing \\ 1 &\to \{1, 2, 3, 4\} \\ f_4 &\to \{1\} \\ f_{12} &\to \{2\} \\ f_{13} &\to \{3\} \\ f_{14} &\to \{4\} \end{aligned}$$

26. Let P, Q, and R be three statements in propositional logic with statement letters A and B. P, Q, and R define the following three functions from $\{\mathrm{T}, \mathrm{F}\}^2$ to $\{\mathrm{T}, \mathrm{F}\}$. Also shown are the contradiction 0 and the tautology 1.

A	B	P	Q	R	0	1
T	T	T	F	F	F	T
T	F	F	T	F	F	T
F	T	F	F	T	F	T
F	F	T	F	F	F	T

a. Let $B = \{P, P', Q, Q', R, R', 0, 1\}$. Then $[B, \vee, \wedge, ', 0, 1]$ is a Boolean algebra. Write the 8×8 tables for the $\vee$ and $\wedge$ operations and the 8×1 table for the $'$ operation.

b. $[\wp(\{1, 2, 3\}, \cup, \cap, ', \varnothing, \{1, 2, 3\}]$ is a Boolean algebra. Write the 8×8 tables for the $\cup$ and $\cap$ operations and the 8×1 table for the $'$ operation.

c. Find an isomorphism from the Boolean algebra of part (a) to the Boolean algebra of part (b).

27. Suppose that $[B, +, \cdot, ', 0, 1]$ and $[b, \&, *, '', \phi, \curlyvee]$ are isomorphic Boolean algebras and that f is an isomorphism from B to b.

a. Prove that $f(0) = \phi$.

b. Prove that $f(1) = \curlyvee$.

28. According to the theorem on finite Boolean algebras, which we did not prove, any finite Boolean algebra must have 2^m elements for some m. Prove the weaker statement that no Boolean algebra can have an odd number of elements. (Note that in the definition of a Boolean algebra, 0 and 1 are distinct elements of B, so B has at least two elements. Arrange the remaining elements of B so that each element is paired with its complement.)

29. A Boolean algebra may also be defined as a partially ordered set with certain additional properties. Let $(B, \preccurlyeq)$ be a partially ordered set. For any $x, y \in B$, we define the *least upper bound* of x and y as an element z such that $x \preccurlyeq z$, $y \preccurlyeq z$, and if there is any element $z*$ with $x \preccurlyeq z*$ and $y \preccurlyeq z*$, then $z \preccurlyeq z*$. The *greatest lower bound* of x and y is an element w such that $w \preccurlyeq x$, $w \preccurlyeq y$, and if there is any element $w*$ with $w* \preccurlyeq x$ and $w* \preccurlyeq y$, then $w* \preccurlyeq w$. A *lattice* is a partially ordered set in which every two elements x and y have a least upper bound, denoted by $x + y$, and a greatest lower bound, denoted by $x \cdot y$.

a. Prove that in any lattice

(i) $x \cdot y = x$ if and only if $x \preccurlyeq y$
(ii) $x + y = y$ if and only if $x \preccurlyeq y$

b. Prove that in any lattice

(i) $x + y = y + x$
(ii) $x \cdot y = y \cdot x$
(iii) $(x + y) + z = x + (y + z)$
(iv) $(x \cdot y) \cdot z = x \cdot (y \cdot z)$

c. A lattice L is *complemented* if there exists a least element 0 and a greatest element 1, and for every $x \in L$ there exists $x' \in L$ such that $x + x' = 1$ and $x \cdot x' = 0$. Prove that in a complemented lattice L,

$$x + 0 = x \qquad \text{and} \qquad x \cdot 1 = x$$

for all $x \in L$.

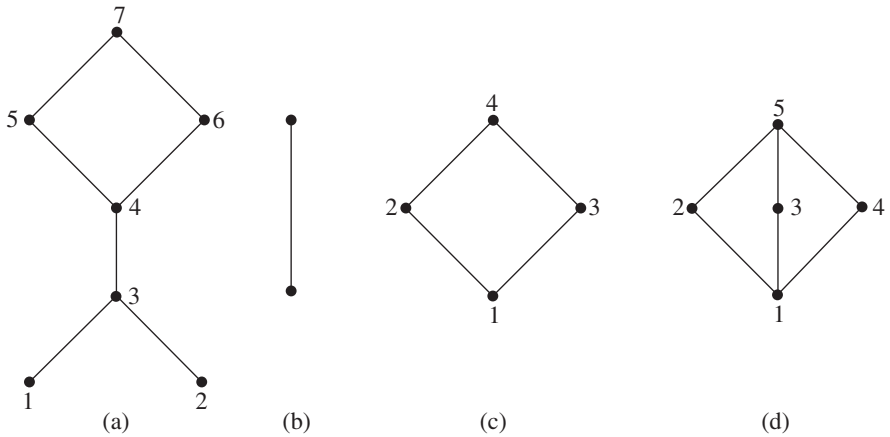
d. A lattice L is *distributive* if

$$x + (y \cdot z) = (x + y) \cdot (x + z)$$

and

$$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$$

for every $x, y, z \in L$. By parts (b) and (c), a complemented, distributive lattice is a Boolean algebra. Which of the following Hasse diagrams of partially ordered sets do not represent Boolean algebras? Why? (*Hint:* In a Boolean algebra, the complement of an element is unique.)



(a) (b) (c) (d)

30. a. Let n be a positive integer and consider B to be the set of all positive integer divisors of n. Prove that $(B, \preccurlyeq)$ is a partially ordered set where $x \preccurlyeq y$ means $x \mid y$.

In the terminology of Exercise 29, the least upper bound of x and y is the least common multiple of x and y, and the greatest lower bound is the greatest common divisor. $(B, \preccurlyeq)$ is a distributive lattice.

b. Prove that for $n = 6$, $(B, \preccurlyeq)$ is a Boolean algebra. (*Hint:* 1 is the least element and 6 is the greatest element).

c. For $n = 8$, $(B, \preccurlyeq)$ is not a Boolean algebra.

(i) Show that this is true by using the definition of a Boolean algebra.

(ii) Show that this is true by using Exercise 6.

SECTION 8.2 LOGIC NETWORKS

Combinational Networks

Basic Logic Elements

In 1938 the American mathematician Claude Shannon perceived the parallel between propositional logic and circuit logic and realized that Boolean algebra could play a part in systematizing this new realm of electronics.

Let us imagine that the electrical voltages carried along wires fall into one of two ranges, high or low, which we represent by 1 and 0, respectively. Voltage fluctuations within these ranges are ignored, so we are forcing a discrete, indeed binary, mask on an analog phenomenon. We also suppose that switches can be wired so that a signal of 1 causes the switch to be closed and a signal of 0 causes the switch to be open (Figure 8.5). Now we combine two such switches, controlled by lines x_1 and x_2, in parallel. If either or both lines carry a 1 value, one or both of the switches will be closed, and the output line will have a value of 1. However, values of $x_1 = 0$ and $x_2 = 0$ will cause both switches to be open and thus break the circuit, so that the voltage level on the output line will be 0. Figure 8.6 illustrates the various cases.

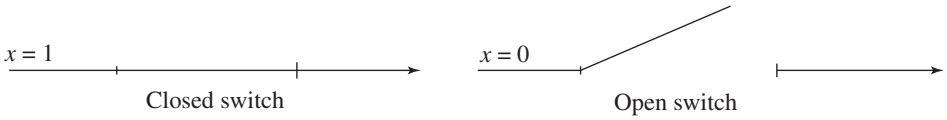


Figure 8.5

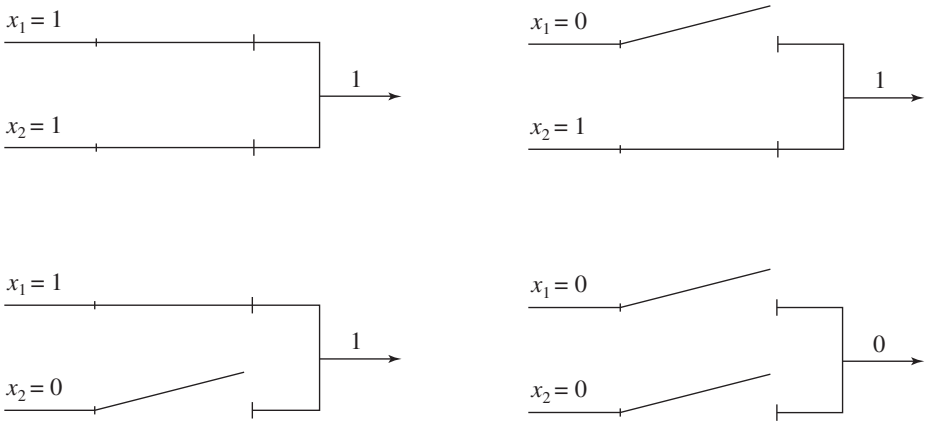
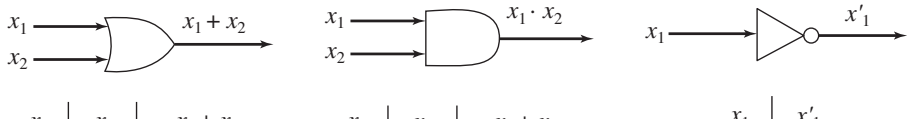


Figure 8.6

TABLE 8.2		
x_1	x_2	Output
1	1	1
1	0	1
0	1	1
0	0	0

Table 8.2 summarizes the behavior of the circuit. Substituting T for 1 and F for 0 in the table results in the truth table for the logical connective of disjunction. Disjunction is an example of the Boolean algebra operation + in the realm of propositional logic. Thus we may think of the circuit more abstractly as an electronic device that performs the Boolean operation +. Similarly, conjunction and negation are examples of the Boolean algebra operations · and ′, respectively, in the realm of propositional logic. Other devices perform these Boolean operations. For example, switches connected in series would serve to implement the · operation; both switches must be closed ($x_1 = 1$ and $x_2 = 1$) in order to have an output of 1. However, we'll ignore the details of implementing the devices; suffice it to say that technology has progressed from mechanical switches through vacuum tubes and then transistors to integrated circuits, and now even bacteria and DNA. We will simply represent these devices by their standard symbols.

The **OR gate**, Figure 8.7a, behaves like the Boolean operation +. The **AND gate**, Figure 8.7b, represents the Boolean operation ·. Figure 8.7c shows an **inverter**, corresponding to the unary Boolean operation ′. Because of the associativity property for + and · , the OR and AND gates can have more than two inputs.



x_1	x_2	$x_1 + x_2$
1	1	1
1	0	1
0	1	1
0	0	0

(a) OR gate

x_1	x_2	$x_1 + x_2$
1	1	1
1	0	0
0	1	0
0	0	0

(b) AND gate

x_1	x'_1
1	0
0	1

(c) Inverter

Figure 8.7

Boolean Expressions

DEFINITION **BOOLEAN EXPRESSION**

A **Boolean expression** in n variables, $x_1, x_2, \ldots, x_n$, is any finite string of symbols formed by applying the following rules:

1. $x_1, x_2, \ldots, x_n$, are Boolean expressions.
2. If P and Q are Boolean expressions, so are $(P + Q)$, $(P \cdot Q)$, and (P').

(The definition of a Boolean expression is another example of a recursive definition; rule 1 is the basis step and rule 2 the inductive step.) When there is no chance of confusion, we can omit the parentheses introduced by rule 2. In addition, we define · to take precedence over + and ′ to take precedence over + or ·, so $x_1 + x_2 \cdot x_3$ stands for $x_1 + (x_2 \cdot x_3)$ and $x_1 + x_2'$ stands for $x_1 + (x_2')$; this convention also allows us to remove some parentheses. Finally, we will generally omit the symbol · and use juxtaposition, so $x_1 \cdot x_2$ is written x_1x_2.

EXAMPLE 8 x_3, $(x_1 + x_2)'x_3$, $(x_1x_3 + x_4')x_2$, and $(x_1' x_2)'x_1$ are all Boolean expressions.

Truth Functions

DEFINITION **TRUTH FUNCTION**

A **truth function** is a function f such that f: $\{0, 1\}^n \to \{0, 1\}$ for some integer $n \geq 1$.

The notation $\{0, 1\}^n$ denotes the set of all n-tuples of 0s and 1s. A truth function thus associates a value of 0 or 1 with each such n-tuple.

EXAMPLE 9 The truth table for the Boolean operation + describes a truth function f with $n = 2$. The domain of f is $\{(1, 1), (1, 0), (0, 1), (0, 0)\}$, and $f(1, 1) = 1, f(1, 0) = 1, f(0, 1) = 1$, and $f(0, 0) = 0$. Similarly, the Boolean operation $\cdot$ describes a different truth function with $n = 2$, and the Boolean operation $'$ describes a truth function for $n = 1$.

PRACTICE 8

a. If we are writing a truth function f: $\{0, 1\}^n \to \{0, 1\}$ in tabular form (like a truth table), how many rows will the table have?
b. How many different truth functions are there that take $\{0, 1\}^2 \to \{0, 1\}$?
c. How many different truth functions are there that take $\{0, 1\}^n \to \{0, 1\}$?

Any Boolean expression defines a unique truth function, just as do the simple Boolean expressions $x_1 + x_2$, x_1x_2, and x_1'.

EXAMPLE 10 The Boolean expression $x_1x_2' + x_3$ defines the truth function given in Table 8.3. (This is just like doing the truth tables of Section 1.1.)

TABLE 8.3

x_1	x_2	x_3	$x_1x_2' + x_3$
1	1	1	1
1	1	0	0
1	0	1	1
1	0	0	1
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	0

Networks and Expressions

It's time to see how these ideas of logic gates, Boolean expressions, and truth functions are related. By combining AND gates, OR gates, and inverters, we can construct a logic network representing a given Boolean expression that produces the same truth function as that expression.

EXAMPLE 11 The logic network for the Boolean expression $x_1x_2' + x_3$ is shown in Figure 8.8.

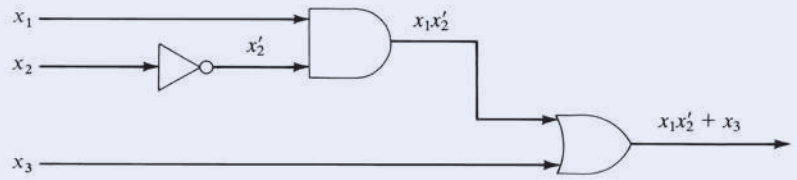


Figure 8.8

PRACTICE 9 Design the logic network for the following Boolean expressions.

a. $x_1 + x_2'$
b. $x_1(x_2 + x_3)'$

Conversely, if we have a logic network, we can write a Boolean expression with the same truth function.

EXAMPLE 12 A Boolean expression for the logic network in Figure 8.9 is

$$(x_1x_2 + x_3)' + x_3$$

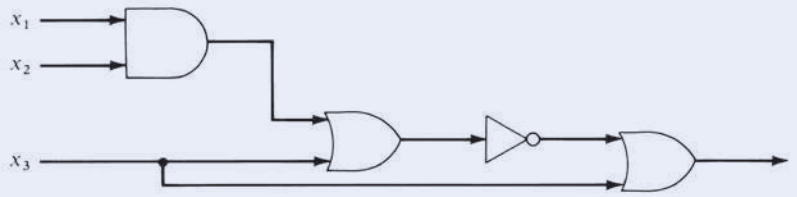


Figure 8.9

PRACTICE 10

a. Write a Boolean expression for the logic network in Figure 8.10.

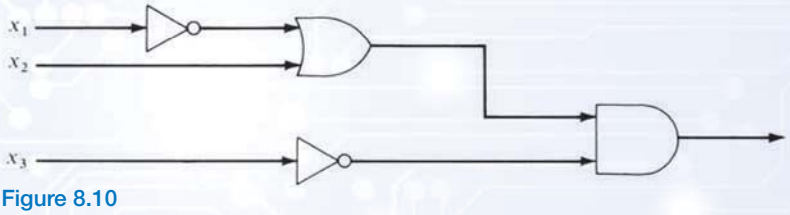


Figure 8.10

b. Write the truth function (in table form) for the network (and expression) of part (a).

Logic networks constructed of AND gates, OR gates, and inverters are also called **combinational networks**. They have several features that we should note. First, input or output lines are not tied together except by passing through gates. Lines can be split, however, to serve as input to more than one device. There are no loops where the output of an element is part of the input to that same element. Finally, the output of a network is an instantaneous function of the input; there are no delay elements that capture and remember input signals. Notice also that the picture of any network is, in effect, a directed graph.

Canonical Form

Here is the situation so far (arrows indicate a procedure that we can carry out):

$$\text{truth function} \leftarrow \text{Boolean expression} \leftrightarrow \text{logic network}$$

We can write a unique truth function from either a network or an expression. Given an expression, we can find a network with the same truth function, and conversely. The last part of the puzzle concerns how to get from an arbitrary truth function to an expression (and hence a network) having that truth function. An algorithm to solve this problem is explained in the next example.

EXAMPLE 13

Suppose we want to find a Boolean expression for the truth function f of Table 8.4. There are four rows in the table (rows 1, 3, 4, and 7) for which f is 1. The basic form of our expression will be a sum of four terms

$$(\ ) + (\ ) + (\ ) + (\ )$$

such that the first term has the value 1 for the input values of row 1 and for no others, the second term has the value 1 for the input values of row 3 and for no others, and so on. Thus, the entire expression has the value 1 for these inputs and for no others—precisely what we want. (Other inputs cause each term in the sum, and hence the sum itself, to be 0.)

TABLE 8.4

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	0
1	0	1	1
1	0	0	1
0	1	1	0
0	1	0	0
0	0	1	1
0	0	0	0

Each term in the sum will be a product of the form $\alpha\beta\gamma$ where α is either x_1 or x_1', β is either x_2 or x_2', and γ is either x_3 or x_3'. If the input value of x_i, $i = 1, 2, 3$, in the row we are working on is 1, then x_i itself is used; if the input value of x_i in the row we are working on is 0, then x_i' is used. These values will force $\alpha\beta\gamma$ to be 1 for that row and 0 for all other rows. Thus, we have

$$\text{row 1: } x_1x_2x_3$$
$$\text{row 3: } x_1x_2'x_3$$
$$\text{row 4: } x_1x_2'x_3'$$
$$\text{row 7: } x_1'x_2'x_3$$

The final expression is

$$(x_1x_2x_3) + (x_1x_2'x_3) + (x_1x_2'x_3') + (x_1'x_2'x_3)$$

The procedure described in Example 13 always leads to an expression that is a sum of products, called the **canonical sum-of-products form**, or the **disjunctive normal form**, for the given truth function. The only case not covered by this procedure is when the function has a value of 0 everywhere. Then we use an expression such as

$$x_1x_1'$$

which is also a sum (one term) of products. Therefore, we can find a sum-of-products expression to represent any truth function. A pseudocode description of the algorithm is given in the accompanying box. For this algorithm, the input is a truth table representing a truth function on n variables $x_1, x_2, \ldots, x_n$; the output is a Boolean expression in disjunctive normal form with the same truth function.

ALGORITHM ***SUM-OF-PRODUCTS***

```
Sum-Of-Products (truth table; integer n)
//the truth table represents a truth function with n arguments;
//result is the canonical sum-of-products expression for this truth function
Local variables:
sum        //sum-of-products expression
product    //single term in sum, a product
i          //index for the columns of the table
row        //index for the rows of the table

   sum = empty
```

```
for row = 1 to 2^n do
    if truth value for row is 1 then
        initialize product;
        for i = 1 to n do
            if x_i = 1 then
                put x_i in product
            else
                put x_i' in product
            end if
        end for
        sum = sum + product
    end if
end for
if sum is empty then
    sum = x_1 x_1'
end if
write ("The canonical sum-of-products expression for this truth function is", sum)
end Sum-Of-Products
```

Because any expression has a corresponding network, any truth function has a logic network representation. Furthermore, the AND gate, OR gate, and inverter are the only devices needed to construct the network. Thus, we can build a network for any truth function with only three kinds of parts—and lots of wire! Later we will see that it is necessary to stock only one kind of part.

Given a truth function, the canonical sum-of-products form just described is one expression that has this truth function, but it is not the only possible one. A method for obtaining a different expression for any truth function is given in Exercise 25 at the end of this section.

EXAMPLE 14 The network for the canonical sum-of-products form of Example 13 is shown in Figure 8.11. We have drawn the inputs to each AND gate separately because it looks neater, but actually a single x_1, x_2, or x_3 input can be split as needed.

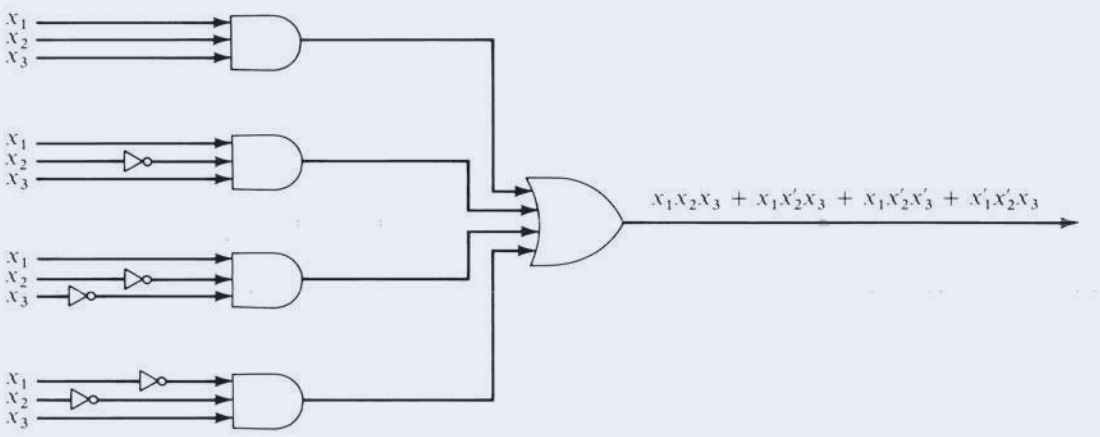


Figure 8.11

PRACTICE 11

a. Find the canonical sum-of-products form for the truth function of Table 8.5.
b. Draw the network for the expression of part (a).

TABLE 8.5

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	0
1	0	1	1
1	0	0	1
0	1	1	0
0	1	0	0
0	0	1	1
0	0	0	1

■

Minimization

As already noted, a given truth function may be represented by more than one Boolean expression and hence by more than one logic network composed of AND gates, OR gates, and inverters.

EXAMPLE 15 The Boolean expression

$$x_1x_3 + x_2'$$

has the truth function of Table 8.5. The logic network corresponding to this expression is given by Figure 8.12. Compare this with your network in Practice 11(b)!

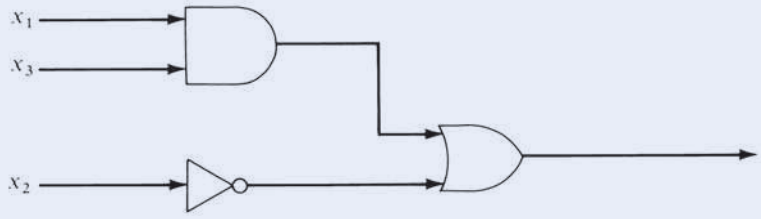


Figure 8.12 ●

DEFINITION EQUIVALENT BOOLEAN EXPRESSIONS

Two Boolean expressions are **equivalent** if they have the same truth functions.

We know that

$$x_1x_2x_3 + x_1x_2'x_3 + x_1x_2'x_3' + x_1'x_2'x_3 + x_1'x_2'x_3'$$

and

$$x_1x_3 + x_2'$$

for example, are equivalent Boolean expressions.

Clearly, equivalence of Boolean expressions is an equivalence relation on the set of all Boolean expressions in n variables. Each equivalence class is associated with a distinct truth function. Given a truth function, algorithm *Sum-Of-Products* produces one particular member of the class associated with that function, namely, the canonical sum-of-products form. However, if we are trying to design the logic network for that function, we want to find a member of the class that is as simple as possible. We would rather build the network of Figure 8.12 than the one for Practice 11(b).

How can we reduce a Boolean expression to an equivalent, simpler expression? We can use the properties of a Boolean algebra because they express the equivalence of Boolean expressions. If P is a Boolean expression containing the subexpression $(x_1 + x_2)(x_1 + x_3)$, for example, and Q is the expression obtained from P by replacing $(x_1 + x_2)(x_1 + x_3)$ with the equivalent expression $x_1 + (x_2x_3)$, then P and Q are equivalent and Q is simpler than P.

EXAMPLE 16 Using the properties of Boolean algebra, we can reduce

$$x_1x_2x_3 + x_1x_2'x_3 + x_1x_2'x_3' + x_1'x_2'x_3 + x_1'x_2'x_3'$$

to

$$x_1x_3 + x_2'$$

as follows:

$$\begin{aligned}
&x_1x_2x_3 + x_1x_2'x_3 + x_1x_2'x_3' + x_1'x_2'x_3 + x_1'x_2'x_3' \\
&\quad = x_1x_2x_3 + x_1x_2'x_3 + x_1x_2'x_3 + x_1x_2'x_3' + x_1'x_2'x_3 + x_1'x_2'x_3' && \text{(idempotent)} \\
&\quad = x_1x_3x_2 + x_1x_3x_2' + x_1x_2'x_3 + x_1x_2'x_3' + x_1'x_2'x_3 + x_1'x_2'x_3' && \text{(1b)} \\
&\quad = x_1x_3(x_2 + x_2') + x_1x_2'(x_3 + x_3') + x_1'x_2'(x_3 + x_3') && \text{(3b)} \\
&\quad = x_1x_3 \cdot 1 + x_1x_2' \cdot 1 + x_1'x_2' \cdot 1 && \text{(5a)} \\
&\quad = x_1x_3 + x_1x_2' + x_1'x_2' && \text{(4b)} \\
&\quad = x_1x_3 + x_2'x_1 + x_2'x_1' && \text{(1b)} \\
&\quad = x_1x_3 + x_2'(x_1 + x_1') && \text{(3b)} \\
&\quad = x_1x_3 + x_2' \cdot 1 && \text{(5a)} \\
&\quad = x_1x_3 + x_2' && \text{(4b)}
\end{aligned}$$

Unfortunately, one must be fairly clever to apply Boolean algebra properties to simplify an expression. In Section 8.3 we will discuss more systematic approaches to this minimization problem that require less ingenuity. For now, we should say

a bit more about why we want to minimize. When logic networks were built from separate gates and inverters, the cost of these elements was a considerable factor in the design, and it was desirable to have as few elements as possible. Now, however, most networks are built using integrated circuit technology, a development that began in the early 1960s. An integrated circuit is itself a logic network representing a certain truth function or functions, just as if some gates and inverters had been combined in the appropriate arrangement inside a package. These integrated circuits are then combined as needed to produce the desired result. Because the integrated circuits are extremely small and relatively inexpensive, it might seem pointless to bother minimizing a network. However, minimization is still important because the reliability of the final network is inversely related to the number of connections between the integrated circuit packages.

Moreover, the designers of integrated circuits are highly interested in the minimization problem. Integrated circuits are embedded into a substrate of silicon or other semiconductor material. The resulting chips may be tiny, yet they can contain the equivalent of 3 billion transistors for implementing truth functions. The distance between two gates may be as small as 45 nanometers (about 1/1000 of the width of a human hair). Minimizing the number of components and the amount of wiring required to realize a desired truth function makes it possible to embed more functions in a single chip.

Programmable Logic Devices

Instead of designing a custom chip to implement particular truth functions, a **PLD** (*p*rogrammable *l*ogic *d*evice) can be used. A PLD is a chip that is already implanted with an array of AND gates and an array of OR gates, together with a rectangular grid of wiring channels and some inverters. Once Boolean expressions in sum-of-products form have been determined for the truth functions, the required components in the PLD are activated. Although this chip is not very efficient and is practical only for smaller-scale circuit logic, on the order of hundreds of gates, the PLD can be mass-produced, and only a small amount of time (i.e., money) is then required to "program" it for the desired functions. A **FPGA** (*f*ield-*p*rogrammable *g*ate *a*rray) is a big brother to the PLD. The term "field-programmable" suggests that, like a PLD, the user can configure the chip for a specific purpose. An FPGA basically connects a number of PLDs in a reconfigurable way, and it can support thousands of gates. Often the FPGA also contains hardwired components such as multipliers or even processors and memory, thus producing a small reconfigurable computer that is "programmed" in hardware rather than software.

EXAMPLE 17

Figure 8.13a shows a PLD for the three inputs x_1, x_2, and x_3. There are four output lines, so four functions can be programmed in this PLD. When the PLD is programmed, the horizontal line going into an AND gate will pick up certain inputs, and the AND gate will form the product of these inputs. The vertical line going into an OR gate will, when programmed, allow the OR gate to form the sum of certain inputs. Figure 8.13b shows the same PLD programmed to produce the truth functions f_1 from Example 13 ($x_1x_2x_3 + x_1x_2'x_3 + x_1x_2'x_3' + x_1'x_2'x_3$) and f_2 from Practice 11 ($x_1x_2x_3 + x_1x_2'x_3 + x_1x_2'x_3' + x_1'x_2'x_3 + x_1'x_2'x_3'$). The dots represent activation points.

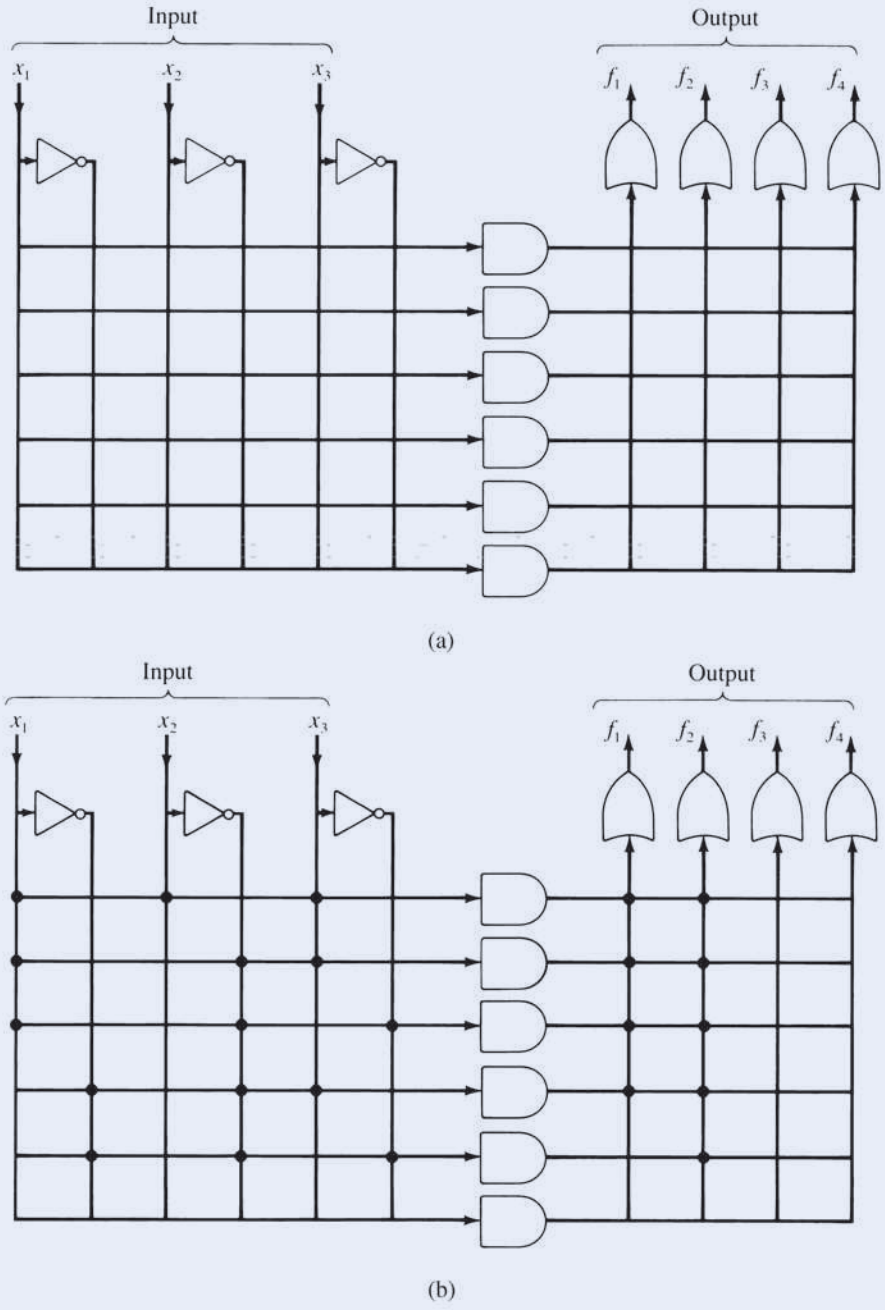


Figure 8.13

A Useful Network

We can design a network that adds binary numbers, a basic operation that a computer must be able to perform. The rules for adding two one-bit numbers are summarized in Table 8.6.

TABLE 8.6

x_1	x_2	Sum
1	1	10
1	0	1
0	1	1
0	0	0

TABLE 8.7

x_1	x_2	s
1	1	0
1	0	1
0	1	1
0	0	0

TABLE 8.8

x_1	x_2	c
1	1	1
1	0	0
0	1	0
0	0	0

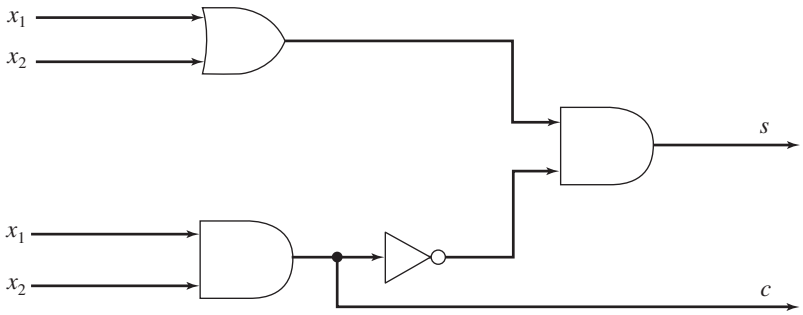
We can express the sum as a single sum bit s (the right-hand bit of the actual sum) together with a single carry bit c; doing this gives us the two truth functions of Tables 8.7 and 8.8, respectively. The canonical sum-of-products form for each truth function is

$$s = x_1'x_2 + x_1x_2'$$
$$c = x_1x_2$$

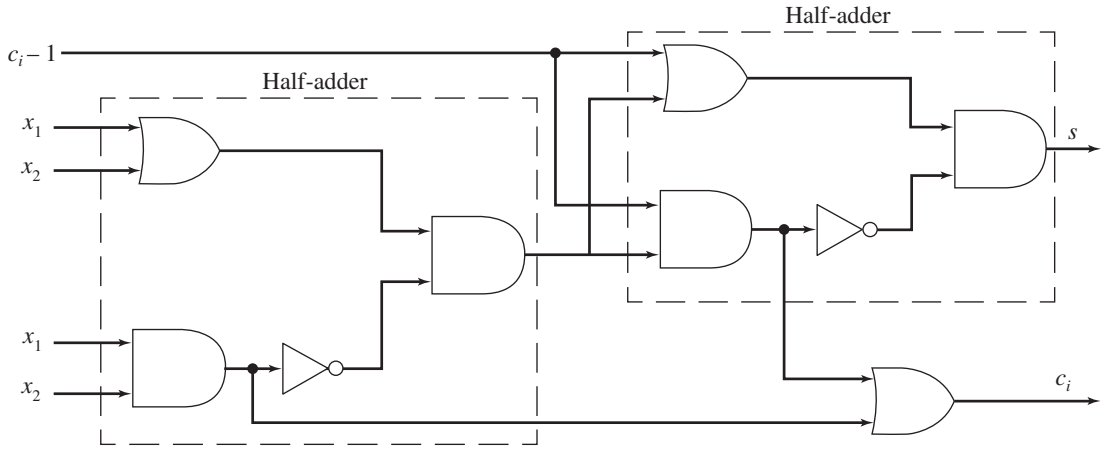
An equivalent Boolean expression for s is

$$s = (x_1 + x_2)(x_1x_2)'$$

Figure 8.14a shows a network with inputs x_1 and x_2 and outputs s and c. This device, for reasons that will be clear shortly, is called a **half-adder**.



(a) Half-adder



(b) Full-adder

Figure 8.14

To add two n-bit binary numbers, we add column by column from the low-order to the high-order bits. The ith column (except for the very first column) has as input its two bits x_1 and x_2 plus the carry bit from the addition of column $i - 1$ to its right. Thus we need a device incorporating the previous carry bit as input. Such a device can be accomplished by adding x_1 and x_2 with a half-adder and then adding the previous carry bit c_{i-1} (using another half-adder) to the result. Again, a sum bit s and final carry bit c_i are output, where c_i is 1 if either half-adder produces a 1 as its carry bit. The **full-adder** is shown in Figure 8.14b. The full-adder is thus composed of two half-adders and an additional OR gate.

To add two n-bit binary numbers, the two low-order bits, where there is no input carry bit, can be added with a half-adder. The remaining bits are added with full-adders. All are chained together. Figure 8.15 shows the modules required to add two 3-bit binary numbers $z_1y_1x_1$ and $z_2y_2x_2$, resulting in the answer $a_3a_2a_1a_0$, where the leading bit a_3 is the final carry bit and could be 0 (leading 0s are usually not written) or 1.

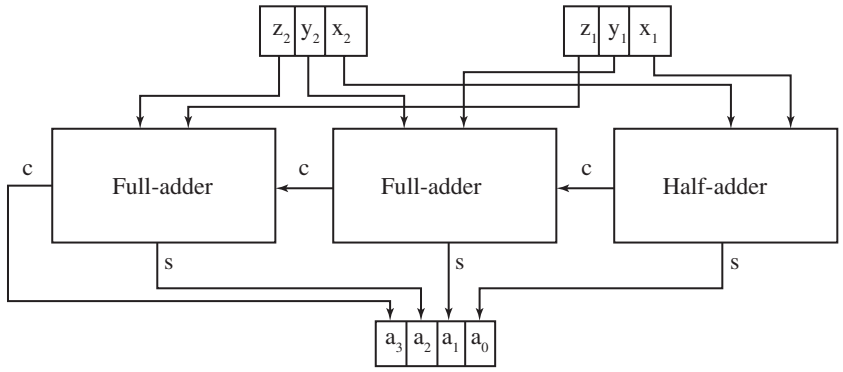


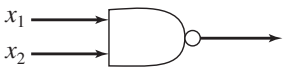
Figure 8.15

The adder circuit shown in Figure 8.15 is called a "ripple-carry adder" because the carry bits have to propagate right to left through each adder in turn. Although we have assumed that gates output instantaneously, there is in fact a small time delay in an n-bit adder due to this ripple effect that can be appreciable for large n. Variations on the basic circuitry that speed up the addition process depend on anticipating the higher-order carry bits.

PRACTICE 12 Trace the operation of the circuit in Figure 8.15 as it adds 101 and 111. ■

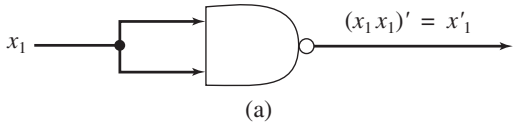
Other Logic Elements

The basic elements used in integrated circuits are not really AND and OR gates and inverters, but NAND and NOR gates. Figure 8.16 shows the standard symbol for the **NAND gate** (the NOT AND gate) and its truth function. The NAND gate alone is sufficient to realize any truth function because networks using only NAND gates can do the job of inverters, OR gates, and AND gates. Figure 8.17 shows these networks.

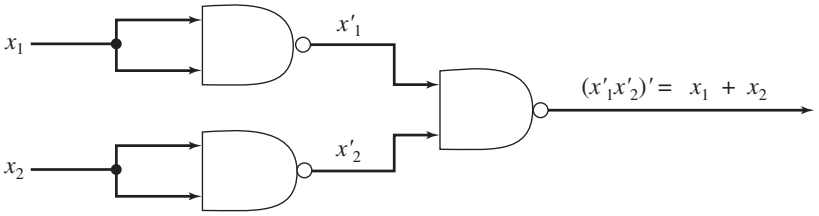


x_1	x_2	$(x_1 + x_2)'$
1	1	0
1	0	1
0	1	1
0	0	1

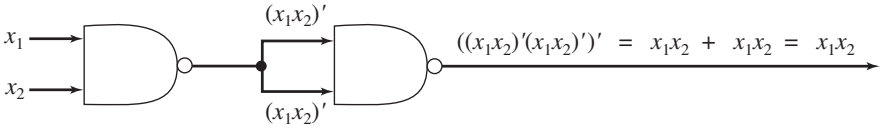
Figure 8.16



(a)



(b)

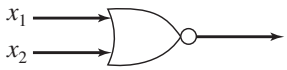


(c)

Figure 8.17

The **NOR gate** (the NOT OR gate) and its truth function appear in Figure 8.18. An exercise at the end of this section asks you to construct networks using only NOR gates for inverters, OR gates, and AND gates.[1]

[1]Exercises 51 and 52 of Section 1.1 give truth tables for binary connectives that agree with the NAND and NOR truth functions. There you were asked to prove that either of these two connectives is sufficient to write any propositional wff; that is, you can write $\vee$, $\wedge$, and $'$ in terms of one of these connectives.



x_1	x_2	$(x_1 + x_2)'$
1	1	0
1	0	0
0	1	0
0	0	1

Figure 8.18

Although we can construct a NAND network for a truth function by replacing AND gates, OR gates, and inverters in the canonical form or a minimized form with the appropriate NAND networks, we can often obtain a simpler network by using the properties of NAND elements directly.

PRACTICE 13

a. Rewrite the network of Figure 8.12 with NAND elements by directly replacing the AND gate, OR gate, and inverter, as in Figure 8.17.
b. Rewrite the Boolean expression $x_1x_3 + x_2'$ for Figure 8.12 using De Morgan's laws, and then construct a network using only two NAND elements. ■

Constructing Truth Functions

We know how to write a Boolean expression and construct a network from a given truth function. Often the truth function itself must first be deduced from the description of the actual problem.

EXAMPLE 18

At a mail-order cosmetics firm, an automatic control device is used to supervise the packaging of orders. The firm sells lipstick, perfume, makeup, and nail polish. As a bonus item, shampoo is included with any order that includes perfume or any order that includes lipstick, makeup, and nail polish. How can we design the logic network that controls whether shampoo is packaged with an order?

The inputs to the network will represent the four items that can be ordered. We label the items

$$x_1 = \text{lipstick}$$
$$x_2 = \text{perfume}$$
$$x_3 = \text{makeup}$$
$$x_4 = \text{nail polish}$$

The value of x_i will be 1 when that item is included in the order and 0 otherwise. The output from the network should be 1 if shampoo is to be packaged with the order and 0 otherwise. The truth table for the circuit appears in Table 8.9. The canonical sum-of-products form for this truth function is lengthy, but the expression

$x_1x_3x_4 + x_2$ also represents the function. Figure 8.19 shows the logic network for this expression.

TABLE 8.9

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	1
1	1	1	0	1
1	1	0	1	1
1	1	0	0	1
1	0	1	1	1
1	0	1	0	0
1	0	0	1	0
1	0	0	0	0
0	1	1	1	1
0	1	1	0	1
0	1	0	1	1
0	1	0	0	1
0	0	1	1	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	0

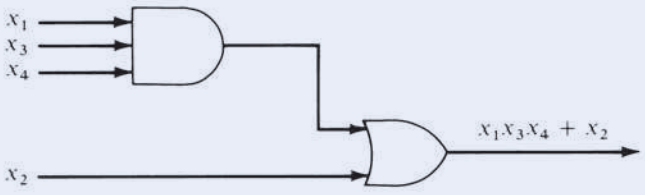


Figure 8.19

PRACTICE 14 A hall light is controlled by two light switches, one at each end of the hall. Find (a) a truth function, (b) a Boolean expression, and (c) a logic network that allows the light to be switched on or off by either switch. ■

In some problems the corresponding truth functions have certain undefined values because certain combinations of input cannot occur (see Exercise 35 at the end of this section). Under these "don't-care" conditions, any value may be assigned to the output.

In a programming language where the Boolean operators AND, OR, and NOT are available, designing the logic of a computer program may consist in part of choosing appropriate truth functions and their corresponding Boolean expressions (see Exercise 36 of Section 1.1).

Pruning Chips and Programs

Computer computations (barring coding errors) are generally viewed as the height of accuracy. Translating truth functions into circuits that produce correct outputs for given inputs has been the theme of this chapter. But some computer applications do not require 100% accuracy and can tolerate a certain amount of error. Image processing is one such application because small variations in an image are imperceptible to the human eye. (This feature of human eyesight is used to advantage in JPEG lossy image compression, as discussed in Section 6.4.)

Recently, researchers looking for increased efficiency in chip design have "pruned" traditional circuits, essentially cutting away the parts that are seldom used. Pruned chips consume less energy and are both smaller and faster than the complete chip. We can lump these three factors—energy, space, and time—together under the general term "efficiency." Experimental pruning of a chip's addition circuit produced a 7.5% gain in overall efficiency. But of course there is a tradeoff; some percentage of error is introduced in the chip's operation. The 7.5% efficiency gain came at the cost of a 0.25% error rate. For specialized applications such as image processing, where small amounts of error can be tolerated, the tradeoff seems to be a good thing. Developers of this technology are looking toward applications such as hearing aids, cameras, or even tablet computers that could run on solar power.

The same approach is also being tried with software. For example, "loop perforation" skips some of the iterations in a loop. This is a particularly evocative name—"perforating a loop" sounds very much like "pruning a chip." Other approaches may skip entire tasks or randomly discard some input values. "Relaxed program" is a general title for software that has built-in nondeterminism that can dynamically prune (skip) some of its instructions or data. The benefit is increased runtime efficiency. The penalty is some percentage of incorrect results. And the trick, of course, is to ensure—and formally verify—that such techniques, while increasing efficiency, keep the output within an acceptable error range.

"Inexact Design—Beyond Fault Tolerance," Anthes, G., *Communications of the ACM*, April, 2013.
http://news.rice.edu/2012/05/17/computing-experts-unveil-superefficient-inexact-chip/
http://web.mit.edu/newsoffice/2010/fuzzy-logic-0103.html
"Proving Acceptability Properties of Relaxed Nondeterministic Approximate Programs," Carbin, M., Kim, D., Misailovic, S., Rinard, M., ACM Conference on Programming Language Design and Implementation, June 11–16, 2012, Beijing, China.
http://web.mit.edu/newsoffice/2012/loop-perforation-0522.html

SECTION 8.2 REVIEW

TECHNIQUES

- Find the truth function corresponding to a given Boolean expression or logic network.
- Construct a logic network with the same truth function as a given Boolean expression.
- Write a Boolean expression with the same truth function as a given logic network.
- Write the Boolean expression in canonical sum-of-products form for a given truth function.
- Find a network composed only of NAND gates that has the same truth function as a given network with AND gates, OR gates, and inverters.
- Find a truth function that satisfies the description of a particular problem.

MAIN IDEAS

- We can effectively convert information from any of the following three forms to any other form:

truth function $\leftrightarrow$ Boolean expression $\leftrightarrow$ logic network

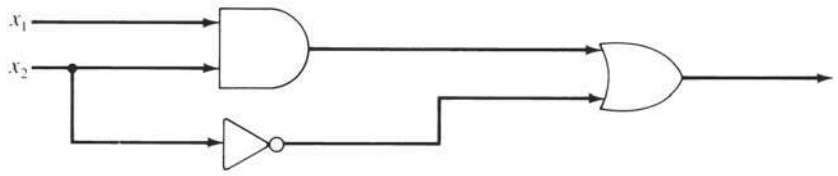
- A Boolean expression can sometimes be converted to a simpler, equivalent expression using the properties of Boolean algebra, thus producing a simpler network for a given truth function.

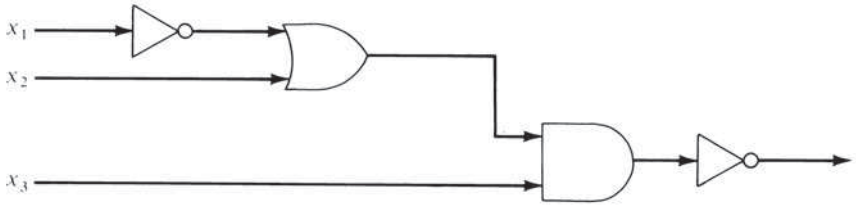
EXERCISES 8.2

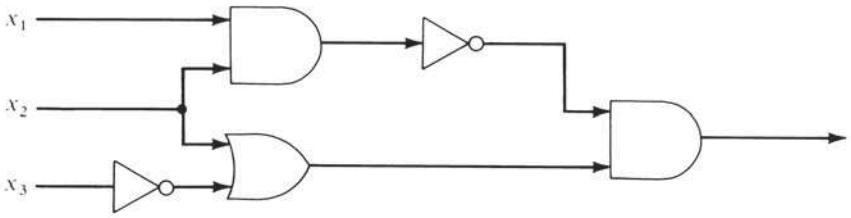
For Exercises 1–4, write a truth function and construct a logic network using AND gates, OR gates, and inverters for each of the given Boolean expressions.

1. $(x_1' + x_2)x_3$
2. $(x_1 + x_2') + x_1'x_3$
3. $x_1'x_2 + (x_1x_2)'$
4. $(x_1 + x_2)'x_3 + x_3'$

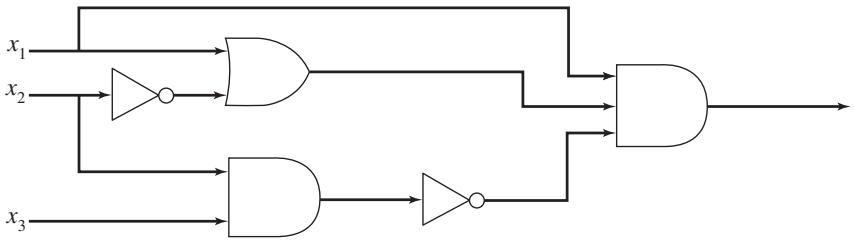
For Exercises 5–8, write a Boolean expression and a truth function for each of the logic networks shown.

5. 

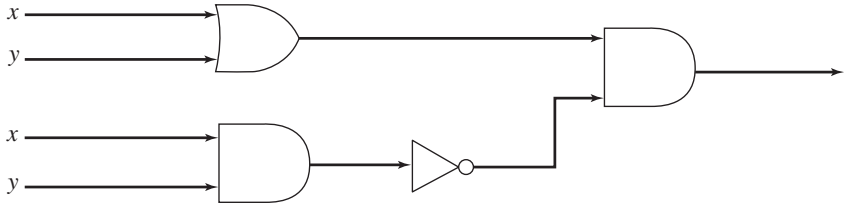
6. 

7. 

8.



9. a. Write the truth function for the Boolean operation $x \oplus y = xy' + yx'$.
   b. Draw the logic network for $x \oplus y$.
   c. Show that the network of the accompanying figure also represents $x \oplus y$. Explain why the network illustrates that $\oplus$ is the **exclusive OR** operation. (Recall that a bitwise exclusive-OR operation is used in DES encoding, as discussed in Section 5.6.)



10. a. Write the truth function for the Boolean expression

$$(xy')'(yx')'$$

   b. Draw the logic network for this expression.
   c. By looking at either the truth function or the logic network, what propositional logic connective does this Boolean expression represent?

For Exercises 11–20, find the canonical sum-of-products form for the truth functions in the given tables.

11.

x_1	x_2	$f(x_1, x_2)$
1	1	0
1	0	0
0	1	0
0	0	1

12.

x_1	x_2	$f(x_1, x_2)$
1	1	1
1	0	0
0	1	1
0	0	0

13.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	0
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	0
0	0	0	0

14.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	0
1	1	0	1
1	0	1	1
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	0
0	0	0	1

15.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	0
1	1	0	0
1	0	1	1
1	0	0	1
0	1	1	0
0	1	0	1
0	0	1	0
0	0	0	0

16.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	0
1	1	0	1
1	0	1	1
1	0	0	0
0	1	1	0
0	1	0	1
0	0	1	0
0	0	0	0

17.

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	1
1	1	1	0	0
1	1	0	1	1
1	1	0	0	0
1	0	1	1	1
1	0	1	0	0
1	0	0	1	1
1	0	0	0	0
0	1	1	1	0
0	1	1	0	0
0	1	0	1	0
0	1	0	0	0
0	0	1	1	1
0	0	1	0	1
0	0	0	1	0
0	0	0	0	0

18.

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	1
1	1	1	0	0
1	1	0	1	1
1	1	0	0	0
1	0	1	1	1
1	0	1	0	1
1	0	0	1	0
1	0	0	0	0
0	1	1	1	1
0	1	1	0	0
0	1	0	1	1
0	1	0	0	0
0	0	1	1	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	0

19.

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	0
1	1	1	0	0
1	1	0	1	0
1	1	0	0	0
1	0	1	1	0
1	0	1	0	1
1	0	0	1	0
1	0	0	0	0
0	1	1	1	1
0	1	1	0	0
0	1	0	1	1
0	1	0	0	0
0	0	1	1	1
0	0	1	0	1
0	0	0	1	1
0	0	0	0	0

20.

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	1
1	1	1	0	1
1	1	0	1	0
1	1	0	0	1
1	0	1	1	1
1	0	1	0	0
1	0	0	1	0
1	0	0	0	0
0	1	1	1	0
0	1	1	0	1
0	1	0	1	0
0	1	0	0	1
0	0	1	1	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	0

21. a. Find the canonical sum-of-products form for the truth function in the accompanying table.
    b. Draw the logic network for the expression of part (a).
    c. Use properties of a Boolean algebra to reduce the expression of part (a) to an equivalent expression whose network requires only two logic elements. Draw the network.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	0
0	0	1	0
0	0	0	0

22. a. Find the canonical sum-of-products form for the truth function in the accompanying table.
    b. Draw the logic network for the expression of part (a).
    c. Use properties of a Boolean algebra to reduce the expression of part (a) to an equivalent expression whose network requires only three logic elements. Draw the network.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	0
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	1
0	0	1	0
0	0	0	0

23. a. Show that the two Boolean expressions

$$(x_1 + x_2)(x_1' + x_3)(x_2 + x_3)$$

and

$$(x_1x_3) + (x_1'x_2)$$

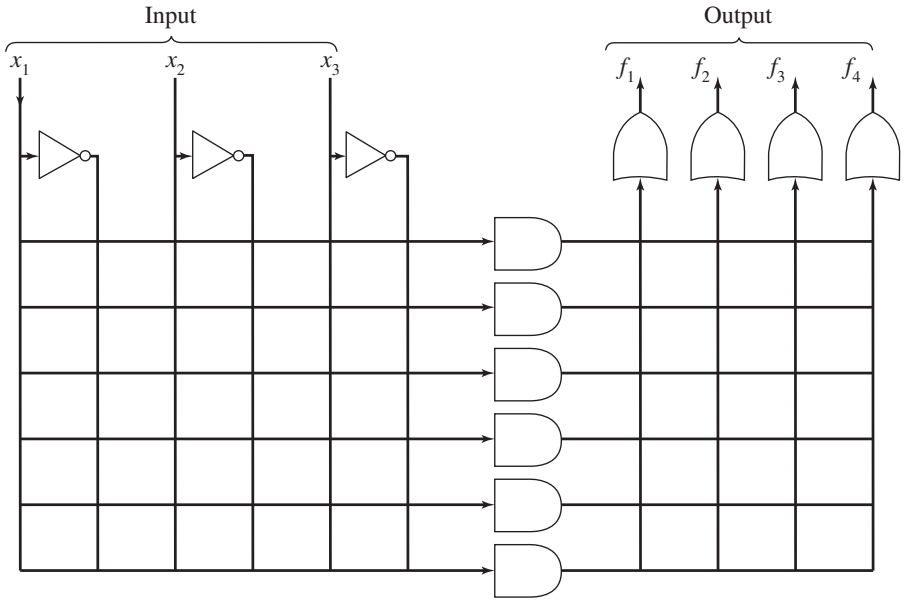
are equivalent by writing the truth function for each.

b. Write the canonical sum-of-products form equivalent to the two expressions of part (a).

c. Use properties of a Boolean algebra to reduce one of the expressions of part (a) to the other.

24. The accompanying figure shows an unprogrammed PLD for three inputs, x_1, x_2, and x_3. Program this PLD to generate the truth functions f_1 and f_3 represented by

$$f_1\colon\ x_1x_2x_3 + x_1'x_2x_3' + x_1'x_2'x_3$$
$$f_3\colon\ x_1x_2'x_3' + x_1'x_2'x_3 + x_1'x_2'x_3'$$



25. There is also a *canonical product-of-sums form* (*conjunctive normal form*) for any truth function. This expression has the form

$$( )( ) \cdots ( )$$

with each factor a sum of the form

$$\alpha + \beta + \cdots + \omega$$

where $\alpha = x_1$ or x_1', $\beta = x_2$ or x_2', and so on. Each factor is constructed to have a value of 0 for the input values of exactly one of the rows of the truth function having value 0. Thus, the entire expression has value 0 for these inputs and no others. Find the canonical product-of-sums form for the truth functions of Exercises 11–15.

26. Consider the truth function given here. Because there are many more 1*s* than 0*s*. the canonical sum-of-products form might seem tedious to compute. Instead, create a Boolean sum-of-products expression using the same formula as before but on the 0 rows instead of the 1 rows. This expression will give outputs of 1*s* for those rows and no others, exactly the opposite of what you want. Then complement the expression (equivalent to sticking an inverter on the end of the network).

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	1
1	0	1	1
1	0	0	0
0	1	1	1
0	1	0	1
0	0	1	0
0	0	0	1

a. Use this approach to create a Boolean expression for this truth function.

b. Prove that the resulting expression is equivalent to the canonical product-of-sums form described in Exercise 25.

27. The *2's complement* of an n-bit binary number p is an n-bit binary number q such that $p + q$ equals an n-bit representation of zero (any carry bit to column $n + 1$ is ignored). Thus 01110 is the 2's complement of 10010 because

$$\begin{array}{r} 10010 \\ +\ 01110 \\ \hline (1)00000 \end{array}$$

The 2's complement idea can be used to represent negative integers in binary form. After all, the negative of p is by definition a number that, when added to p, results in zero.

Given a binary number p, the 2's complement of p is found by scanning p from low-order to high-order bits (right to left). As long as bit i of p is 0, bit i of q is 0. When the first 1 of p is encountered, say at bit j, then bit j of q is 1, but for the remaining bits, $j < i \leq n$, $q_i = p_i'$. For $p = 10010$, for instance, the

rightmost 0 bit of p stays a 0 bit in q, and the first 1 bit stays a 1 bit. The remaining bits of q, however, are the reverse of the bits in p.

$$\begin{array}{rcl} & & \text{First 1} \\ & & \swarrow \\ p = & 100 & 10 \\ q = & 011 & 10 \\ & q_i = p_i' & q_i = p_i \end{array}$$

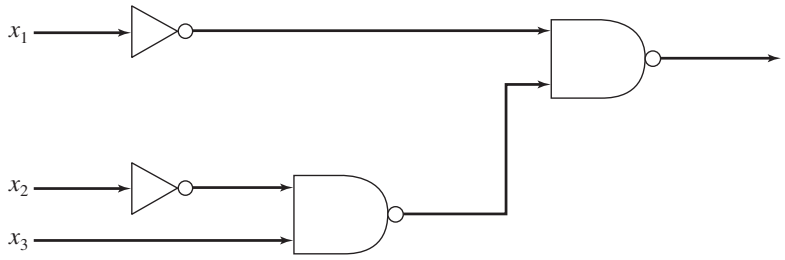
For each binary number p, find the 2's complement of p, namely q, and then calculate $p + q$.

a. 1100 b. 1001 c. 001

28. For any bit x_i in a binary number p, let r_i be the corresponding bit in q, the 2's complement of p (see Exercise 27). The value of r_i depends on the value of x_i and also on the position of x_i relative to the first 1 bit in p. For the ith bit, let c_{i-1} denote a 0 if the bits p_j, $1 \leq j \leq i - 1$, are 0 and a 1 otherwise. A value c_i must be computed to move on to the next bit.
    a. Give a truth function for r_i with inputs x_i and c_{i-1}. Give a truth function for c_i with inputs x_i and c_{i-1}.
    b. Write Boolean expressions for the truth functions of part (a). Simplify as much as possible.
    c. Design a circuit module to output r_i and c_i from inputs x_i and c_{i-1}.
    d. Using the modules of part (c), design a circuit to find the 2's complement of a three-bit binary number zyx. Trace the operation of the circuit in computing the 2's complement of 110.
29. a. Construct a network for the following expression using only NAND elements. Replace the AND and OR gates and inverters with the appropriate NAND networks.

$$x_3'x_1 + x_2'x_1 + x_3'$$

    b. Use the properties of a Boolean algebra to reduce the expression of part (a) to one whose network would require only three NAND gates. Draw the network.
30. Replace the following network with an equivalent network using one AND gate, one OR gate, and one inverter.



31. Using only NOR elements, construct networks that can replace (a) an inverter, (b) an OR gate, and (c) an AND gate.
32. Find an equivalent network for the half-adder module that uses exactly five NAND gates. Draw the network.
33. A thermostat controls a heating system that should raise the temperature above 67°F during working hours (between 7:00 AM and 6:00 PM). The input values are

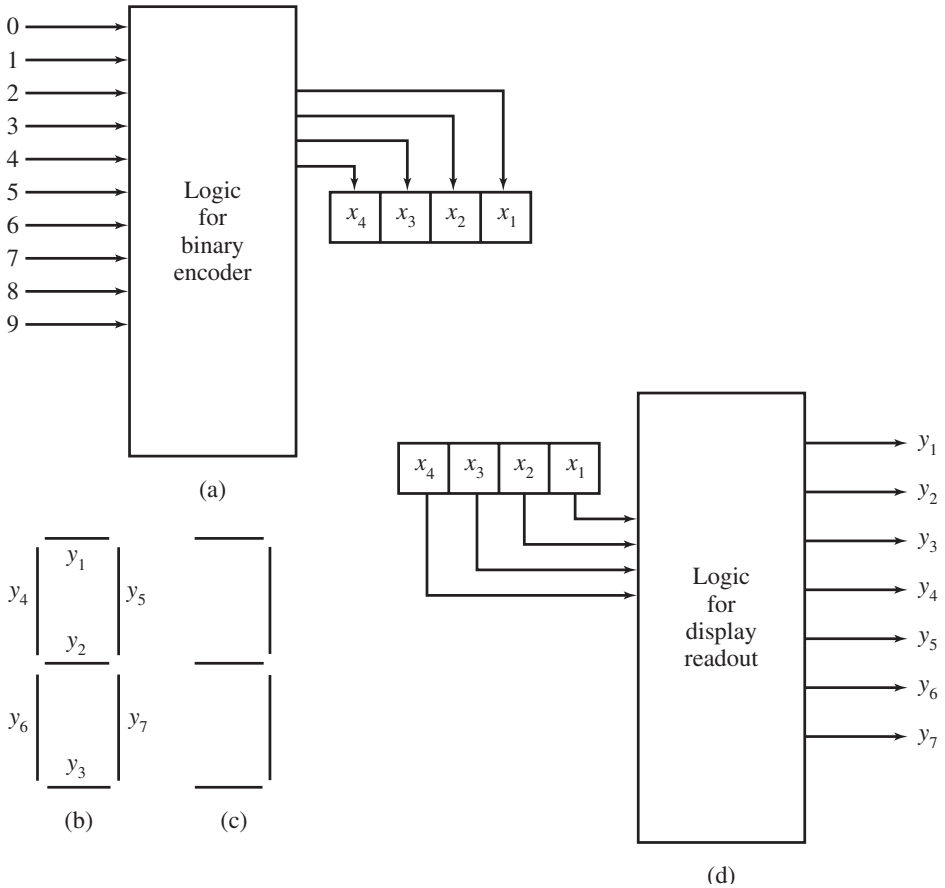
$x_1 = 1$ when the temperature is less than 67°F
$x_1 = 0$ otherwise
$x_2 = 1$ when the time is less than 7:00 AM or greater than 6:00 PM
$x_2 = 0$ otherwise

Find a truth function, a Boolean expression, and a logic network for when the heating system should be on (value 1) or off (value 0).

34. The space shuttle was controlled by three on-board computers; binary output from these three computers was compared and a majority vote was required for certain actions to take place. Find a truth function, a Boolean expression, and a logic network that outputs the majority vote of the three input values.

35. You have just been hired at Mercenary Motors. Your job is to design a logic network so that a car can be started only when the automatic transmission is in neutral or park and the driver's seat belt is fastened. Find a truth function, a Boolean expression, and a logic network. (There is a don't-care condition to the truth function, since the car cannot be in both neutral and park.)

36. Mercenary Motors has expanded into the calculator business. You need to design the circuitry for the display readout on a new calculator. This design involves a two-step process.
    a. Any digit 0, 1, … , 9 put into the calculator is first converted to binary form. Part (a) of the accompanying figure illustrates this conversion, which involves four separate networks, one each for x_1 to x_4. Each network has 10 inputs, but only 1 input can be on at any given moment. Write a Boolean expression and then draw a network for x_2.
    b. The binary form of the digit is then converted into a visual display by activating a pattern of seven outputs arranged as shown in figure (b). To display the digit 3, for example, y_1, y_2, y_3, y_5, and y_7 must be on, as in figure (c). Thus, the second step of the process can be represented by figure (d), which involves seven separate networks, one each for y_1 to y_7, each with four inputs, x_1 to x_4. Write a truth function, a Boolean expression, and a network for y_5 and for y_6.



37. A *multiplexor* is a control circuit with 2^n input lines (numbered 0 through $2^n - 1$), n selector lines, and exactly one output line. The signal on the output line is to match the signal on one of the input lines; the selector lines determine which of the input line signals will be propagated to the output line. Signals on the n selector lines determine an n-bit binary number, which can range in value from 0 to $2^n - 1$; hence the numeric value on the selector lines identifies exactly one input line. The arithmetic-logic unit of a computer processor contains circuits for various operations (addition, subtraction, comparison, and so forth), and when an arithmetic or comparison operation is to be done, the ALU may activate all these circuit. A multiplexor then picks out the one desired result.
    a. Write a truth function for a multiplexor where $n = 1$; that is, there are 2 input lines, 1 selector line and, of course, one output line.
    b. Draw the logic network (as simple as possible).
38. A *decoder* is a control circuit with n input lines and 2^n output lines (numbered 0 through $2^n - 1$). The pattern of n bits on the input lines represents a binary number between 0 and $2^n - 1$. A decoder activates the output line with the corresponding identification number by putting a 1 output on that line and 0 outputs on all other output lines. A decoder in a computer can, for example, read input lines that represent a binary memory address and then activate the line to that memory cell for a read operation.
    a. Write truth functions for a decoder where $n = 2$, that is, there are 2 input lines and $2^2 = 4$ output lines (hence four truth functions).
    b. Draw the logic network that incorporates all four truth functions.
39. At the beginning of this chapter, you were

    > . . . hired by Rats R Us to build the control logic for the production facilities for a new anticancer chemical compound being tested on rats. The control logic must manage the opening and closing of two valves, *A* and *B*, downstream of the mixing vat. Valve *A* is to open whenever the pressure in the vat exceeds 50 psi (pounds per square inch) and the salinity of the mixture exceeds 45 g/L (grams per liter). Valve *B* is to open whenever valve *A* is closed and the temperature exceeds 53°C and the acidity falls below 7.0 pH (lower pH values mean more acidity).
    >
    > How many and what type of logic gates will be needed in the circuit?

    Answer this question by finding the canonical sum-of-products form for the logic circuits to control *A* and *B*.

SECTION 8.3 MINIMIZATION

Minimization Process

Remember from Section 8.2 that a given truth function is associated with an equivalence class of Boolean expressions. If we want to design a logic network for the function, the ideal would be to have a procedure that chooses the simplest Boolean expression from the class. What we consider simple will depend on the technology employed in building the network, what kind of logic elements are available, and so on. At any rate, we probably want to minimize the total number of connections that must be made and the total number of logic elements used. (As we discuss minimization procedures, keep in mind that other factors may influence the economics of the situation. If a network is to be built only once, the time spent on minimization is costlier than building the network. But if the network is to be mass-produced, then the cost of minimization time may be worthwhile.)

We have had some experience in simplifying Boolean expressions by applying the properties of Boolean algebra. However, we had no procedure to use. We simply had to guess, attacking each problem individually. What we want now is a mechanical procedure that we can use without having to be clever or insightful. Unfortunately, we won't develop the ideal procedure. However, we already know how to select the canonical sum-of-products form from the equivalence class of expressions for a given truth function. In this section we will discuss two procedures to reduce a canonical sum-of-products form to a minimal sum-of-products form. Therefore, we can minimize within the framework of a sum-of-products form and reduce, if not completely minimize, the number of elements and connections required.

EXAMPLE 19 The Boolean expression

$$x_1x_2x_3 + x_1'x_2x_3 + x_1'x_2x_3'$$

is in sum-of-products form. An equivalent minimal sum-of-products form is

$$x_2x_3 + x_2x_1'$$

Implementing a network for this form would require two AND gates, one OR gate, and an inverter. Using one of the distributive laws of Boolean algebra, this expression reduces to

$$x_2(x_3 + x_1')$$

which requires only one AND gate, one OR gate, and an inverter, but it is no longer in sum-of-products form. Thus, a minimal sum-of-products form may not be minimal in an absolute sense. ●

There are two extremely useful equivalences in minimizing a sum-of-products form. They are

$$x_1x_2 + x_1'x_2 = x_2$$

and

$$x_1 + x_1'x_2 = x_1 + x_2$$

PRACTICE 15 Use properties of Boolean algebra to reduce the following expressions as indicated:

a. $x_1x_2 + x_1'x_2$ to x_2
b. $x_1 + x_1'x_2$ to $x_1 + x_2$ ■

The equivalence $x_1x_2 + x_1'x_2 = x_2$ means, for example, that the expression $x_1'x_2x_3'x_4 + x_1'x_2'x_3'x_4$ reduces to $x_1'x_3'x_4$. Thus, when we have a sum of two products that differ in only one factor, we can eliminate that factor. However, the canonical

sum-of-products form for a truth function of, say, four variables might be quite long and require some searching to locate two product terms differing by only one factor. To help us in this search, we can use the *Karnaugh map*. The Karnaugh map is a visual representation of the truth function so that terms in the canonical sum-of-products form that differ by only one factor can be matched quickly.

Karnaugh Map

In the canonical sum-of-products form for a truth function, we are interested in values of the input variables that produce outputs of 1. The Karnaugh map records the 1*s* of the function in an array that forces products of inputs differing by only one factor to be adjacent. The array form for a two-variable function is given in Figure 8.20. Notice that the square corresponding to x_1x_2, the upper left-hand square, is adjacent to squares $x_1'x_2$ and x_1x_2', which differ in one factor from x_1x_2; however, it is not adjacent to the $x_1'x_2'$ square, which differs in two factors from x_1x_2.

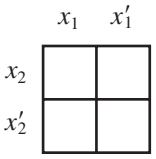


Figure 8.20

EXAMPLE 20 The truth function of Table 8.10 is represented by the Karnaugh map of Figure 8.21. At once we can observe 1*s* in two adjacent squares, so there are two terms in the canonical sum-of-products form differing by one variable; again from the map, we see that the variable that changes is x_1. It can be eliminated. We conclude that the function can be represented by x_2. Indeed, the canonical sum-of-products form for the function is $x_1x_2 + x_1'x_2$, which, by our basic reduction rule, reduces to x_2. However, we did not have to write the canonical form—we only had to look at the map.

TABLE 8.10

x_1	x_2	$f(x_1, x_2)$
1	1	1
1	0	0
0	1	1
0	0	0

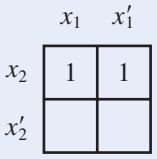


Figure 8.21

PRACTICE 16 Draw the Karnaugh map and use it to find a reduced expression for the function in Table 8.11.

TABLE 8.11

x_1	x_2	$f(x_1, x_2)$
1	1	0
1	0	0
0	1	1
0	0	1

Maps for Three and Four Variables

The array forms for functions of three and four variables are shown in Figure 8.22. In these arrays, adjacent squares also differ by only one variable. However, in Figure 8.22a, the leftmost and rightmost squares in a row also differ by one variable, so we consider them adjacent. (They would in fact be adjacent if we wrapped the map around a cylinder and glued the left and right edges together.) In Figure 8.22b, the leftmost and rightmost squares in a row are adjacent (differ by exactly one variable), and also the top and bottom squares in a column are adjacent.

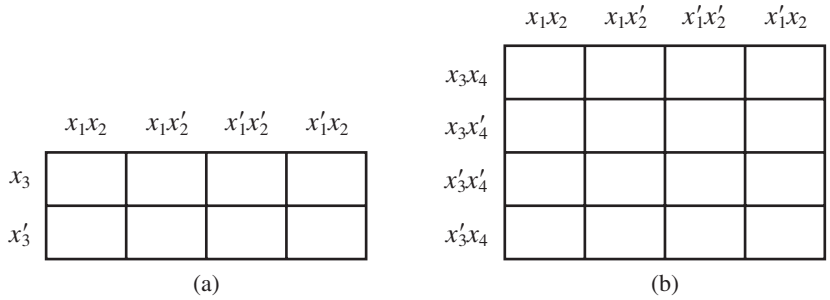


Figure 8.22

REMINDER

It is crucial to label the Karnaugh map so that adjacent squares differ by one variable.

In three-variable maps, when two adjacent squares are marked with 1, one variable can be eliminated; when four adjacent squares are marked with 1 (either in a single row or arranged in a square), two variables can be eliminated.

The labelings for two-, three-, and four-variable maps can be done in various ways, but they must be done so that adjacent squares differ by one variable. It's probably best to just memorize the labeling schemes we've used here.

EXAMPLE 21

In the map of Figure 8.23, the squares that combine for a reduction are shown as a block. These four adjacent squares reduce to x_3 (eliminate the changing variables x_1 and x_2). The reduction uses our basic reduction rule more than once:

$$\begin{aligned} x_1x_2x_3 + x_1x_2'x_3 + x_1'x_2'x_3 + x_1'x_2x_3 &= x_1x_3(x_2 + x_2') + x_1'x_3(x_2' + x_2) \\ &= x_1x_3 + x_1'x_3 \\ &= x_3(x_1 + x_1') \\ &= x_3 \end{aligned}$$

But, again, you don't have to go through this process; you can just look at the Karnaugh map in Figure 8.23.

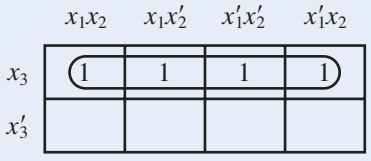


Figure 8.23

In four-variable maps, when two adjacent squares are marked with 1, one variable can be eliminated; when four adjacent squares are marked with 1, two variables can be eliminated; when eight adjacent squares are marked with 1, three variables can be eliminated.

Figure 8.24 illustrates some instances of two adjacent marked squares, Figure 8.25 illustrates some instances of four adjacent marked squares, and Figure 8.26 shows instances of eight.

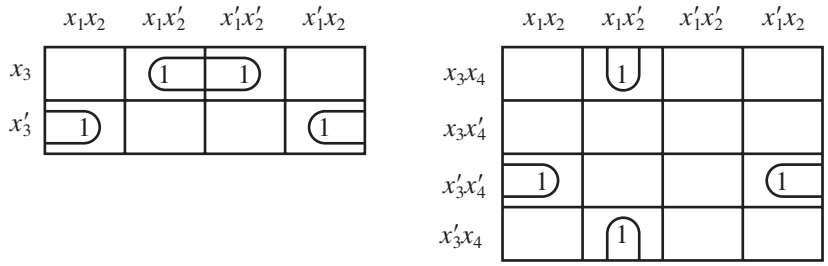


Figure 8.24

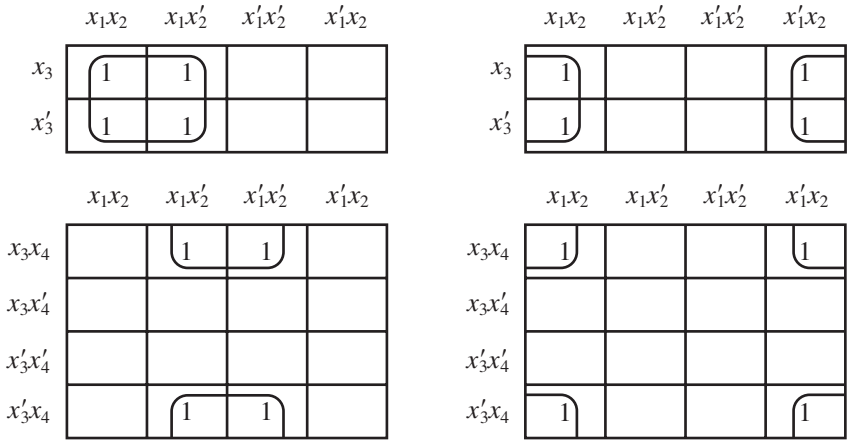


Figure 8.25

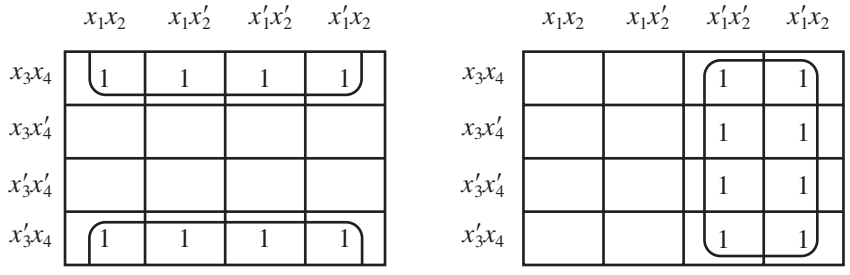


Figure 8.26

EXAMPLE 22 In the map of Figure 8.27, the four outside corners reduce to x_2x_4 and the inside square reduces to $x_2'x_4'$.

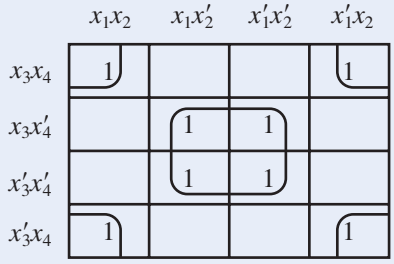


Figure 8.27

PRACTICE 17 Find the two terms represented by the map in Figure 8.28.

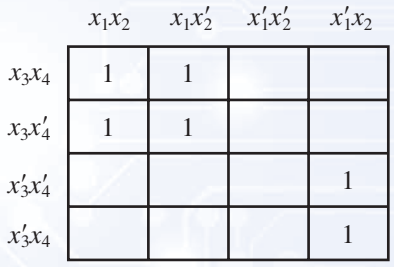


Figure 8.28

Using the Karnaugh Map

How do we find a minimal sum-of-products form from a Karnaugh map (or from a truth function or a canonical sum-of-products form)? We must use every marked square of the map, and we want to include every marked square in the largest combination of marked squares possible, since doing so will reduce the expression as much as possible. Surprisingly, we cannot begin by simply looking for the largest blocks of marked squares on the map.

EXAMPLE 23 In the Karnaugh map of Figure 8.29, if we simply looked for the largest block of marked squares, we would use the column of 1*s* and reduce it to $x_1'x_2'$. However, we would still have four marked squares unaccounted for. Each of these marked squares can be combined into a two-square block in only one way (Figure 8.30), and each of these blocks has to be included. But when this adjustment is made, every square in the column of 1*s* is used, and the term $x_1'x_2'$ is superfluous. The minimal sum-of-products form for this map becomes

$$x_2'x_3x_4 + x_1'x_3x_4' + x_2'x_3'x_4' + x_1'x_3'x_4$$

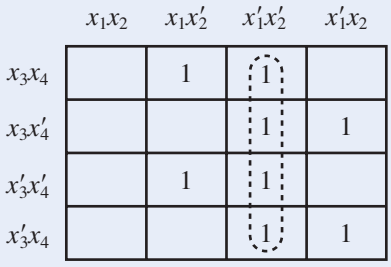


Figure 8.29

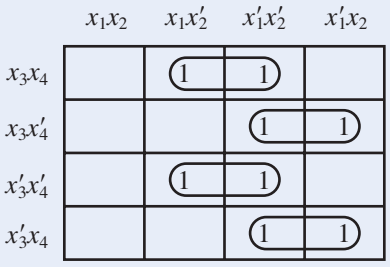


Figure 8.30

To avoid the redundancy illustrated by Example 23, we analyze the map as follows. First, we form terms for those marked squares that cannot be combined with anything. Then we use the remaining marked squares to find those that can be combined only into two-square blocks and in only one way. Then among the unused marked squares—that is, those not already assigned to a block—we find those that can be combined only into four-square blocks and in only one way; then we look for any unused squares that go uniquely into eight-square blocks. At each step, if an unused marked square can go into more than one block, we do nothing with it. Finally, we take any unused marked squares that are left (for which there was a choice of blocks) and select blocks that include them in the most efficient manner.

Table 8.12 shows the steps involved. Note, however, that this procedure for handling Karnaugh maps is not, strictly speaking, an algorithm because it doesn't always produce the correct result. If there are many 1*s* in the map, thus allowing many different blockings, even this procedure may not lead to a minimal form (see Example 28).

TABLE 8.12

Steps in Using Karnaugh Maps
1. Set up the grid, using correct labeling for the number of Boolean variables.
2. Insert 1*s* in the table for the terms in the canonical sum-of-products expression.
3. Form terms for any isolated marked squares.
4. Combine squares uniquely into two-square blocks, if possible.
5. Combine squares uniquely into four-square blocks, if possible.
6. Combine squares uniquely into eight-square blocks, if possible.
7. Combine any remaining unused marked squares into blocks as efficiently as possible.

EXAMPLE 24 In Figure 8.31a we show the only square that cannot be combined into a larger block. In Figure 8.31b, we have formed the unique two-square block for the $x_1x_2'x_3'$ square and the unique two-square block for the $x_1'x_2'x_3$ square. All marked squares are covered. The minimal sum-of-products expression is

$$x_1x_2x_3 + x_2'x_3' + x_1'x_2'$$

Formally, the last two terms are obtained by expanding $x_1'x_2'x_3'$ into $x_1'x_2'x_3' + x_1'x_2'x_3'$ and then combining it with each of its neighbors.

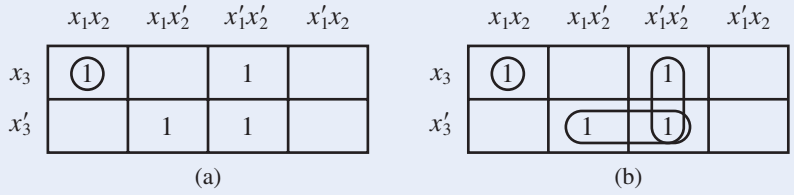


Figure 8.31

EXAMPLE 25

Figure 8.32a shows the unique two-square blocks for the $x_1'x_2'x_3x_4$ square and the $x_1x_2'x_3'x_4'$ square. In Figure 8.32b the two unused squares have been combined into a unique four-square block. The minimal sum-of-products expression is

$$x_1x_3 + x_2'x_3x_4 + x_1x_2'x_4'$$

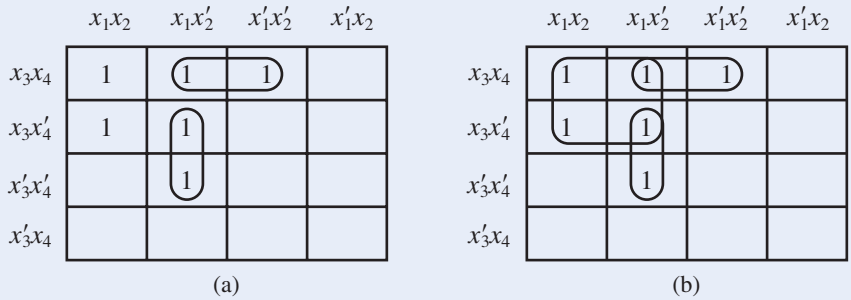


Figure 8.32

EXAMPLE 26

Figure 8.33a shows the unique two-square blocks. We can assign the remaining unused marked square to either of two different two-square blocks; these blocks are shown in Figure 8.33b. There are two minimal sum-of-products forms,

$$x_1x_2'x_4' + x_1'x_2x_3 + x_2'x_3x_4'$$

and

$$x_1x_2'x_4' + x_1'x_2x_3 + x_1'x_3x_4'$$

Either can be used, as they are equally efficient.

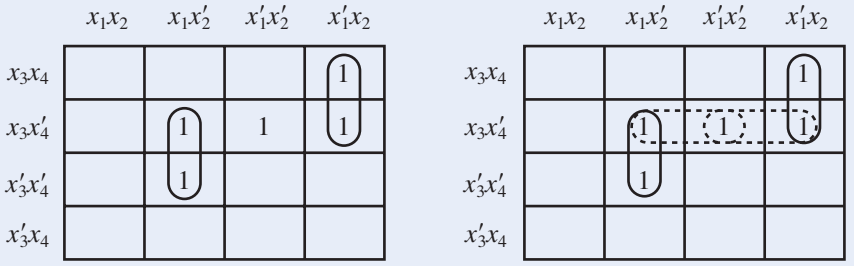


Figure 8.33

EXAMPLE 27

Figure 8.34a shows the unique two-square and four-square blocks. The remaining two unused marked squares can be assigned to two-square blocks in two different ways, as shown in parts (b) and (c). Assigning them together to a single two-square block is more efficient because it produces a sum-of-products form with three terms rather than four. The minimal sum-of-products expression is

$$x_1x_3 + x_1'x_2x_3' + x_2'x_3'x_4'$$

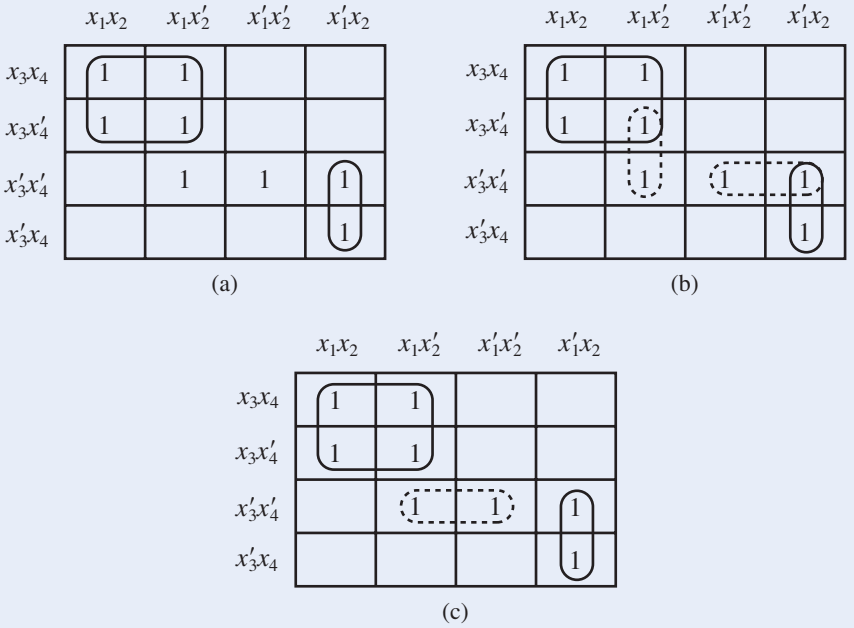


Figure 8.34

EXAMPLE 28

Consider the map of Figure 8.35a. Here the two unique four-square blocks determined by the squares with $*$ have been chosen. In Figure 8.35b, the remaining unmarked squares, for which there was a choice of blocks, are combined into blocks as efficiently as possible. The resulting sum-of-products form is

$$x_1x_3 + x_1'x_3' + x_3x_4 + x_1'x_2 + x_1x_2'x_4'$$

Yet in Figure 8.35c, choosing a different four-square block at the top leads to a simpler sum-of-products form,

$$x_2x_3 + x_1'x_3' + x_3x_4 + x_1x_2'x_4'$$

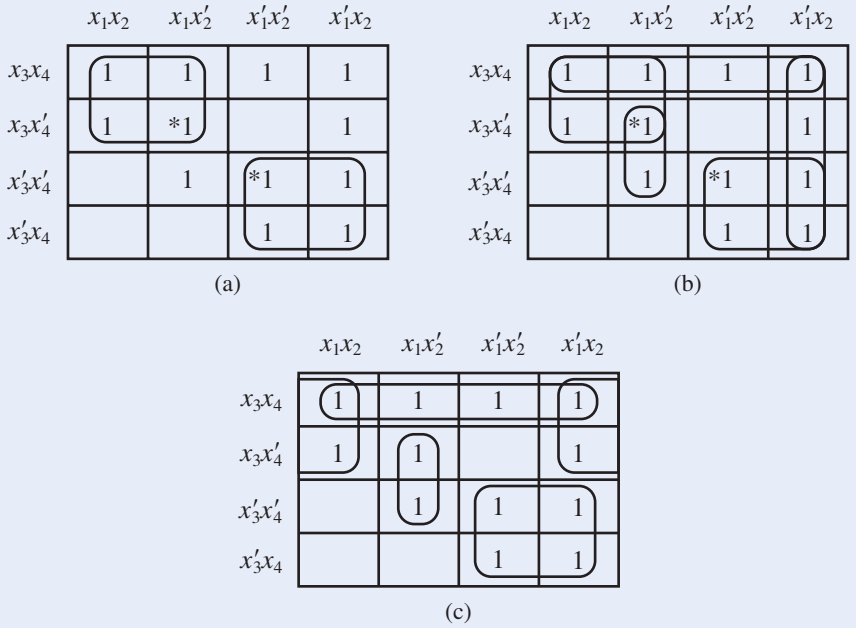


Figure 8.35

PRACTICE 18 Write the minimal sum-of-products expression for the map shown in Figure 8.36.

Figure 8.36

If the Karnaugh map corresponds to a function with don't-care conditions, then the don't-care squares on the map can be left blank or assigned the value 1, whichever aids the minimization process.

We have used Karnaugh maps for functions of two, three, and four variables. By using three-dimensional drawings or overlapping transparency sheets, Karnaugh maps for functions of five, six, or even more variables can be constructed, but the visualization gets too complicated to be worthwhile. The next procedure works for any number of variables.

Quine–McCluskey Procedure

Remember that the key to reducing the canonical sum-of-products form for a truth function lies in recognizing terms of the sum that differ in only one factor. In the Karnaugh map, we see where such terms occur. A second method of reduction, the *Quine–McCluskey procedure*, organizes information from the canonical sum-of-products form into a table to simplify the search for terms differing by only one factor.

The procedure is a two-step process paralleling the use of the Karnaugh map. First we find groupings of terms (just as we looped together marked squares in the Karnaugh map); then we eliminate redundant groupings and make choices for terms that can belong to several groups.

EXAMPLE 29

Let's illustrate the Quine–McCluskey procedure by using the truth function for Example 23. We did not write the actual truth function there, but the information is contained in the Karnaugh map. The truth function is shown in Table 8.13. The eight 4-tuples of 0*s* and 1*s* producing a function value of 1 are listed in Table 8.14, which is separated into four groupings according to the number of 1*s*. Note that terms of the canonical sum-of-products form differing by only one factor must be in adjacent groupings, which simplifies the search for such terms.

TABLE 8.13

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	0
1	1	1	0	0
1	1	0	1	0
1	1	0	0	0
1	0	1	1	1
1	0	1	0	0
1	0	0	1	0
1	0	0	0	1
0	1	1	1	0
0	1	1	0	1
0	1	0	1	1
0	1	0	0	0
0	0	1	1	1
0	0	1	0	1
0	0	0	1	1
0	0	0	0	1

TABLE 8.14

Number of 1s	x_1	x_2	x_3	x_4
Three	1	0	1	1
Two	0	1	1	0
	0	1	0	1
	0	0	1	1
One	1	0	0	0
	0	0	1	0
	0	0	0	1
None	0	0	0	0

We compare the first term, 1011, with each of the three terms of the second group, 0110, 0101, and 0011, to locate terms differing by only one factor. Such a term is 0011. The combination 1011 and 0011 reduces to −011 when the changing variable x_1 is eliminated. We will write this reduced term with a dash in the x_1 position in the first row of a new table. The new table is Table 8.15b, where we've just seen how the first row was obtained. We've rewritten the original Table 8.14 as Table 8.15a, but we have also marked the two terms 1011 and 0011 in this table with a superscript 1. This superscript 1 is a pointer that indicates the row number of the reduced term in Table 8.15b that is formed from these two terms (numbering terms corresponds to putting loops in the Karnaugh map).

TABLE 8.15

Number of 1s	x_1	x_2	x_3	x_4	
Three	1	0	1	1	1
Two	0	1	1	0	2
	0	1	0	1	3
	0	0	1	1	1, 4, 5
One	1	0	0	0	6
	0	0	1	0	2, 4, 7
	0	0	0	1	3, 5, 8
None	0	0	0	0	6, 7, 8

(a)

Number of 1s	x_1	x_2	x_3	x_4
Two	–	0	1	1
One	0	–	1	0
	0	–	0	1
	0	0	1	–
	0	0	–	1
None	–	0	0	0
	0	0	–	0
	0	0	0	–

(b)

We continue this process with all the terms in Table 8.15a. A numbered term may still be used in other combinations, just as a marked square in a Karnaugh map can be in more than one loop. When we are done, the result is the completed Table 8.15b shown, where the terms in this table are again grouped by the number of 1*s*.

We now build still another table by processing the terms in Table 8.15b. Here not only the groupings but also the dashes help organize the search process, since terms differing by only one variable must have dashes in the same location. Tables 8.16a and 8.16b are the same as Tables 8.15a and 8.15b, and Table 8.16c is the new table. Again, numbers on terms in Table 8.16b that combine serve as pointers to the reduced terms in Table 8.16c. When we have processed all the terms in Table 8.16b, the reduction process cannot be continued. The unnumbered terms are irreducible, so they represent the possible maximum-sized loops on a Karnaugh map.

TABLE 8.16

Number of 1s	x_1	x_2	x_3	x_4	
Three	1	0	1	1	1
Two	0	1	1	0	2
	0	1	0	1	3
	0	0	1	1	1, 4, 5
One	1	0	0	0	6
	0	0	1	0	2, 4, 7
	0	0	0	1	3, 5, 8
None	0	0	0	0	6, 7, 8

(a)

Number of 1s	x_1	x_2	x_3	x_4	
Two	–	0	1	1	
One	0	–	1	0	
	0	–	0	1	
	0	0	1	–	1
	0	0	–	1	1
None	–	0	0	0	
	0	0	–	0	1
	0	0	0	–	1

(b)

Number of 1s	x_1	x_2	x_3	x_4
None	0	0	–	–

(c)

For the second step of the process, we compare the original terms with the irreducible terms. We form a table with the original terms as column headers and the irreducible terms (the unnumbered terms in the reduction tables just constructed) as row labels. A check in the comparison table (Table 8.17) indicates that the original term in that column eventually led to the irreducible term in that row, which can be determined by following the pointers.

TABLE 8.17

	1011	0110	0101	0011	1000	0010	0001	0000
–011	✓			✓				
0–10		✓				✓		
0–01			✓				✓	
–000					✓			✓
00––				✓		✓	✓	✓

If a column in the comparison table has a check in only one row, the irreducible term for that row is the only one covering the original term, so it is an essential term and must appear in the final sum-of-products form. Thus, we see from Table 8.17 that the terms –011, 0–10, 0–01, and –000 are essential and must be in the final expression. We also note that all columns with a check in row 5 also have checks in another row and so are covered by an essential reduced term already in the expression. Thus, 00–– is redundant. As in Example 23, the minimal sum-of-products form is

$$x_2'x_3x_4 + x_1'x_3x_4' + x_1'x_3'x_4 + x_2'x_3'x_4'$$

In situations where there is more than one minimal sum-of-products form, the comparison table will have nonessential, nonredundant reduced terms. A selection must be made from these reduced terms to cover all columns not covered by essential terms.

EXAMPLE 30

We will use the Quine–McCluskey procedure on the problem presented in Example 26. The reduction tables are given in Table 8.18, and the comparison table appears in Table 8.19.

TABLE 8.18

Number of 1s	x_1	x_2	x_3	x_4	
Three	0	1	1	1	1
Two	1	0	1	0	2, 3
	0	1	1	0	1, 4
One	0	0	1	0	2, 4
	1	0	0	0	3

(a)

Number of 1s	x_1	x_2	x_3	x_4
Two	0	1	1	–
One	–	0	1	0
	1	0	–	0
	0	–	1	0

(b)

TABLE 8.19

	0111	1010	0110	0010	1000
011–	✓		✓		
–010		✓		✓	
10–0		✓			✓
0–10			✓	✓	

We see from the comparison table that 011– and 10–0 are essential reduced terms and that there are no redundant terms. The only original term not covered by essential terms is 0010, column 4, and the choice of the reduced term for row 2 or for row 4 will cover it. Thus, the minimal sum-of-products form, as before, is

$$x_1'x_2x_3 + x_1x_2'x_4' + x_2'x_3x_4'$$

or

$$x_1'x_2x_3 + x_1x_2'x_4' + x_1'x_3x_4'$$

PRACTICE 19 Use the Quine–McCluskey procedure to find a minimal sum-of-products form for the truth function in Table 8.20.

TABLE 8.20

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	1
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	0
0	0	1	1
0	0	0	1

■

While the Quine–McCluskey procedure applies to truth functions with any number of input variables, for a large number of variables, the procedure is extremely tedious to do by hand. However, it is exactly the kind of systematic, mechanical process that lends itself to a computerized solution. In contrast, Karnaugh maps make use of the human ability to quickly recognize visual patterns.

If the truth function f has few 0 values and a large number of 1 values, it may be simpler to implement the Quine–McCluskey procedure for the complement of the function, f', which will have 1 values where f has 0 values, and vice versa. Once a minimal sum-of-products expression is obtained for f', it can be complemented to obtain an expression for f, although the new expression will not be in sum-of-products form. (In fact, by De Morgan's laws, it will be equivalent to a product-of-sums form.) We can obtain the network for f from the sum-of-products network for f' by tacking an inverter on the end.

The whole object of minimizing a network is to simplify the internal configuration while preserving the external behavior. In Chapter 9 we will attempt the same sort of minimization on finite-state machine structures.

SECTION 8.3 REVIEW

TECHNIQUES

- Minimize the canonical sum-of-products form for a truth function by using a Karnaugh map.
- Minimize the canonical sum-of-products form for a truth function by using the Quine–McCluskey procedure.

MAIN IDEA

- Algorithms exist for reducing a canonical sum-of-products form to a minimized sum-of-products form.

EXERCISES 8.3

For Exercises 1–8, write the minimal sum-of-products form for the Karnaugh maps of the given figures.

1.

	x_1x_2	x_1x_2'	$x_1'x_2'$	$x_1'x_2$
x_3			1	1
x_3'	1	1		1

2.

	x_1x_2	x_1x_2'	$x_1'x_2'$	$x_1'x_2$
x_3	1			1
x_3'		1		

3.

	x_1x_2	x_1x_2'	$x_1'x_2'$	$x_1'x_2$
x_3	1	1	1	1
x_3'	1			1

4.

	x_1x_2	x_1x_2'	$x_1'x_2'$	$x_1'x_2$
x_3				
x_3'	1		1	1

5.

	x_1x_2	x_1x_2'	$x_1'x_2'$	$x_1'x_2$
x_3x_4		1		
x_3x_4'		1	1	1
$x_3'x_4'$	1	1	1	
$x_3'x_4$		1		

6.

	x_1x_2	x_1x_2'	$x_1'x_2'$	$x_1'x_2$
x_3x_4				
x_3x_4'	1	1		1
$x_3'x_4'$	1			1
$x_3'x_4$				

7.

	x_1x_2	x_1x_2'	$x_1'x_2'$	$x_1'x_2$
x_3x_4		1		
x_3x_4'		1	1	1
$x_3'x_4'$				
$x_3'x_4$		1		

8.

	x_1x_2	x_1x_2'	$x_1'x_2'$	$x_1'x_2$
x_3x_4				1
x_3x_4'	1	1		
$x_3'x_4'$		1	1	
$x_3'x_4$			1	

For Exercises 9 and 10, use a Karnaugh map to find the minimal sum-of-products form for the truth functions shown.

9.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	0
0	0	0	0

10.

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	1
1	1	1	0	1
1	1	0	1	1
1	1	0	0	1
1	0	1	1	0
1	0	1	0	1
1	0	0	1	0
1	0	0	0	1
0	1	1	1	1
0	1	1	0	1
0	1	0	1	1
0	1	0	0	1
0	0	1	1	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	0

11. Use a Karnaugh map to find the minimal sum-of-products form for the truth function of Exercise 17, Section 8.2.

12. Use a Karnaugh map to find the minimal sum-of-products form for the truth function of Exercise 18, Section 8.2.

13. a. Use a Karnaugh map to find the minimal sum-of-products form for the truth function of Exercise 19, Section 8.2.
    b. Draw the logic network for the reduced expression of part (a).

14. a. Use a Karnaugh map to find the minimal sum-of-products form for the truth function of Exercise 20, Section 8.2.
    b. Draw the logic network for the reduced expression of part (a).

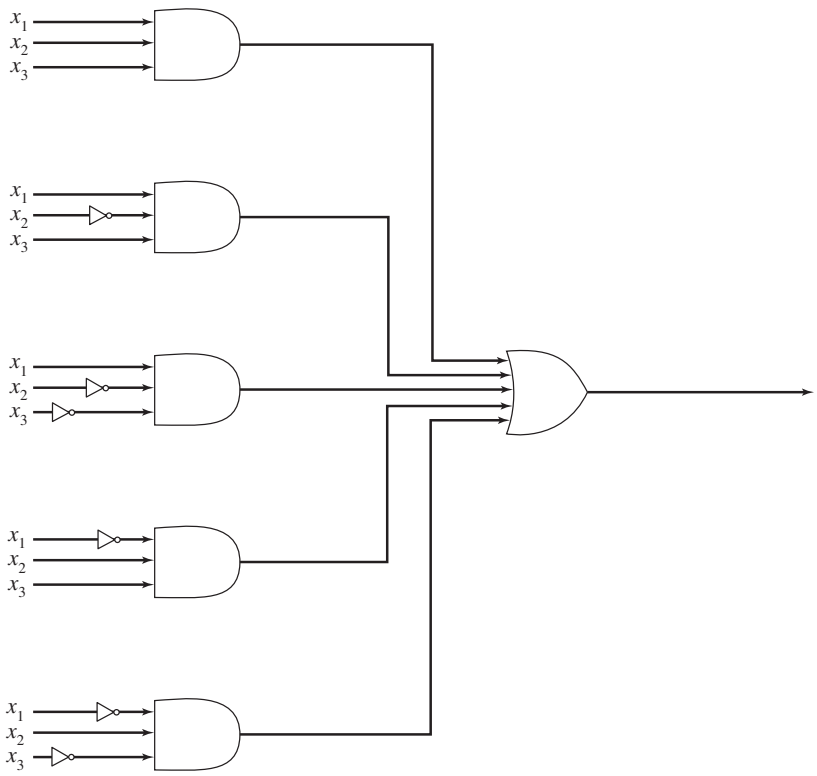
15. Use a Karnaugh map to find the minimal sum-of-products form for the following Boolean expression.

$$x_1'x_2'x_3x_4 + x_1x_2x_3'x_4 + x_1'x_2'x_3'x_4 + x_1x_2'x_3x_4' + x_1'x_2x_3x_4 + x_1'x_2x_3'x_4 + x_1'x_2'x_3x_4'$$

16. Use a Karnaugh map to find the minimal sum-of-products form for the following Boolean expression.

$$x_1'x_2'x_3'x_4' + x_1x_2x_3'x_4 + x_1'x_2'x_3'x_4 + x_1x_2x_3'x_4' + x_1'x_2x_3x_4 + x_1x_2'x_3'x_4'$$

17. Use a Karnaugh map to find a minimal sum-of-products expression for the network of three variables shown in the figure. Sketch the new network.



18. At Rats R Us, you found a standard sum-of-products form for the logic to control valves A and B (Exercise 39, Section 8.2). Now earn yourself a raise by using Karnaugh maps to minimize these expressions.

19. Use a Karnaugh map to find a minimal sum-of-products form for the truth function in the table. Don't-care conditions are shown by dashes.

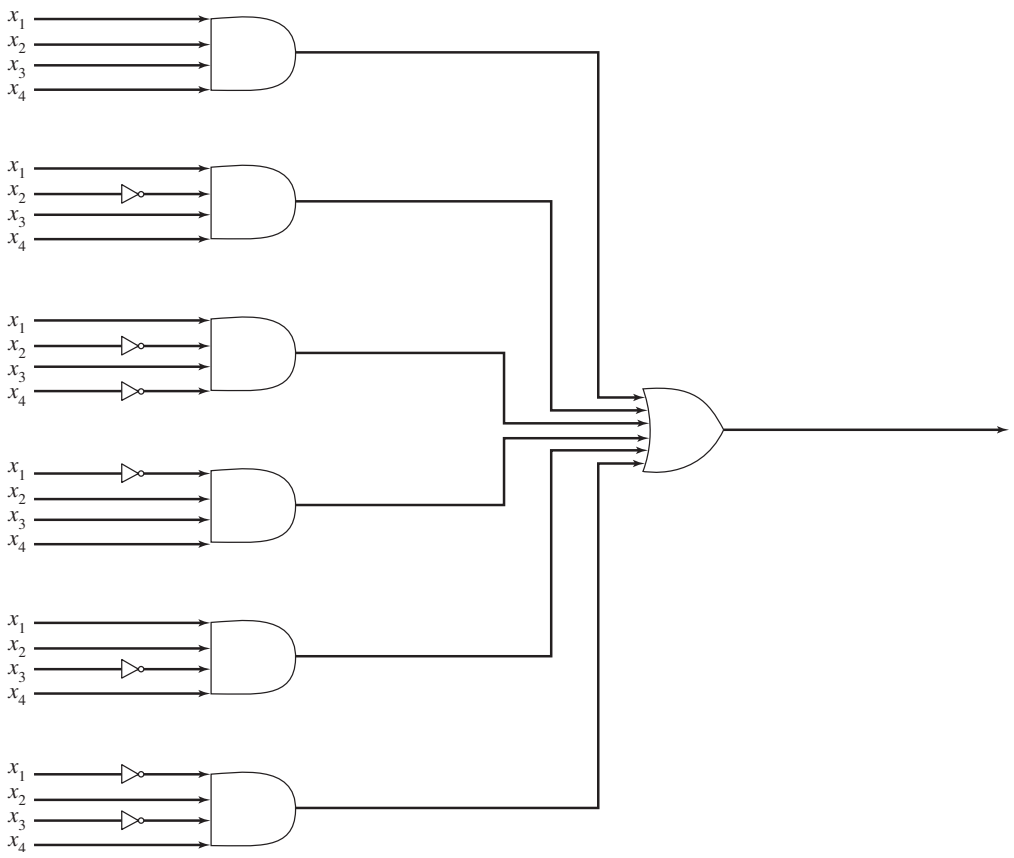
x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	0
1	1	1	0	1
1	1	0	1	0
1	1	0	0	–
1	0	1	1	0
1	0	1	0	–
1	0	0	1	0
1	0	0	0	0
0	1	1	1	0
0	1	1	0	1
0	1	0	1	0
0	1	0	0	1
0	0	1	1	1
0	0	1	0	0
0	0	0	1	–
0	0	0	0	0

20. Use a Karnaugh map to find a minimal sum-of-products form for the truth function in the table. Don't-care conditions are shown by dashes.

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	0
1	1	1	0	1
1	1	0	1	0
1	1	0	0	–
1	0	1	1	–
1	0	1	0	0
1	0	0	1	0
1	0	0	0	0
0	1	1	1	1
0	1	1	0	0
0	1	0	1	1
0	1	0	0	0
0	0	1	1	1
0	0	1	0	0
0	0	0	1	–
0	0	0	0	0

21. Use the Quine–McCluskey procedure to find a minimal sum-of-products form for the truth function illustrated by the map for Exercise 3.

22. Use the Quine–McCluskey procedure to find a minimal sum-of-products form for the network in the figure. Sketch the new network.



For Exercises 23 and 24, use the Quine–McCluskey procedure to find the minimal sum-of-products form for the truth functions in the given tables.

23.

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	0
1	1	1	0	1
1	1	0	1	0
1	1	0	0	0
1	0	1	1	0
1	0	1	0	1
1	0	0	1	1
1	0	0	0	1
0	1	1	1	0
0	1	1	0	0
0	1	0	1	0
0	1	0	0	1
0	0	1	1	1
0	0	1	0	1
0	0	0	1	0
0	0	0	0	1

24.

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	1
1	1	1	0	0
1	1	0	1	1
1	1	0	0	0
1	0	1	1	1
1	0	1	0	0
1	0	0	1	1
1	0	0	0	1
0	1	1	1	1
0	1	1	0	0
0	1	0	1	1
0	1	0	0	1
0	0	1	1	1
0	0	1	0	0
0	0	0	1	1
0	0	0	0	1

In Exercises 25–28, use the Quine–McCluskey procedure to find the minimal sum-of-products form for the Boolean expressions.

25. $x_1x_2'x_3x_4' + x_1'x_2'x_3x_4 + x_1'x_2x_3x_4 + x_1'x_2'x_3'x_4' + x_1'x_2x_3'x_4' + x_1'x_2'x_3'x_4$
26. $x_1x_2x_3x_4 + x_1x_2'x_3x_4 + x_1x_2x_3x_4' + x_1x_2'x_3x_4' + x_1'x_2x_3x_4' + x_1x_2x_3'x_4' + x_1'x_2x_3'x_4' + x_1x_2x_3'x_4 + x_1'x_2x_3'x_4$
27. $x_1x_2x_3x_4 + x_1x_2x_3x_4' + x_1'x_2x_3x_4' + x_1x_2x_3'x_4' + x_1'x_2'x_3'x_4' + x_1'x_2x_3'x_4' + x_1x_2'x_3'x_4 + x_1'x_2'x_3'x_4 + x_1x_2x_3'x_4$
28. $x_1'x_2x_3'x_4x_5' + x_1'x_2x_3x_4'x_5 + x_1x_2x_3x_4x_5 + x_1'x_2'x_3x_4'x_5 + x_1x_2'x_3x_4x_5 + x_1'x_2'x_3'x_4'x_5 + x_1x_2'x_3x_4'x_5 + x_1x_2x_3x_4'x_5' + x_1x_2x_3'x_4x_5 + x_1'x_2'x_3'x_4x_5'$
29. Use the Quine–McCluskey procedure to find a minimal sum-of-products form for the truth function illustrated by the map in Figure 8.34.

CHAPTER 8 REVIEW

TERMINOLOGY

AND gate (p. 639)
Boolean algebra (p. 620)
Boolean expression (p. 639)
canonical sum-of-products form (p. 643)
combinational network (p. 642)
complement (of a Boolean algebra element) (p. 625)
De Morgan's laws (for a Boolean algebra) (p. 624)
disjunctive normal form (p. 643)
double negation (for a Boolean algebra) (p. 624)
dual (of a Boolean algebra property) (p. 623)
equivalent Boolean expressions (p. 645)
FPGA (p. 647)
full-adder (p. 650)
half-adder (p. 649)
idempotent property (of a Boolean algebra) (p. 622)
inverter (p. 639)
isomorphic instances of a structure (p. 626)
isomorphism (p. 626)
isomorphism for Boolean algebras (p. 629)
NAND gate (p. 650)
NOR gate (p. 651)
OR gate (p. 639)
PLD (p. 647)
truth function (p. 640)
universal bound property (of a Boolean algebra) (p. 624)

SELF-TEST

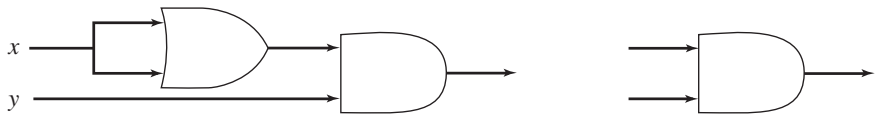
Answer the following true-false questions.

Section 8.1

1. In any Boolean algebra, $x + x' = 0$.
2. Set theory is an instance of a Boolean algebra in which $+$ is set union and $\cdot$ is set intersection.
3. In any Boolean algebra, $x + (y + x \cdot z) = x + y$.
4. The dual of the equation in the previous statement is $x \cdot [y \cdot (x + z)] = x \cdot y$.
5. Any two Boolean algebras with 16 elements are isomorphic.

Section 8.2

1. A logic network for the Boolean expression $(x + y)'$ could be built using one AND gate and two inverters.
2. The canonical sum-of-products form for a truth function f: $\{0, 1\}^n \to \{0, 1\}$ has n terms.
3. Two single-bit binary numbers can be added using a network consisting of two half-adders.
4. The following two logic networks represent the same truth function:



5. The most efficient way to construct a logic network for a given truth function using only NAND gates is to construct the logic network using AND, OR, and NOT gates and then replace each of these elements with its equivalent form in NAND gates.

Section 8.3

1. A Karnaugh map is a device to help change a canonical sum-of-products form to a reduced sum-of-products form.
2. The 1*s* in a Karnaugh map correspond to the 1 values of the truth function.
3. When using a Karnaugh map to reduce a Boolean expression, the largest possible blocks should be determined first because they provide the greatest reduction.
4. In the Quine–McCluskey procedure, terms that combine must have dashes in the same locations.
5. In the Quine–McCluskey procedure, a check in some row of the comparison table indicates that the term for that row is an essential term that must appear in the reduced expression.

ON THE COMPUTER

For Exercises 1–4, write a computer program that produces the desired output from the given input.

1. *Input*: *n* and tables defining two binary operations and one unary operation on a set of *n* objects
   Output: Indication of whether the structure is a Boolean algebra
   Algorithm: Testing the 10 properties for all cases

2. *Input*: *n*, tables defining a binary operation on each of two sets of *n* elements, and a table defining a bijection from one set to the other
   Output: Indication of whether the function is an isomorphism
   Algorithm: Testing all possible cases

3. *Input*: *n* and a table representing a truth function with *n* arguments
   Output: Canonical sum-of-products Boolean expression for the truth function

4. *Input*: *n* and a table representing a truth function with *n* arguments
   Output: Minimal sum-of-products Boolean expression for the truth function
   Algorithm: Using the Quine–McCluskey procedure

Chapter

Modeling Arithmetic, Computation, and Languages

CHAPTER OBJECTIVES

After studying this chapter, you will be able to:

- See how algebraic structures, finite-state machines, and Turing machines are all models of various kinds of computation, and how formal languages attempt to model natural languages.
- Recognize certain well-known group structures.
- Prove some properties about groups.
- Understand what it means for groups to be isomorphic.
- Be able to construct group codes for single-error correction of binary m-tuples.
- Be able to decode received n-tuples for a single-error correcting perfect code.
- Trace the operation of a given finite-state machine on an input string.
- Construct finite-state machines to recognize certain sets.
- For a given finite-state machine, find an equivalent machine with fewer states if one exists.
- Build a circuit for a finite-state machine.
- Trace the operation of a given Turing machine on an input tape.
- Construct Turing machines to perform certain recognition or computation tasks.
- Understand the Church–Turing thesis and what it implies for the Turing machine as a model of computation.
- Be aware of the $P = NP$ question regarding computational complexity.
- Given a grammar G, construct the derivation of strings in $L(G)$.
- Understand the relationship between different classes of formal languages and different computational devices.

Your team at Babel, Inc., is writing a compiler for a new programming language, currently code-named ScrubOak after a tree outside your office window. During the first phase of compilation (called the lexical analysis phase) the compiler must break down statements into individual units called tokens. In particular, the compiler must be able to recognize identifiers in the language, which are strings of letters, and also recognize the two keywords in the language, which are **if** and **in**.

Question: **How can the compiler recognize the individual tokens in a statement?**

A mathematical structure, as discussed in Chapter 8, is a formal model intended to capture common properties or behavior found in different contexts. A structure consists of an abstract set of objects, together with operations on or relationships among those objects that obey certain rules. The Boolean algebra structure of Chapter 8 is a model of the properties and behavior common to both propositional logic and set theory. As a formal model, it is an abstract entity, an idea; propositional logic and set theory are two instances, or realizations, of this idea.

In this chapter we study other structures. In Section 9.1, algebraic structures are defined that model various types of arithmetic such as addition of integers and multiplication of positive real numbers. As an aside, section 9.2 looks at coding theory, an important application of one of these algebraic structures. This type of encoding (and decoding) is not about secrecy but about detecting and perhaps correcting bit errors in data transmission or storage.

The "arithmetics" of Section 9.1 represent a limited form of computation, but we will see models of much broader forms of computation in Sections 9.3 and 9.4. Our initial choice for such a model, the finite-state machine, is a useful device for certain tasks, such as the lexical analysis task facing your team at Babel. But the finite-state machine is ultimately too limited to model computation in the general sense. For a model that captures the notion of computation in all its generality, we turn to the Turing machine. Using the Turing machine as a model of computation will reveal that some well-defined tasks are not computable at all.

Finally, Section 9.5 discusses formal grammars and languages, which were developed as attempts to model natural languages such as English. While less than completely successful in this regard, formal grammars and languages do serve to model many constructs in programming languages and play an important role in compiler theory.

SECTION 9.1 ALGEBRAIC STRUCTURES

Definitions and Examples

We begin by analyzing a simple form of arithmetic, namely the addition of integers. There is a set $\mathbb{Z}$ of objects (the integers) and a binary operation on those objects (addition). Recall from Section 4.1 that a binary operation on a set must be *well defined* (giving a unique answer whenever it is applied to any two members of the set) and that the set must be *closed* under the operation (the answer must be a member of the set). The notation $[\mathbb{Z}, +]$ will denote the set together with the binary operation on that set.

In $[\mathbb{Z}, +]$, an equation such as

$$2 + (3 + 5) = (2 + 3) + 5$$

is true. On each side of the equation the integers remain in the same order, but the grouping of those integers, which indicates the order in which the additions are performed, changes. Changing the grouping has no effect on the answer. Another type of equation that holds in $[\mathbb{Z}, +]$ is

$$2 + 3 = 3 + 2$$

Changing the order of the integers being added has no effect on the answer.

Equations such as

$$2 + 0 = 2$$
$$0 + 3 = 3$$
$$-125 + 0 = -125$$

are also true. Adding zero to any integer does not change the value of that integer.

Finally, equations such as

$$2 + (-2) = 0$$
$$5 + (-5) = 0$$
$$-20 + 20 = 0$$

are true; adding the negative of an integer to the integer gives 0 as a result.

These equations represent four properties that occur so often they each have a name.

DEFINITIONS **PROPERTIES OF BINARY OPERATIONS**

Let S be a set and let $\cdot$ denote a binary operation on S. (Here $\cdot$ does *not* necessarily denote multiplication but simply any binary operation.)

1. The operation $\cdot$ is **associative** if
$$(\forall x)(\forall y)(\forall z)[x \cdot (y \cdot z) = (x \cdot y) \cdot z]$$
Associativity allows us to write $x \cdot y \cdot z$ without using parentheses because grouping does not matter.
2. The operation $\cdot$ is **commutative** if
$$(\forall x)(\forall y)(x \cdot y = y \cdot x)$$
3. $[S, \cdot]$ has an **identity element** if
$$(\exists i)(\forall x)(x \cdot i = i \cdot x = x)$$
4. If $[S, \cdot]$ has an identity element i, then each element in S has an **inverse** with respect to $\cdot$ if
$$(\forall x)(\exists x^{-1})(x \cdot x^{-1} = x^{-1} \cdot x = i)$$

In the statements of the properties, the universal quantifiers range over the set S; if the associative property holds, the equation $x \cdot (y \cdot z) = (x \cdot y) \cdot z$ is true for any $x, y, z \in S$, and similarly for the commutative property. The existential quantifier also applies to the set S, so an identity element i, if it exists, must be an element of S, and an inverse element x^{-1}, if it exists, must be an element of S. Note the order of the quantifiers: In the definition of an identity, the existential quantifier comes first—there must be one identity element i that satisfies the equation $x \cdot i = i \cdot x = x$ for every x in S, just like the integer 0 in $[\mathbb{Z}, +]$. In the definition of the inverse element, the existential quantifier comes second—for each x, there is an x^{-1}, and if x is changed, then x^{-1} can change, just like the inverse of 2 in $[\mathbb{Z}, +]$ is -2 and the inverse of 5 is -5. If there is no identity element, then it does not make sense to talk about inverse elements.

DEFINITIONS **GROUP, COMMUTATIVE GROUP**

$[S, \cdot]$ is a **group** if S is a nonempty set and $\cdot$ is a binary operation on S such that

1. $\cdot$ is associative.
2. an identity element exists (in S).
3. each element in S has an inverse (in S) with respect to $\cdot$.

A group in which the operation $\cdot$ is commutative is called a **commutative group**.

Once again, the dot in the definitions is a generic symbol representing a binary operation. In any specific case, the particular binary operation has to be defined. If the operation is addition, for example, then the $+$ sign replaces the generic symbol, as in $[\mathbb{Z}, +]$. As an analogy with programming, we can think of the generic symbol as a formal parameter to be replaced by an actual argument—the specific operation—when its value becomes known. If it is clear what the binary operation is, we may refer to "the group S" rather than "the group $[S, \cdot]$."

From the discussion, it should be clear that $[\mathbb{Z}, +]$ is a commutative group, with an identity element of 0. The idea of a group would not be useful if there were not a number of other instances. (Again as an analogy with programming, one can think of a group as an abstract data type—a pattern—with many possible instances of that data type.)

EXAMPLE 1 Let $\mathbb{R}^+$ denote the positive real numbers, and let $\cdot$ denote real-number multiplication, which is a binary operation on $\mathbb{R}^+$. Then $[\mathbb{R}^+, \cdot]$ is a commutative group. Multiplication is associative and commutative. The positive real number 1 serves as an identity because

$$x \cdot 1 = 1 \cdot x = x$$

for every positive real number x. Every positive real number x has an inverse with respect to multiplication, namely the positive real number $1/x$, because

$$x \cdot 1/x = 1/x \cdot x = 1$$

PRACTICE 1 The set in Example 1 is limited to the positive real numbers. Is $[\mathbb{R}, \cdot]$ a commutative group? Why or why not?

EXAMPLE 2 Let $M_2(\mathbb{Z})$ denote the set of 2×2 matrices with integer entries, and let $+$ denote matrix addition. Then $+$ is a binary operation on $M_2(\mathbb{Z})$ (note that closure holds). This is a commutative group because the integers are a commutative group, so each corner of the matrix behaves properly. For example, matrix addition is commutative because

$$\begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix} + \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{bmatrix} = \begin{bmatrix} a_{1,1} + b_{1,1} & a_{1,2} + b_{1,2} \\ a_{2,1} + b_{2,1} & a_{2,2} + b_{2,2} \end{bmatrix}$$
$$= \begin{bmatrix} b_{1,1} + a_{1,1} & b_{1,2} + a_{1,2} \\ b_{2,1} + a_{2,1} & b_{2,2} + a_{2,2} \end{bmatrix}$$
$$= \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{bmatrix} + \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix}$$

The matrix

$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

is an identity. The matrix

$$\begin{bmatrix} 1 & -4 \\ 2 & 5 \end{bmatrix}$$

is an inverse of the matrix

$$\begin{bmatrix} -1 & 4 \\ -2 & -5 \end{bmatrix}$$

REMINDER

It's important to understand all this new terminology. You can't prove that $P \rightarrow Q$ if you don't know what you're starting with or where you want to go.

A structure called a **monoid** results from dropping the inverse property in the definition of a group; thus a monoid has an associative operation and an identity element, but in a monoid that is not also a group, at least one element has no inverse. A **semigroup** results from dropping the identity property and the inverse property in the definition of a group; thus a semigroup has an associative operation, but in a semigroup that is not also a monoid, no identity element exists. Many familiar forms of arithmetic are instances of semigroups, monoids, and groups.

EXAMPLE 3 Consider $[M_2(\mathbb{Z}), \cdot]$ where $\cdot$ denotes matrix multiplication. Closure holds. It can be shown (Exercise 2) that matrix multiplication is associative. The matrix

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

serves as an identity because

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Thus $[M_2(\mathbb{Z}), \cdot]$ is at least a monoid.

PRACTICE 2 Prove that $[M_2(\mathbb{Z}), \cdot]$ is not a commutative monoid. ■

PRACTICE 3 Prove that $[M_2(\mathbb{Z}), \cdot]$ is not a group. ■

Although the requirements for a structure to be a semigroup are relatively modest, not every arithmetic structure qualifies.

PRACTICE 4 Prove that $[\mathbb{Z}, -]$ is not a semigroup, where $-$ denotes integer subtraction. ■

PRACTICE 5 Let S be the set of noninteger rational numbers, and let $\cdot$ denote multiplication. Is $[S, \cdot]$ a semigroup? ■

EXAMPLE 4 Each of the following is an instance of a commutative semigroup. You should be able to verify closure, associativity, and commutativity for each:

$$[\mathbb{N}, +], [\mathbb{N}, \cdot], [\mathbb{Q}, \cdot], [\mathbb{R}^+, +], [\mathbb{R}, +]$$

EXAMPLE 5 For any Boolean algebra $[B, +, \cdot, ', 0, 1]$, $[B, +]$ and $[B, \cdot]$ are commutative semigroups. Therefore for any set S, $[\wp(S), \cup]$ and $[\wp(S), \cap]$ are commutative semigroups.

Because the requirements that must be satisfied in going from semigroup to monoid to group keep getting stiffer, we expect some examples to drop out, but those remaining should have richer and more interesting personalities.

PRACTICE 6 Which of the following semigroups are monoids? Name the identities.

$$[\mathbb{N}, +], [\mathbb{N}, \cdot], [\mathbb{Q}, \cdot], [\mathbb{R}^+, +], [\mathbb{R}, +], [\wp(S), \cup], [\wp(S), \cap]$$

■

PRACTICE 7 Which of the monoids from the list in Practice 6 are groups? ■

Next we look at a selection of other examples of semigroups, monoids, and groups where the elements are not just simple numbers or where the operations are less familiar.

EXAMPLE 6 An expression of the form

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_0$$

where $a_i \in \mathbb{R}$, $i = 0, 1, \ldots, n$, and $n \in \mathbb{N}$ is a **polynomial in x with real-number coefficients** (or a **polynomial in x over** $\mathbb{R}$.) For each i, a_i is the **coefficient** of x^i.

If i is the largest integer greater than 0 for which $a_i \neq 0$, the polynomial is of **degree** i; if no such i exists, the polynomial is of **zero degree**. Terms with zero coefficients are generally not written. Thus, $\pi x^4 - 2/3x^2 + 5$ is a polynomial of degree 4, and the constant polynomial 6 is of zero degree. The set of all polynomials in x over $\mathbb{R}$ is denoted by $\mathbb{R}[x]$.

We define binary operations of $+$ and $\cdot$ in $\mathbb{R}[x]$ to be the familiar operations of polynomial addition and multiplication. For polynomials $f(x)$ and $g(x)$ members of $\mathbb{R}[x]$, the products $f(x) \cdot g(x)$ and $g(x) \cdot f(x)$ are equal because the coefficients are real numbers, and we can use all the properties of real numbers under multiplication and addition (properties such as commutativity and associativity). Similarly, for $f(x)$, $g(x)$, and $h(x)$ members of $\mathbb{R}[x]$, $(f(x) \cdot g(x)) \cdot h(x) = f(x) \cdot (g(x) \cdot h(x))$. The constant polynomial 1 is an identity because $1 \cdot f(x) = f(x) \cdot 1 = f(x)$ for every $f(x) \in \mathbb{R}[x]$. Thus, $[\mathbb{R}[x], \cdot]$ is a commutative monoid. It fails to be a group because only the nonzero constant polynomials have inverses. For example, there is no polynomial $g(x)$ such that $g(x) \cdot x = x \cdot g(x) = 1$, so the polynomial x has no inverse. (Note that while $x \cdot (1/x) = 1$, $1/x = x^{-1}$ is not a polynomial.) However, $[\mathbb{R}[x], +]$ is a commutative group. ●

PRACTICE 8

a. For $f(x), g(x), h(x) \in \mathbb{R}[x]$, write the equations saying that $\mathbb{R}[x]$ under $+$ is commutative and associative.
b. What is an identity element in $[\mathbb{R}[x], +]$?
c. What is an inverse of $7x^4 - 2x^3 + 4$ in $[\mathbb{R}[x], +]$? ■

Polynomials play a special part in the history of group theory (the study of groups) because much research in group theory was prompted by the very practical problem of solving polynomial equations of the form $f(x) = 0, f(x) \in \mathbb{R}[x]$. The quadratic formula provides an algorithm for finding solutions for every $f(x)$ of degree 2, and the algorithm uses only the algebraic operations of addition, subtraction, multiplication, division, and taking roots. Other such algorithms exist for polynomials of degrees 3 and 4. One of the highlights of abstract algebra is the proof that no algorithm using only these operations exists for every $f(x)$ of degree 5. (Notice that this statement is much stronger than simply saying that no algorithm has yet been found; it says to stop looking for one.)

The next example uses modular arithmetic. You may recall from Section 5.1 (Example 15 and the subsequent discussion) that each computer has some limit on the size of the integers that it can store. Although we would like a computer to be able to exhibit the behavior of $[\mathbb{Z}, +]$, the best we can obtain is some finite approximation. The approximation is achieved by performing addition modulo n. The "answer" to the computation $x + y$ for $x, y \in \mathbb{Z}$ is then either the actual value $x + y$ if this value falls within the limit that can be stored or a remainder value obtained by doing modular arithmetic, which is equivalent to $x + y$ under the equivalence relation of congruence modulo n.

EXAMPLE 7 Let $\mathbb{Z}_5 = \{0, 1, 2, 3, 4\}$ and define *addition modulo* 5, denoted by $+_5$, on $\mathbb{Z}_5$ by $x +_5 y = r$, where r is the remainder when $x + y$ is divided by 5. In other words, $x +_5 y = (x + y) \bmod 5$. For example, $1 +_5 2 = 3$ and $3 +_5 4 = 2$. *Multiplication modulo* 5 is defined by $x \cdot_5 y = (x \cdot y) \bmod 5$. Thus, $2 \cdot_5 3 = 1$ and $3 \cdot_5 4 = 2$. Then $[\mathbb{Z}_5, +_5]$ is a commutative group, and $[\mathbb{Z}_5, \cdot_5]$ is a commutative monoid. ●

PRACTICE 9

a. Complete the following tables defining $+_5$ and $\cdot_5$ on $\mathbb{Z}_5$.

$+_5$	0	1	2	3	4
0					
1			3		
2					
3					
4					

$\cdot_5$	0	1	2	3	4
0					
1					
2				1	
3					2
4					

b. What is an identity in $[\mathbb{Z}_5, +_5]$? In $[\mathbb{Z}_5, \cdot_5]$?
c. What is an inverse of 2 in $[\mathbb{Z}_5, +_5]$?
d. Which elements in $[\mathbb{Z}_5, +_5]$ have inverses? ■

As we did on $\mathbb{Z}_5$, we can define operations of **addition modulo *n*** and **multiplication modulo *n*** on the set $\mathbb{Z}_n = \{0, 1, \ldots, n - 1\}$ where n is any positive integer. Again $[\mathbb{Z}_n, +_n]$ is a commutative group and $[\mathbb{Z}_n, \cdot_n]$ is a commutative monoid. (See Exercise 47 for the relationship between the group $[\mathbb{Z}_n, +_n]$ and the set of equivalence classes under the binary relation of congruence modulo n.)

PRACTICE 10

a. Give the table for $\cdot_6$ on $\mathbb{Z}_6$.
b. Which elements in $[\mathbb{Z}_6, \cdot_6]$ have inverses? ■

Notice that when we use a table to define an operation on a finite set, it is easy to check for commutativity by looking for symmetry around the main diagonal. It is also easy to find an identity element because its row looks like the top of the table and its column looks like the side. And it is easy to locate an inverse of an element. Look along the row until you find a column where the identity appears; then check to see that changing the order of the elements still gives the identity. However, associativity (or the lack of it) is not immediately apparent from the table.

EXAMPLE 8

Let Z_2^n be the set of all binary n-tuples with $n \geq 1$. We'll usually use a little shorthand and write, for example, 1101 instead of (1, 1, 0, 1). Using componentwise addition modulo 2, $+_2$. it's easy to see that the addition operation is associative and commutative and that the n-tuple of all 0s is an identity. Each n-tuple has an inverse. Therefore $[Z_2^n, +_2]$ is a commutative group. ●

REMINDER

In the group $[\mathbb{Z}_2, +_2]$, the set is $\{0, 1\}$ and the operation is addition modulo 2. In the group $[Z_2^n, +_2]$ for any n, the set is all binary n-tuples and the operation is componentwise addition modulo 2.

PRACTICE 11 In the group $[Z_2^5, +_2]$, what is

a. $01101 +_2 11011$?
b. -10100? ■

The next two examples give us algebraic structures where the elements are *functions*, mappings from a domain to a codomain.

EXAMPLE 9

Let A be a set and consider the set S of all functions f such that $f: A \to A$. The binary operation is function composition, denoted by $\circ$. Note that S is closed under $\circ$ and that function composition is associative (see Practice 12). Thus $[S, \circ]$ is a semigroup, called the **semigroup of transformations on *A***. Actually $[S, \circ]$ is a monoid because the identity function i_A that takes each member of A to itself has the property that for any $f \in S$,

$$f \circ i_A = i_A \circ f = f$$ ●

PRACTICE 12 Prove that function composition on the set S just defined is associative. ■

EXAMPLE 10

Again let A be a set and consider the set S_A of all bijections f such that $f: A \to A$ (permutations of A). Bijectiveness is preserved under function composition, function composition is associative, the identity function i_A is a permutation, and for any $f \in S_A$, the inverse function f^{-1} exists and is a permutation. Furthermore,

$$f \circ f^{-1} = f^{-1} \circ f = i_A$$

Thus, $[S_A, \circ]$ is a group, called the **group of permutations** on A. ●

If $A = \{1, 2, \ldots, n\}$ for some positive integer n, then S_A is called the **symmetric group of degree *n*** and denoted by S_n. Thus, S_3, for example, is the set of all permutations on $\{1, 2, 3\}$. There are $3! = 6$ such permutations, which we name as follows (using the cycle notation of Section 5.4):

$$\alpha_1 = \mathrm{i} \qquad \alpha_2 = (1, 2) \qquad \alpha_3 = (1, 3)$$
$$\alpha_4 = (2, 3) \qquad \alpha_5 = (1, 2, 3) \qquad \alpha_6 = (1, 3, 2)$$

Recall that the notation (1, 2), for example, means that 1 maps to 2, 2 maps to 1, and unnamed elements map to themselves. The composition (1, 2) ∘ (1, 3) is done from right to left, so

$$\begin{array}{ccc} & \text{By } (1,3) & \text{By } (1,2) \\ 1 & \to 3 & \to 3 \\ 2 & \to 2 & \to 1 \\ 3 & \to 1 & \to 2 \end{array}$$

resulting in (1, 3, 2). Thus $\alpha_2 \circ \alpha_3 = (1, 2) \circ (1, 3) = (1, 3, 2) = \alpha_6$.

PRACTICE 13

a. Complete the group table for $[S_3, \circ]$.

$\circ$	α_1	α_2	α_3	α_4	α_5	α_6
α_1						
α_2			α_6			
α_3						
α_4						
α_5						
α_6						

b. Is $[S_3, \circ]$ a commutative group? ■

$[S_3, \circ]$ is our first example of a noncommutative group (although $[M_2(\mathbb{Z}), \cdot]$ was a noncommutative monoid).

The next example is very simple but particularly appropriate because it appears in several areas of computer science, including formal language theory and automata theory.

EXAMPLE 11

Let A be a finite set; its elements are called **symbols** and A itself is called an **alphabet**. A^* denotes the set of all finite-length **strings**, or **words**, over A. A^* can be defined recursively (as in Example 6 in Chapter 3), where · denotes **concatenation** of strings:

a. The **empty string** λ (the string with no symbols) belongs to A^*.
b. Any single member of A belongs to A^*.
c. If x and y are strings in A^*, so is $x \cdot y$.

Thus, if $A = \{a, b\}$, then *abbaa*, *bbbbba*, and *a* are all strings over A, and $abbaa \cdot a$ gives the string abbaaa. From the recursive definition, any string over A contains only a finite number of symbols. The number of symbols in a string is called its **length**. The empty string λ is the only zero-length string.

The empty string λ should not be confused with the empty set $\varnothing$; even if A itself is $\varnothing$, then $A^* = \{\lambda\}$. If A is nonempty, then whatever the size of A, A^* is a denumerable (countably infinite) set. If A contains only one element, say $A = \{a\}$, then $\lambda, a, aa, aaa, \ldots$, is an enumeration of A^*. If A contains more than one element, then a lexicographical (alphabetical) ordering can be imposed on the elements of A. An enumeration of A^* is then obtained by counting the empty string first, then lexicographically ordering all strings of length 1 (there is a finite number of these), then lexicographically ordering all strings of length 2 (there is a finite number of these), and so forth. Note also that if A is nonempty, strings of arbitrary length can be found in A^*.

Concatenation is a binary operation on A^*, and it is associative. The empty string λ is an identity because for any string $x \in A^*$,

$$x \cdot \lambda = \lambda \cdot x = x$$

Therefore, $[A^*, \cdot]$ is a monoid, called the **free monoid generated by A**.

PRACTICE 14 For $A = \{a, b\}$

a. Is $[A^*, \cdot]$ a commutative monoid?
b. Is $[A^*, \cdot]$ a group?

Basic Results about Groups

We will now prove some basic theorems about groups. There are hundreds of theorems about groups and many books devoted exclusively to group theory, so we are barely scratching the surface here. The results we will prove follow almost immediately from the definitions involved.

REMINDER
To prove that something is unique …

By definition, a group $[G, \cdot]$ (or a monoid) has an identity element, and we have tried to be careful to refer to *an* identity element rather than *the* identity element. However, it is legal to say *the* identity because there is only one. To prove that the identity element is unique, suppose that i_1 and i_2 are both identity elements. Then

$$i_1 = i_1 \cdot i_2 = i_2$$

PRACTICE 15 Justify the foregoing equality signs.

Because $i_1 = i_2$, the identity element is unique. Thus, we have proved the following theorem.

THEOREM **ON THE UNIQUENESS OF THE IDENTITY IN A GROUP**
In any group (or monoid) $[G, \cdot]$, the identity element i is unique.

Each element x in a group $[G, \cdot]$ has an inverse element x^{-1}. Therefore, G contains many different inverse elements, but for each x, the inverse is unique.

THEOREM **ON THE UNIQUENESS OF INVERSES IN A GROUP**
For each x in a group $[G, \cdot]$, x^{-1} is unique.

PRACTICE 16 Prove the preceding theorem. (*Hint*: Assume two inverses for x, namely y and z, and let i be the identity. Then $y = y \cdot i = y \cdot (x \cdot z) = \cdots$.) ■

If x and y belong to a group $[G, \cdot]$, then $x \cdot y$ belongs to G and must have an inverse element in G. Naturally, we expect that inverse to have some connection with x^{-1} and y^{-1}, which we know exist in G. We can show that $(x \cdot y)^{-1} = y^{-1} \cdot x^{-1}$; thus the inverse of a product is the product of the inverses in reverse order.

THEOREM **ON THE INVERSE OF A PRODUCT**
For x and y members of a group $[G, \cdot]$, $(x \cdot y)^{-1} = y^{-1} \cdot x^{-1}$.

REMINDER
If it walks like a duck ...

Proof: We will show that $y^{-1} \cdot x^{-1}$ has the two properties required of $(x \cdot y)^{-1}$. Then, because inverses are unique, $(y^{-1} \cdot x^{-1})$ must be $(x \cdot y)^{-1}$.

$$\begin{aligned}(x \cdot y) \cdot (y^{-1} \cdot x^{-1}) &= x \cdot (y \cdot y^{-1}) \cdot x^{-1} \\ &= x \cdot i \cdot x^{-1} \\ &= x \cdot x^{-1} \\ &= i\end{aligned}$$

Similarly, $(y^{-1} \cdot x^{-1}) \cdot (x \cdot y) = i$. Notice how associativity and the meaning of i and inverses all come into play in this proof. *End of Proof.*

PRACTICE 17 Write 10 as $7 +_{12} 3$ and use the theorem on the inverse of a product to find $(10)^{-1}$ in the group $[\mathbb{Z}_{12}, +_{12}]$. ■

We know that many familiar number systems such as $[\mathbb{Z}, +]$ and $[\mathbb{R}, +]$ are groups. We make use of group properties when we do arithmetic or algebra in these systems. In $[\mathbb{Z}, +]$, for example, if we see the equation $x + 5 = y + 5$, we conclude that $x = y$. We are making use of the right cancellation law, which, we will soon see, holds in any group.

DEFINITION **CANCELLATION LAWS**
A set S with a binary operation $\cdot$ satisfies the **right cancellation law** if for $x, y, z \in S$, $x \cdot z = y \cdot z$ implies $x = y$. It satisfies the **left cancellation law** if $z \cdot x = z \cdot y$ implies $x = y$.

Suppose that x, y, and z are members of a group $[G, \cdot]$ and that $x \cdot z = y \cdot z$. To conclude that $x = y$, we take advantage of z^{-1}. Thus,

$$x \cdot z = y \cdot z$$

implies

$$\begin{aligned}(x \cdot z) \cdot z^{-1} &= (y \cdot z) \cdot z^{-1} \\ x \cdot (z \cdot z^{-1}) &= y \cdot (z \cdot z^{-1}) \\ x \cdot i &= y \cdot i \\ x &= y\end{aligned}$$

Hence, G satisfies the right cancellation law.

PRACTICE 18 Show that any group $[G, \cdot]$ satisfies the left cancellation law. ■

We have proved the following theorem.

THEOREM **ON CANCELLATION IN A GROUP**
Any group $[G, \cdot]$ satisfies the left and right cancellation laws.

EXAMPLE 12 We know that $[\mathbb{Z}_6, \cdot_6]$ is not a group. Here the equation

$$4 \cdot_6 2 = 1 \cdot_6 2$$

holds, but of course $4 \neq 1$. ●

Again, working in $[\mathbb{Z}, +]$, we would solve the equation $6 + x = 13$ by adding -6 to both sides, producing a unique answer of $x = (-6) + 13 = 7$. The property of being able to solve linear equations for unique solutions holds in all groups. Consider the equation $a \cdot x = b$ in the group $[G, \cdot]$ where a and b belong to G and x is to be found. Then $x = a^{-1} \cdot b$ is an element of G satisfying the equation. Should x_1 and x_2 both be solutions to the equation $ax = b$, then $a \cdot x_1 = a \cdot x_2$ and, by left cancellation, $x_1 = x_2$. Similarly, the unique solution to $x \cdot a = b$ is $x = b \cdot a^{-1}$.

THEOREM **ON SOLVING LINEAR EQUATIONS IN A GROUP**
Let a and b be any members of a group $[G, \cdot]$. Then the linear equations $a \cdot x = b$ and $x \cdot a = b$ have unique solutions in G.

PRACTICE 19 Solve the equation $x +_8 3 = 1$ in $[\mathbb{Z}_8, +_8]$. ■

The theorem on solving linear equations tells us something about tables for finite groups. As we look along row a of the group operation table, does element b

appear twice? If so, then the table says that there are two distinct elements x_1 and x_2 of the group such that $a \cdot x_1 = b$ and $a \cdot x_2 = b$. But by the theorem on solving linear equations, this double occurrence cannot happen. Thus, a given element of a finite group appears at most once in a given row of the group table. However, to complete the row, each element must appear at least once. A similar result holds for columns. Therefore, in a group table, each element appears exactly once in each row and each column. This property alone, however, is not sufficient to insure that a table represents a group; the operation must also be associative (see Exercise 31).

PRACTICE 20 Assume that $\circ$ is an associative binary operation on $\{1, a, b, c, d\}$. Complete the following table to define a group with identity 1,

$\circ$	1	a	b	c	d
1	1				
a			c	d	1
b		c	d		
c		d		a	
d				b	c

■

If $[G, \cdot]$ is a group where G is finite with n elements, then n is said to be the **order of the group**, denoted by $|G|$. If G is an infinite set, the group is of infinite order.

PRACTICE 21

a. Name a commutative group of order 18.
b. Name a noncommutative group of order 6.

■

More properties of groups appear in the exercises at the end of this section.

Subgroups

We know what groups are and we know what subsets are, so it should not be hard to guess what a subgroup is. However, we will look at an example before we give the definition. We know that $[\mathbb{Z}, +]$ is a group. Now let A be any nonempty subset of $\mathbb{Z}$. For any x and y in A, x and y are also in $\mathbb{Z}$, so $x + y$ exists and is unique. The set A "inherits" a well-defined operation, $+$, from $[\mathbb{Z}, +]$. The associativity property is also inherited, because for any $x, y, z \in A$, it is also true that $x, y, z \in \mathbb{Z}$ and the equation

$$(x + y) + z = x + (y + z)$$

holds. Perhaps A under the inherited operation has all the structure of $[\mathbb{Z}, +]$ and is itself a group. Whether this is true depends on A.

Suppose that $A = E$, the set of even integers. E is closed under addition, E contains 0 (the identity element), and the inverse of every even integer (its negative) is an even integer. $[E, +]$ is thus a group. But suppose that $A = O$, the set of odd integers. $[O, +]$ fails to be a group for several reasons. For one thing, it is not closed—adding two odd integers produces an even integer. (Closure depends on the set as well as the operation, so it is not an inherited property). For another, a subgroup must have an identity with respect to addition; 0 is the only integer that will serve, and 0 is not an odd integer.

DEFINITION **SUBGROUP**

Let $[G, \cdot]$ be a group and $A \subseteq G$. Then $[A, \cdot]$ is a **subgroup** of $[G, \cdot]$ if $[A, \cdot]$ is itself a group.

For $[A, \cdot]$ to be a group, it must have an identity element, which we'll denote by i_A. Of course G also has an identity element, which we'll denote by i_G. It turns out that $i_A = i_G$, but this equation does not follow from the uniqueness of a group identity because the element i_A, as far as we know, may not be an identity for all of G, and we cannot yet say that i_G is an element of A. However, $i_A = i_A \cdot i_A$ because i_A is the identity for $[A, \cdot]$, and $i_A = i_A \cdot i_G$ because i_G is the identity for $[G, \cdot]$. Because of the left cancellation law holding in the group $[G, \cdot]$, it follows that $i_A = i_G$.

To test whether $[A, \cdot]$ is a subgroup of $[G, \cdot]$, we can assume the inherited properties of a well-defined operation and associativity, and we check for the three remaining properties required.

THEOREM **ON SUBGROUPS**

For $[G, \cdot]$ a group with identity i and $A \subseteq G$, $[A, \cdot]$ is a subgroup of $[G, \cdot]$ if it meets the following three tests:

1. A is closed under $\cdot$.
2. $i \in A$.
3. Every $x \in A$ has an inverse element in A.

PRACTICE 22 The definition of a group requires that the set be nonempty. In the theorem on subgroups, why isn't there a specific test that $A \neq \varnothing$?

EXAMPLE 13

a. $[\mathbb{Z}, +]$ is a subgroup of the group $[\mathbb{R}, +]$. $\mathbb{Z}$ is closed under addition, $0 \in \mathbb{Z}$, and the negative of every integer is an integer.

b. $[\{1,4\}, \cdot_5]$ is a subgroup of the group $[\{1, 2, 3, 4\}, \cdot_5]$. Closure holds:

$\cdot_5$	1	4
1	1	4
4	4	1

The identity $1 \in \{1, 4\}$, and $1^{-1} = 1, 4^{-1} = 4$).

PRACTICE 23

a. Show that $[\{0, 2, 4, 6\}, +_8]$ is a subgroup of the group $[\mathbb{Z}_8, +_8]$.
b. Show that $[\{1, 2, 4\}, \cdot_7]$ is a subgroup of the group $[\{1, 2, 3, 4, 5, 6\}, \cdot_7]$. ■

If $[G, \cdot]$ is a group with identity i, then it is true that $[\{i\}, \cdot]$ and $[G, \cdot]$ are subgroups of $[G, \cdot]$. These somewhat trivial subgroups of $[G, \cdot]$ are called **improper subgroups**. Any other subgroups of $[G, \cdot]$ are **proper subgroups**.

PRACTICE 24 Find all the proper subgroups of S_3, the symmetric group of degree 3. (You can find them by looking at the group table; see Practice 13.) ■

One point of confusing terminology: The set of *all* bijections on a set A into itself under function composition (like S_3) is called *the group of permutations on A*, and any subgroup of this set (such as those in Practice 24) is called a **permutation group**. The distinction is that *the* group of permutations on a set A includes all bijections on A into itself, but *a* permutation group may not. Permutation groups are of particular importance, not only because they were the first groups to be studied, but also because they are the only groups if we consider isomorphic structures to be the same. We will see this result shortly.

There is an interesting subgroup we can always find in the symmetric group S_n for $n > 1$. We know that every member of S_n can be written as a composition of cycles, but it is also true that each cycle can be written as the composition of cycles of length 2, called **transpositions**. In S_7, for example, $(5, 1, 7, 2, 3, 6) = (5, 6) \circ (5, 3) \circ (5, 2) \circ (5, 7) \circ (5, 1)$. We can verify this by computing $(5, 6) \circ (5, 3) \circ (5, 2) \circ (5, 7) \circ (5, 1)$. Working from right to left,

$$1 \to 5 \to 7 \to 7 \to 7 \to 7$$

so 1 maps to 7. Similarly,

$$7 \to 7 \to 5 \to 2 \to 2 \to 2$$

so 7 maps to 2, and so on, resulting in $(5, 1, 7, 2, 3, 6)$. It is also true that $(5, 1, 7, 2, 3, 6) = (1, 5) \circ (1, 6) \circ (1, 3) \circ (1, 2) \circ (2, 4) \circ (1, 7) \circ (4, 2)$.

For any $n > 1$, the identity permutation i in S_n can be written as $i = (a, b) \circ (a, b)$ for any two elements a and b in the set $\{1, 2, \ldots, n\}$. This equation also shows that the inverse of the transposition (a, b) in S_n is (a, b). Now we borrow (without proof) one more fact: Even though there are various ways to write a cycle as the composition of transpositions, for a given cycle the number of transpositions will either always be even or always be odd. Consequently, we classify any permutation in S_n, $n > 1$, as **even** or **odd** according to the number of transpositions in any representation of that permutation. For example, in S_7, $(5, 1, 7, 2, 3, 6)$ is odd. If we denote by A_n the set of all even permutations in S_n, then A_n determines a subgroup of $[S, \circ]$. The composition of even permutations produces an even permutation, and $i \in A_n$. If $\alpha \in A_n$, and α as a product of transpositions is $\alpha = \alpha_1 \circ \alpha_2 \circ \cdots \circ \alpha_k$, then $\alpha^{-1} = \alpha_k^{-1} \circ \alpha_{k-1}^{-1} \circ \cdots \circ \alpha_1^{-1}$. Each inverse of a transposition is a transposition, so α^{-1} is also even.

The order of the group $[S_n, \circ]$ (the number of elements) is $n!$ What is the order of the subgroup $[A, \circ]$? We might expect half the permutations in S_n to be even and half to be odd. Indeed, this is the case. If we let O_n denote the set of odd permutations in S_n (which is not closed under function composition), then the mapping $f\colon A_n \to O_n$ defined by $f(\alpha) = \alpha \circ (1, 2)$ is a bijection.

PRACTICE 25 Prove that $f\colon A_n \to O_n$, given by $f(\alpha) = \alpha \circ (1, 2)$ is one-to-one and onto. ■

Because there is a bijection from A_n onto O_n, each set has the same number of elements. But $A_n \cap O_n = \varnothing$ and $A_n \cup O_n = S_n$, so $|A_n| = |S_n|/2 = n!/2$.

THEOREM **ON ALTERNATING GROUPS**

For $n \in \mathbb{N}$, $n > 1$, the set A_n of even permutations determines a subgroup, called the **alternating group**, of $[S_n, \circ]$ of order $n!/2$.

We have now seen several examples of subgroups of finite groups. In Example 13b and Practice 23, there were three such examples, and the orders of the groups and subgroups were

Group of order 4, subgroup of order 2
Group of order 8, subgroup of order 4
Group of order 6, subgroup of order 3

The theorem on alternating groups says that a particular group of order $n!$ has a subgroup of order $n!/2$.

Based on these examples, one might conclude that subgroups are always half the size of the parent group. This conclusion is not always true, but there is a relationship between the size of a group and the size of a subgroup. This relationship is stated in Lagrange's theorem, proved by the great French mathematician Joseph-Louis Lagrange in 1771 (we will omit the proof here).

THEOREM **LAGRANGE'S THEOREM**

The order of a subgroup of a finite group divides the order of the group.

Lagrange's theorem helps us narrow down the possibilities for subgroups of a finite group. If $|G| = 12$, for example, we would not look for any subgroups of order 7 because 7 does not divide 12. Also, the fact that 6 divides 12 does not imply the existence of a subgroup of G of order 6. In fact, A_4 is a group of order $4!/2 = 12$, but it can be shown that A_4 has no subgroups of order 6. Therefore the converse to Lagrange's theorem does not always hold. In certain cases the converse can be shown to be true—for example, in finite commutative groups (note that A_4 is not commutative).

Finally, we consider subgroups of the group $[\mathbb{Z}, +]$. For n any fixed element of $\mathbb{N}$, the set $n\mathbb{Z}$ is defined as the set of all integral multiples of n; $n\mathbb{Z} = \{nz | z \in \mathbb{Z}\}$. Thus, for example, $3\mathbb{Z} = \{0, \pm 3, \pm 6, \pm 9, \ldots\}$.

PRACTICE 26 Show that for any $n \in \mathbb{N}$, $[n\mathbb{Z}, +]$ is a subgroup of $[\mathbb{Z}, +]$. ■

Not only is $[n\mathbb{Z}, +]$ a subgroup of $[\mathbb{Z}, +]$ for any fixed n, but sets of the form $n\mathbb{Z}$ are the only subgroups of $[\mathbb{Z}, +]$ To illustrate, let $[S, +]$ be any subgroup of $[\mathbb{Z}, +]$. If $S = \{0\}$, then $S = 0\mathbb{Z}$. If $S \neq \{0\}$, let m be a member of S, $m \neq 0$. Either m is positive or, if m is negative, $-m \in S$ and $-m$ is positive. The subgroup S, therefore, contains at least one positive integer. Let n be the smallest positive integer in S (which exists by the principle of well-ordering). We will now see that $S = n\mathbb{Z}$.

First, since 0, n, and $-n$ are members of S and S is closed under $+$, $n\mathbb{Z} \subseteq S$. To obtain inclusion in the other direction, let $s \in S$. Now we divide the integer s by the integer n to get an integer quotient q and an integer remainder r with $0 \leq r < n$. Thus, $s = nq + r$. Solving for r, $r = s + (-nq)$. But $nq \in S$; therefore $-nq \in S$, and $s \in S$, so by closure of S under $+$, $r \in S$. If r is positive, we have a contradiction of the definition of n as the smallest positive number in S. Therefore, $r = 0$ and $s = nq + r = nq$. We now have $S \subseteq n\mathbb{Z}$, and thus $S = n\mathbb{Z}$, which completes the proof of the following theorem.

THEOREM ON SUBGROUPS OF $[\mathbb{Z}, +]$

Subgroups of the form $[n\mathbb{Z}, +]$ for $n \in \mathbb{N}$ are the only subgroups of $[\mathbb{Z}, +]$.

Isomorphic Groups

Suppose that $[S, \cdot]$ and $[T, +]$ are isomorphic groups; what would this mean? From the discussion of isomorphism in Section 8.1, isomorphic structures are the same except for relabeling. There must be a bijection from S to T that accomplishes the relabeling. This bijection must also preserve the effects of the binary operation; that is, it must be true that "operate and map" yields the same result as "map and operate." The following definition is more precise.

DEFINITION GROUP ISOMORPHISM

Let $[S, \cdot]$ and $[T, +]$ be groups. A mapping $f: S \to T$ is an **isomorphism** from $[S, \cdot]$ to $[T, +]$ if

1. the function f is a bijection.
2. for all $x, y \in S, f(x \cdot y) = f(x) + f(y)$.

Property (2) is expressed by saying that f is a **homomorphism**.

PRACTICE 27 Illustrate the homomorphism property of the definition of group isomorphism by a commutative diagram. ■

If isomorphic groups are really the same except for the relabeling accomplished by the bijection, then we would expect that the identity of one group maps

to the identity of the other, that inverses map to inverses, and that if one group is commutative, so is the other. Indeed, we can prove that these expectations are correct. (The proofs do not make use of the one-to-one property of the isomorphism, so an onto homomorphism also maps the identity to the identity, inverses to inverses, and preserves commutativity.)

Suppose, then, that f is an isomorphism from the group $[S, \cdot]$ to the group $[T, +]$ and that i_S and i_T are the identities in the respective groups. Under the function f, i_S maps to an element $f(i_S)$ in T. Let t be any element in T. Then, because f is an onto function, $t = f(s)$ for some $s \in S$. It follows that

$$\begin{aligned} f(i_S) + t &= f(i_S) + f(s) \\ &= f(i_S \cdot s) && \text{(because } f \text{ is a homomorphism)} \\ &= f(s) && \text{(because } i_S \text{ is the identity in } S\text{)} \\ &= t \end{aligned}$$

Therefore

$$f(i_S) + t = t$$

Similarly,

$$t + f(i_S) = t$$

The element $f(i_S)$ acts like an identity element in $[T, +]$, and because the identity is unique, $f(i_S) = i_T$.

PRACTICE 28 Prove that if f is an isomorphism from the group $[S, \cdot]$ to the group $T, +]$, then for any $s \in S, f(s^{-1}) = -f(s)$ (inverses map to inverses). (*Hint*: Show that $f(s^{-1})$ acts like the inverse of $f(s)$.) ■

PRACTICE 29 Prove that if f is an isomorphism from the commutative group $[S, \cdot]$ to the group $[T, +]$, then $[T, +]$ is a commutative group. ■

EXAMPLE 14 $[\mathbb{R}^+, \cdot]$ and $[\mathbb{R}, +]$ are both groups. Let b be a positive real number, $b \neq 1$, and let f be the function from $\mathbb{R}^+$ to $\mathbb{R}$ defined by

$$f(x) = \log_b x$$

Then f is an isomorphism. To prove it, we must show that f is a bijection (one-to-one and onto) and that f is a homomorphism (preserves the operation). We can show that f is onto: For $r \in \mathbb{R}$, $b^r \in \mathbb{R}^+$ and $f(b^r) = \log_b b^r = r$. Also, f is one-to-one: If $f(x_1) = f(x_2)$, then $\log_b x_1 = \log_b x_2$. Let $p = \log_b x_1 = \log_b x_2$. Then $b^p = x_1$ and $b^p = x_2$, so $x_1 = x_2$. Finally f is a homomorphism: For $x_1, x_2 \in \mathbb{R}^+, f(x_1 \cdot x_2) = \log_b(x_1 \cdot x_2) = \log_b x_1 + \log_b x_2 = f(x_1) + f(x_2)$. Note that $\log_b 1 = 0$, so f maps 1, the identity of $[\mathbb{R}^+, \cdot]$ to 0, the identity of $[\mathbb{R}, +]$.

Also note that

$$\log_b(1/x) = \log_b 1 - \log_b x = 0 - \log_b x = -\log_b x = -f(x)$$

so f maps the inverse of x in $[\mathbb{R}^+, \cdot]$ to the inverse of $f(x)$ in $[\mathbb{R}, +]$. Finally, both groups are commutative.

Because the two groups in Example 14 are isomorphic, each is the mirror image of the other, and each can be used to simulate a computation in the other. Suppose, for example, that $b = 2$. Then $[\mathbb{R}, +]$ can be used to simulate the computation $64 \cdot 512$ in $[\mathbb{R}^+, \cdot]$. First, map from $\mathbb{R}^+$ to $\mathbb{R}$:

$$f(64) = \log_2 64 = 6$$
$$f(512) = \log_2 512 = 9$$

Now in $[\mathbb{R}, +]$ perform the computation

$$6 + 9 = 15$$

Finally, use f^{-1} to map back to $\mathbb{R}^+$:

$$f^{-1}(15) = 2^{15} = 32{,}768$$

(In the age BC—before calculators and computers—large numbers were multiplied by using tables of common logarithms, where $b = 10$, to convert a multiplication problem to an addition problem, as addition is less prone to human error.) Either of two isomorphic groups can always simulate computations in the other, just as in Example 14.

EXAMPLE 15 Consider the two groups $[S, \cdot]$ and $[T, +]$ as defined by the following tables:

$\cdot$	2	5	9
2	9	2	5
5	2	5	9
9	5	9	2

$+$	0	1	4
0	0	1	4
1	1	4	0
4	4	0	1

Both are groups of order 3, so an isomorphism is certainly possible. If f is to be an isomorphism, it must map i_S to i_T. Looking at the operation tables, $i_S = 5$ and $i_T = 0$, so let $f(5) = 0$. As a guess, let $f(2) = 1$ and $f(9) = 4$. Now let's reorganize the $[T, +]$ table:

$+$	1	0	4
1	4	1	0
0	1	0	4
4	0	4	1

This table contains exactly the same data that were in the original T table; the rows and columns have just been shuffled. Written in this form, it's clear that the T table is just a relabeling (using f) of the S table, so f is indeed an isomorphism. Note that inverses map to inverses:

$$f(2^{-1}) = f(9) = 4 \text{ and } -f(2) = -1 = 4$$
$$f(9^{-1}) = f(2) = 1 \text{ and } -f(9) = -4 = 1$$

And we can simulate the computation $9 \cdot 2 = 5$ in S by mapping to T, applying the $+$ operation, and mapping back to S:

$$\begin{aligned} f(9) &= 4, f(2) = 1 \\ 4 + 1 &= 0 \\ f^{-1}(0) &= 5, \text{ which is } 9 \cdot 2 \end{aligned}$$

EXAMPLE 16 Let f: $M_2(\mathbb{Z}) \to M_2(\mathbb{Z})$ be given by

$$f\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = \begin{bmatrix} a & c \\ b & d \end{bmatrix}$$

To show that f is one-to-one, let

$$f\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = f\left(\begin{bmatrix} e & f \\ g & h \end{bmatrix}\right)$$

Then

$$\begin{bmatrix} a & c \\ b & d \end{bmatrix} = \begin{bmatrix} e & g \\ f & h \end{bmatrix}$$

so $a = e$, $c = g$, $b = f$, and $d = h$, or

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

To show that f is onto, let

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \in M_2(\mathbb{Z})$$

Then

$$\begin{bmatrix} a & c \\ b & d \end{bmatrix} \in M_2(\mathbb{Z}) \quad \text{and} \quad f\left(\begin{bmatrix} a & c \\ b & d \end{bmatrix}\right) = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Also, f is a homomorphism from $[M_2(\mathbb{Z}), +]$ to $[M_2(\mathbb{Z}), +]$ because

$$f\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix} + \begin{bmatrix} e & f \\ g & h \end{bmatrix}\right) = f\left(\begin{bmatrix} a+e & b+f \\ c+g & d+h \end{bmatrix}\right) = \begin{bmatrix} a+e & c+g \\ b+f & d+h \end{bmatrix}$$
$$= \begin{bmatrix} a & c \\ b & d \end{bmatrix} + \begin{bmatrix} e & g \\ f & h \end{bmatrix} = f\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) + f\left(\begin{bmatrix} e & f \\ g & h \end{bmatrix}\right)$$

The function f is therefore an isomorphism from $[M_2(\mathbb{Z}), +]$ to $[M_2(\mathbb{Z}), +]$. ●

PRACTICE 30 Let $5\mathbb{Z} = \{5z \mid z \in \mathbb{Z}\}$. Then $[5\mathbb{Z}, +]$ is a group. Show that $f\colon \mathbb{Z} \to 5\mathbb{Z}$ given by $f(x) = 5x$ is an isomorphism from $[\mathbb{Z}, +]$ to $[5\mathbb{Z}, +]$. ■

If f is an isomorphism from $[S, \cdot]$ to $[T, +]$, then f^{-1} exists and is a bijection. Further, f^{-1} is also a homomorphism, this time from T to S. To see this, let t_1 and t_2 belong to T and consider $f^{-1}(t_1 + t_2)$. Because $t_1, t_2 \in T$ and f is onto, $t_1 = f(s_1)$ and $t_2 = f(s_2)$ for some s_1 and s_2 in S. Thus,

$$\begin{aligned} f^{-1}(t_1 + t_2) &= f^{-1}(f(s_1) + f(s_2)) \\ &= f^{-1}(f(s_1 \cdot s_2)) \\ &= (f^{-1} \circ f)(s_1 \cdot s_2) \\ &= s_1 \cdot s_2 \\ &= f^{-1}(t_1) \cdot f^{-1}(t_2) \end{aligned}$$

Therefore we can speak of S and T as being simply isomorphic, denoted by $S \simeq T$, without having to specify that the isomorphism is from S to T or vice versa.

Checking whether a given function is an isomorphism from S to T, as in Practice 30, is not hard. Deciding whether S and T are isomorphic may be harder. To prove that they are isomorphic, we must produce a function. To prove that they are not isomorphic, we must show that no such function exists. Since we can't try all possible functions, we use ideas such as the following: There is no one-to-one correspondence between S and T, S is commutative but T is not, and so on.

We have noted that isomorphic groups are alike except for relabeling and that each can be used to simulate the computations in the other. Isomorphism of groups is really an equivalence relation, as Practice 31 shows; thus isomorphic groups belong to the same equivalence class. Thinking of isomorphic groups as "alike except for labeling" is consistent with the idea that elements in an equivalence class represent different names for the same thing.

PRACTICE 31

a. Let $f\colon S \to T$ be an isomorphism from the group $[S, \cdot]$ to the group $[T, +]$ and $g\colon T \to U$ be an isomorphism from $[T, +]$ to the group $[U, *]$. Show that $g \circ f$ is an isomorphism from S to U.

b. Let $\mathcal{T}$ be a collection of groups and define a binary relation ρ on $\mathcal{T}$ by $S \rho T \leftrightarrow S \simeq T$. Show that ρ is an equivalence relation on $\mathcal{T}$. ■

We will finish this section by looking at some equivalence classes of groups under isomorphism. Often we pick out one member of an equivalence class and note that it is the typical member of that class and that all other groups in the class look just like it (with different names).

A result concerning the nature of very small groups follows immediately from Exercise 24 at the end of this section.

THEOREM **ON SMALL GROUPS**

Every group of order 2 is isomorphic to the group whose group table is

$\cdot$	1	a
1	1	a
a	a	1

Every group of order 3 is isomorphic to the group whose group table is

$\cdot$	1	a	b
1	1	a	b
a	a	b	1
b	b	1	a

Every group of order 4 is isomorphic to one of the two groups whose group tables are

$\cdot$	1	a	b	c
1	1	a	b	c
a	a	1	c	b
b	b	c	1	a
c	c	b	a	1

$\cdot$	1	a	b	c
1	1	a	b	c
a	a	b	c	1
b	b	c	1	a
c	c	1	a	b

We can also prove that any group is essentially a permutation group. Suppose $[G, \cdot]$ is a group. We want to establish an isomorphism from G to a permutation group; each element g of G must be associated with a permutation α_g on some set. In fact, the set will be G itself; for any $x \in G$, we define $\alpha_g(x)$ to be $g \cdot x$. We must show that $\{\alpha_g \mid g \in G\}$ forms a permutation group and that this permutation group is isomorphic to G. First we need to show that for any $g \in G$, α_g is indeed a permutation on G. From the definition $\alpha_g(x) = g \cdot x$, it is clear that α_g: $G \to G$, but it must be shown that α_g is a bijection.

PRACTICE 32 Show that α_g as just defined is a permutation (bijection) on G. ■

Now we consider $P = \{\alpha_g \mid g \in G\}$ and show that P is a group under function composition. P is nonempty because G is nonempty, and associativity always holds for function composition. We must show that P is closed and has an identity and that each $\alpha_g \in P$ has an inverse in P. To show closure, let α_g and $\alpha_h \in P$. For any $x \in G$, $(\alpha_g \circ \alpha_h)(x) = \alpha_g(\alpha_h(x)) = \alpha_g(h \cdot x) = g \cdot (h \cdot x) = (g \cdot h) \cdot x$. Thus, $\alpha_g \circ \alpha_h = \alpha_{g \cdot h}$ and $\alpha_{g \cdot h} \in P$.

PRACTICE 33

a. Let 1 denote the identity of G. Show that α_1 is an identity for P under function composition.
b. For $\alpha_g \in P, \alpha_{g^{-1}} \in P$; show that $\alpha_{g^{-1}} = (\alpha_g)^{-1}$. ■

We now know that $[P, \circ]$ is a permutation group, and it only remains to show that the function $f\colon G \to P$ given by $f(g) = \alpha_g$ is an isomorphism. Clearly, f is an onto function.

PRACTICE 34 Show that $f\colon G \to P$ defined by $f(g) = \alpha_g$ is

a. one-to-one.
b. a homomorphism. ■

We have now proved the following theorem, first stated and proved by the English mathematician Arthur Cayley in the mid-1800s.

THEOREM **CAYLEY'S THEOREM**
Every group is isomorphic to a permutation group.

SECTION 9.1 REVIEW

TECHNIQUES

- Test whether a given set and operation have the properties necessary to form a semigroup, monoid, or group structure.
- Test whether a given subset of a group is a subgroup.
- Test whether a given function from one group to another is an isomorphism.
- Decide whether two groups are isomorphic.

MAIN IDEAS

- Many elementary arithmetic systems are instances of a semigroup, monoid, or group structure.
- In any group structure, the identity and inverse elements are unique, cancellation laws hold, and linear equations are solvable; these and other properties follow from the definitions involved.
- A subset of a group may itself be a group under the inherited operation.
- The order of a subgroup of a finite group divides the order of the group.
- The only subgroups of the group $[\mathbb{Z}, +]$ are of the form $[n\mathbb{Z}, +]$, where $n\mathbb{Z}$ is the set of all integral multiples of a fixed $n \in \mathbb{N}$.
- If f is an isomorphism from one group to another, f maps the identity to the identity and inverses to inverses, and it preserves commutativity.
- If S and T are isomorphic groups, they are identical except for relabeling, and each simulates any computation in the other.
- Isomorphism is an equivalence relation on groups.
- To within an isomorphism, there is only one group of order 2, one group of order 3, and two groups of order 4.
- Every group is essentially a permutation group.

EXERCISES 9.1

1. a. A binary operation $\cdot$ is defined on the set $\{a, b, c, d\}$ by the table on the left. Is $\cdot$ commutative? Is $\cdot$ associative?

   b. Let $S = \{p, q, r, s\}$. An associative binary operation $\cdot$ is partly defined on S by the table on the right. Complete the table to preserve associativity. Is $\cdot$ commutative?

·	a	b	c	d
a	a	c	d	a
b	b	c	a	d
c	c	a	b	d
d	d	b	a	c

·	p	q	r	s
p	p	q	r	s
q	q	r	s	p
r			p	
s	s		q	r

2. Show that matrix multiplication on $M_2(\mathbb{Z})$ is associative.
3. Each of the following cases defines a binary operation, denoted by $\cdot$, on a given set. Which are associative? Which are commutative?
   a. On $\mathbb{Z}$: $x \cdot y = \begin{cases} x & \text{if } x \text{ is even} \\ x + 1 & \text{if } x \text{ is odd} \end{cases}$
   b. On $\mathbb{N}$: $x \cdot y = (x + y)^2$
   c. On $\mathbb{R}^+$: $x \cdot y = x^4$
   d. On $\mathbb{Q}$: $x \cdot y = xy/2$
   e. On $\mathbb{R}^+$: $x \cdot y = 1/(x + y)$
4. Define binary operations on the set $\mathbb{N}$ that are
   a. commutative but not associative.
   b. associative but not commutative.
   c. neither associative nor commutative.
   d. both associative and commutative.

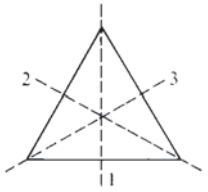
For Exercises 5–7, determine whether the structures $[S, \cdot]$ are semigroups, monoids, groups, or none of these. Name the identity element in any monoid or group structure.

5. a. $S = \mathbb{N}; x \cdot y = \min(x, y)$
   b. $S = \mathbb{R}; x \cdot y = (x + y)^2$
   c. $S = \{a\sqrt{2} \mid a \in \mathbb{N}\}; \cdot$ = multiplication
   d. $S = \{a + b\sqrt{2} \mid a, b \in \mathbb{Z}\}: \cdot$ = multiplication
   e. $S = \{a + b\sqrt{2} \mid a, b \in \mathbb{Q}, a \text{ and } b \text{ not both } 0\}; \cdot$ = multiplication
   f. $S = \{1, -1, i, -i\}; \cdot$ = multiplication (where $i^2 = -1$)
6. a. $S = \{1, 2, 4\}; \cdot = \cdot_6$
   b. $S = \{1, 2, 3, 5, 6, 10, 15, 30\}; x \cdot y =$ least common multiple of x and y
   c. $S = \mathbb{N} \times \mathbb{N}; (x_1, y_1) \cdot (x_2, y_2) = (x_1, y_2)$
   d. $S = \mathbb{N} \times \mathbb{N}; (x_1, y_1) \cdot (x_2, y_2) = (x_1 + x_2, y_1y_2)$
   e. $S =$ set of even integers; $\cdot$ = addition
   f. $S =$ set of odd integers; $\cdot$ = addition
7. a. $S =$ set of all polynomials in $\mathbb{R}[x]$ of degree ≤ 3; $\cdot$ = polynomial addition
   b. $S =$ set of all polynomials in $\mathbb{R}[x]$ of degree ≤ 3; $\cdot$ = polynomial multiplication
   c. $S = \left\{ \begin{bmatrix} 1 & z \\ 0 & 1 \end{bmatrix} \middle| z \in \mathbb{Z} \right\}; \cdot$ = matrix multiplication
   d. $S = \{1, 2, 3, 4\}; \cdot = \cdot_5$

e. $S = \mathbb{R} - \{-1\}; x \cdot y = x + y + xy$

f. $S = \{f \mid f: \mathbb{N} \to \mathbb{N}\}; \cdot$ = function addition, that is, $(f + g)(x) = f(x) + g(x)$

8. Let $A = \{1, 2\}$.
   a. Describe the elements and write the table for the semigroup of transformations on A.
   b. Describe the elements and write the table for the group of permutations on A.
9. Given an equilateral triangle, six permutations can be performed on the triangle that will leave its image in the plane unchanged. Three of these permutations are clockwise rotations in the plane of 120°, 240°, and 360° about the center of the triangle; these permutations are denoted R_1, R_2, and R_3, respectively. The triangle can also be flipped about any of the axes 1, 2, and 3 (see the accompanying figure); these permutations are denoted F_1, F_2, and F_3, respectively. During any of these permutations, the axes remain fixed in the plane. Composition of permutations is a binary operation on the set D_3 of all six permutations. For example, $F_3 \circ R_2 = F_2$. The set D_3 under composition is a group, called the *group of symmetries of an equilateral triangle*. Complete the group table below for $[D_3, \circ]$. What is an identity element in $[D_3, \circ]$? What is an inverse element for F_1? For R_2?



$\circ$	R_1	R_2	R_3	F_1	F_2	F_3
R_1						
R_2						
R_3						
F_1						
F_2						
F_3		F_2				

10. The set S_3, the symmetric group of degree 3, is isomorphic to D_3, the group of symmetries of an equilateral triangle (see Exercise 9). Find a bijection from the elements of S_3 to the elements of D_3 that preserves the operation. (*Hint*: R_1 of D_3 may be considered a permutation in S_3 sending 1 to 2, 2 to 3, and 3 to 1.)
11. In each case, decide whether the structure on the left is a subgroup of the group on the right. If not, why not? (Note that here S^* denotes $S - \{0\}$.)
   a. $[\mathbb{Z}_5^*, \cdot_5]; [\mathbb{Z}_5, +_5]$
   b. $[P, +]; [\mathbb{R}[x], +]$ where P is the set of all polynomials in x over $\mathbb{R}$ of degree ≥ 3
   c. $[\mathbb{Z}^*, \cdot]; [\mathbb{Q}^*, \cdot]$
   d. $[K, +]; [\mathbb{R}[x], +]$ where K is the set of all polynomials in x over $\mathbb{R}$ of degree $\leq k$ for some fixed k
12. In each case, decide whether the structure on the left is a subgroup of the group on the right. If not, why not?
   a. $[A, \circ]; [S, \circ]$ where S is the set of all bijections on $\mathbb{N}$ and A is the set of all bijections on $\mathbb{N}$ mapping 3 to 3
   b. $[\mathbb{Z}, +]; [M_2(\mathbb{Z}), +]$
   c. $[\{0, 3, 6\}, +_8]; [\mathbb{Z}_8, +_8]$
   d. $[A, +_2]; [Z_2^5, +_2]$ where $A = \{00000, 01111, 10101, 11010\}$
13. Find all the distinct subgroups of $[\mathbb{Z}_{12}, +_{12}]$.
14. a. Show that the subset

$$\alpha_1 = i \qquad \alpha_3 = (1, 4) \circ (2, 3)$$
$$\alpha_2 = (1, 2) \circ (3, 4) \qquad \alpha_4 = (1, 3) \circ (2, 4)$$

forms a subgroup of the symmetric group S_4.

b. Show that the subset

$$\begin{aligned} \alpha_1 &= i & \alpha_5 &= (1,2)\circ(3,4) \\ \alpha_2 &= (1,2,3,4) & \alpha_6 &= (1,4)\circ(2,3) \\ \alpha_3 &= (1,3)\circ(2,4) & \alpha_7 &= (2,4) \\ \alpha_4 &= (1,4,3,2) & \alpha_8 &= (1,3) \end{aligned}$$

forms a subgroup of the symmetric group S_4.

15. Find the elements of the alternating group A_4.
16. Let $A = \{p, q, r\}$. Then $[A^*, \cdot]$ is the free monoid generated by A.
    a. What is ppqrp · qprr ?
    b. Let $B =$ the set of all strings over A with an even number of q's. Then $B \subseteq A$. Prove that $[B, \cdot]$ is also a monoid.
17. In each case, decide whether the given function is a homomorphism from the group on the left to the one on the right. Are any of the homomorphisms also isomorphisms?
    a. $[\mathbb{Z}, +], [\mathbb{Z}, +]; f(x) = 2$
    b. $[\mathbb{R}, +], [\mathbb{R}, +]; f(x) = |x|$
    c. $[\mathbb{R}^*, \cdot], [\mathbb{R}^*, \cdot]$ (where $\mathbb{R}^*$ denotes the set of nonzero real numbers); $f(x) = |x|$
18. In each case, decide whether the given function is a homomorphism from the group on the left to the one on the right. Are any of the homomorphisms also isomorphisms?
    a. $[\mathbb{R}[x], +], [\mathbb{R}, +]; f(a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0) = a_n + a_{n-1} + \cdots + a_0$
    b. $[S_3, \circ], [\mathbb{Z}_2, +_2]$; $f(\alpha) = \begin{cases} 1 \text{ if } \alpha \text{ is an even permutation} \\ 0 \text{ if } \alpha \text{ is an odd permutation} \end{cases}$
    c. $[\mathbb{Z} \times \mathbb{Z}, +]$ where $+$ denotes componentwise addition, $[\mathbb{Z}, +]; f(x, y) = x + 2y$
19. In each case, decide whether the given groups are isomorphic. If they are, produce an isomorphism function. If they are not, give a reason why they are not.
    a. $[\mathbb{Z}, +], [12\mathbb{Z}, +]$ (where $12\mathbb{Z} = \{12z | z \in \mathbb{Z}\}$)
    b. $[\mathbb{Z}_5, +_5], [5\mathbb{Z}, +]$
    c. $[5\mathbb{Z}, +], [12\mathbb{Z}, +]$
    d. $[S_3, \circ], [\mathbb{Z}_6, +_6]$
20. In each case, decide whether the given groups are isomorphic. If they are, produce an isomorphism function. If they are not, give a reason why they are not.
    a. $[\{a_1 x + a_0 | a_1, a_0 \in \mathbb{R}\}, +], [\mathbb{C}, +]$
    b. $[\mathbb{Z}_6, +_6], [S_6, \circ]$
    c. $[\mathbb{Z}_2, +_2], [S_2, \circ]$
    d. $[Z_2^3, +_2], [\mathbb{Z}_8, +_8]$
21. Let $M_2^0(\mathbb{Z})$ be the set of all 2×2 matrices of the form

$$\begin{bmatrix} 1 & z \\ 0 & 1 \end{bmatrix}$$

where $z \in \mathbb{Z}$.

a. Show that $[M_2^0(\mathbb{Z}), \cdot]$ is a group, where · denotes matrix multiplication.

b. Let a function f: $M_2^0(\mathbb{Z}) \to \mathbb{Z}$ be defined by

$$f\left(\begin{bmatrix} 1 & z \\ 0 & 1 \end{bmatrix}\right) = z$$

Prove that f is an isomorphism from $[M_2^0(\mathbb{Z}), \cdot]$ to $[\mathbb{Z}, +]$

c. Use $[\mathbb{Z}, +]$ to simulate the computation

$$\begin{bmatrix} 1 & 7 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & -3 \\ 0 & 1 \end{bmatrix}$$

in $[M_2^0(\mathbb{Z}), \cdot]$.

d. Use $[M_2^0(\mathbb{Z}), \cdot]$ to simulate the computation $2 + 3$ in $[\mathbb{Z}, +]$.

22. a. Let $S = \{1, -1\}$. Show that $[S, \cdot]$ is a group where $\cdot$ denotes ordinary integer multiplication.

b. Let f be the function from the group $[S_n, \circ]$ to the group $[S, \cdot]$ given by

$$f(\alpha) = \begin{cases} 1 \text{ if } \alpha \text{ is even} \\ -1 \text{ if } \alpha \text{ is odd} \end{cases}$$

Prove that f is a homomorphism.

23. In any group $[G, \cdot]$, show that

a. $i^{-1} = i$

b. $(x^{-1})^{-1} = x$ for any $x \in G$

24. a. Show that any group of order 2 is commutative by constructing a group table on the set $\{1, a\}$ with 1 as the identity.

b. Show that any group of order 3 is commutative by constructing a group table on the set $\{1, a, b\}$ with 1 as the identity. (You may assume associativity.)

c. Show that any group of order 4 is commutative by constructing a group table on the set $\{1, a, b, c\}$ with 1 as the identity. (You may assume associativity.) There will be four such tables, but three of them are isomorphic because the elements have simply been relabeled from one to the other. Find these three groups and indicate the relabeling. Thus, there are two essentially different groups of order 4, and both of these are commutative.

25. Let $[G, \cdot]$ be a group and let $x, y \in G$. Define a relation ρ on G by $x \rho y \leftrightarrow g \cdot x \cdot g^{-1} = y$ for some $g \in G$.

a. Prove that ρ is an equivalence relation on G.

b. Prove that for each $x \in G$, $[x] = \{x\}$ if and only if G is commutative.

26. For x a member of a group $[G, \cdot]$, we can define x^n for any positive integer n by $x^1 = x$, $x^2 = x \cdot x$, and $x^n = x^{n-1} \cdot x$ for $n > 2$. Prove that in a finite group $[G, \cdot]$, for each $x \in G$ there is a positive integer k such that $x^k = i$.

27. Let $[S, \cdot]$ be a semigroup. An element $i_L \in S$ is a *left identity* element if for all $x \in S$, $i_L \cdot x = x$. An element $i_R \in S$ is a *right identity* element if for all $x \in S$, $x \cdot i_R = x$.

a. Prove that if a semigroup $[S, \cdot]$ has both a left identity element and a right identity element, then $[S, \cdot]$ is a monoid.

b. Give an example of a finite semigroup with two left identities and no right identity.

c. Give an example of a finite semigroup with two right identities and no left identity.

d. Give an example of a semigroup with neither a right nor a left identity.

28. Let $[S, \cdot]$ be a monoid with identity i, and let $x \in S$. An element x_L^{-1} in S is a *left inverse* of x if $x_L^{-1} \cdot x = i$. An element x_R^{-1} in S is a *right inverse* of x if $x \cdot x_R^{-1} = i$.
    a. Prove that if every element in a monoid $[S, \cdot]$ has both a left inverse and a right inverse, then $[S, \cdot]$ is a group.
    b. Let S be the set of all functions f such that $f: \mathbb{N} \to \mathbb{N}$. Then S under function composition is a monoid. Define a function $f \in S$ by $f(x) = 2x, x \in \mathbb{N}$. Then define a function $g \in S$ by

$$g(x) = \begin{cases} x/2 & \text{if } x \in \mathbb{N}, x \text{ even} \\ 1 & \text{if } x \in \mathbb{N}, x \text{ odd} \end{cases}$$

    Prove that g is a left inverse for f. Also prove that f has no right inverse.
29. Let $[S, \cdot]$ be a semigroup having a left identity i_L (see Exercise 27) and the property that for every $x \in S$, x has a left inverse y such that $y \cdot x = i_L$. Prove that $[S, \cdot]$ is a group. (*Hint*: y also has a left inverse in S.)
30. An element of a semigroup $[S, \cdot]$ is idempotent if $x \cdot x = x$. Prove that a group contains exactly one idempotent element.
31. Prove that if $[S, \cdot]$ is a semigroup in which the linear equations $a \cdot x = b$ and $x \cdot a = b$ are solvable for any $a, b \in S$, then $[S, \cdot]$ is a group. (*Hint*: Use Exercise 29.)
32. Prove that a finite semigroup that satisfies the left and right cancellation laws is a group. (*Hint*: Use Exercise 29.)
33. Prove that a group $[G, \cdot]$ is commutative if and only if $(x \cdot y)^2 = x^2 \cdot y^2$ for each $x, y \in G$.
34. Prove that a group $[G, \cdot]$ in which $x \cdot x = i$ for each $x \in G$ is commutative.
35. Let $[G, \cdot]$ be a commutative group with identity i. For a fixed positive integer k, let $B_k = \{x | x \in G, x^k = i\}$. Prove that $[B_k, \cdot]$ is a subgroup of $[G, \cdot]$.
36. Let $[G, \cdot]$ be a commutative group with subgroups $[S, \cdot]$ and $[T, \cdot]$. Let $ST = \{s \cdot t | s \in S, t \in T\}$. Prove that $[ST, \cdot]$ is a subgroup of $[G, \cdot]$.
37. a. Let $[G, \cdot]$ be a group and let $[S, \cdot]$ and $[T, \cdot]$ be subgroups of $[G, \cdot]$. Prove that $[S \cap T, \cdot]$ is a subgroup of $[G, \cdot]$.
    b. Will $[S \cup T, \cdot]$ be a subgroup of $[G, \cdot]$? Prove or give a counterexample.
38. For any group $[G, \cdot]$, the *center* of the group is $A = \{x \in G | x \cdot g = g \cdot x \text{ for all } g \in G\}$.
    a. Prove that $[A, \cdot]$ is a subgroup of $[G, \cdot]$
    b. Find the center of the group of symmetries of an equilateral triangle, $[D_3, \circ]$ (see Exercise 9).
    c. Prove that G is commutative if and only if $G = A$.
    d. Let x and y be members of G with $x \cdot y^{-1} \in A$. Prove that $x \cdot y = y \cdot x$.
39. a. Let S_A denote the group of permutations on a set A, and let a be a specific element of A. Prove that the set H_a of all permutations in S_A that map a to a forms a subgroup of S_A.
    b. If A has n elements, what is $|H_a|$?
40. a. Let $[G, \cdot]$ be a group and $A \subseteq G, A \neq \varnothing$. Prove that $[A, \cdot]$ is a subgroup of $[G, \cdot]$ if for each $x, y \in A, x \cdot y^{-1} \in A$. This subgroup test is sometimes more convenient to use than the theorem on subgroups.
    b. Use the test of part (a) to work Exercise 35.
41. a. Let $[G, \cdot]$ be any group with identity i. For a fixed $a \in G, a^0$ denotes i and a^{-n} means $(a^n)^{-1}$. Let $A = \{a^z | z \in \mathbb{Z}\}$. Prove that $[A, \cdot]$ is a subgroup of G. (*Hint*: Use Exercise 40.)
    b. The group $[G, \cdot]$ is a *cyclic group* if for some $a \in G, A = \{a^z | z \in \mathbb{Z}\}$ is the entire group G. In this case, a is a *generator* of $[G, \cdot]$. For example, 1 is a generator of the group $[\mathbb{Z}, +]$; remember that the operation is addition. Thus, $1^0 = 0, 1^1 = 1, 1^2 = 1 + 1 = 2, 1^3 = 1 + 1 + 1 = 3 \ldots .; 1^{-1} = (1)^{-1} = -1; 1^{-2} = (1^2)^{-1} = -2; 1^{-3} = (1^3)^{-1} = -3, \ldots$. Every integer can be written as an integral

"power" of 1, and $[\mathbb{Z}, +]$ is cyclic with generator 1. Prove that the group $[\mathbb{Z}_7, +_7]$ is cyclic with generator 2.

c. Prove that 5 is also a generator of the cyclic group $[\mathbb{Z}_7, +_7]$.

d. Prove that 3 is a generator of the cyclic group $[\mathbb{Z}_4, +_4]$.

42. Let $[G, \cdot]$ be a cyclic group with generator a (see Exercise 41). Show that G is commutative.

43. a. Let $[S, \cdot]$ be a semigroup. An isomorphism from S to S is called an *automorphism* on S. Let Aut(S) be the set of all automorphisms on S, and prove that Aut(S) is a group under function composition.

b. For the group $[\mathbb{Z}_4, +_4]$, find the set of automorphisms and show its group table under $\circ$.

44. Let $[G, \cdot]$ be a commutative group with identity i. Prove that the function f: $G \to G$ given by $f(x) = x^{-1}$ is an isomorphism.

45. Let f be a homomorphism from a group G onto a group H. Show that f is an isomorphism if and only if the only element of G that is mapped to the identity of H is the identity of G.

46. Let $[G, \cdot]$ be a group and g a fixed element of G. Define f: $G \to G$ by $f(x) = g \cdot x \cdot g^{-1}$ for any $x \in G$. Prove that f is an isomorphism from G to G.

47. a. Consider the equivalence relation on the integers of congruence modulo n defined in Section 5.1. If $n = 5$, there are 5 equivalence classes:

$$[0] = \{\ldots, -10, -5, 0, 5, 10, \ldots\}$$
$$[1] = \{\ldots, -9, -4, 1, 6, 11, \ldots\}$$
$$[2] = \{\ldots, -8, -3, 2, 7, 12, \ldots\}$$
$$[3] = \{\ldots, -7, -2, 3, 8, 13, \ldots\}$$
$$[4] = \{\ldots, -6, -1, 4, 9, 14, \ldots\}$$

Let $E_5 = \{[0], [1], [2], [3], [4]\}$. An operation $+$ is defined on E_5 by

$$[x] + [y] = [x + y]$$

For example, $[2] + [4] = [2 + 4] = [6] = [1]$ (recall that an equivalence class can be named by any of its elements). Prove that $[E_5, +]$ is a commutative group.

b. $[\mathbb{Z}_5, +_5]$ is a commutative group with elements $\{0, 1, 2, 3, 4\}$ (see Example 7). Prove that $[\mathbb{Z}_5, +_5]$ is isomorphic to the group $[E_5, +]$.

c. Results a and b hold for any value of n. In the group E_{14}, what is the inverse of [10]? What is the preimage of [21] under the isomorphism from $\mathbb{Z}_{14}$ to E_{14}?

SECTION 9.2 CODING THEORY

Introduction

We talked about cryptographic codes (codes for secrecy) in Chapter 5. The codes we will talk about in this section are designed not to keep data secret but to cope with degraded data. Data can degrade over time in a storage device or can be corrupted over space, that is, during a transmission from one site to another over some medium. Bits are changed from 0 to 1 or vice versa through interference ("noise"), hardware failures, media damage, and so forth. The goal is to be able to detect, perhaps even to correct, such errors.